

分 类 号 TP309

密 级

U D C

编 号 10486

武汉大学

硕 士 学 位 论 文

基于同源的口令认证密钥协商
方法及系统

研 究 生 姓 名:

学 号: 2022202210131

指导教师姓名、职称:

专 业 名 称: 网络空间安全

研 究 方 向: 密钥协商

二〇二五年四月

Method and System of Isogeny-based Password-Authenticated Key Exchange based on shuffle algorithm

Candidate:

StudentNumber: 2022202210131

Supervisor:

Major: Cyberspace Security

Speciality: Cryptography



School of Cyber Science And Engineering
WUHAN UNIVERSITY

April, 2025

论文原创性声明

本人郑重声明：所呈交的学位论文，是本人在导师指导下，独立进行研究工作所取得的研究成果。除文中已经标明引用的内容外，本论文不包含任何其他个人或集体已经发表或撰写过的研究成果。对本文的研究做出贡献的个人和集体，均已在文中以明确方式标明。本声明的法律结果由本人承担。

学位论文作者（签名）：

年 月 日

摘 要

口令认证密钥交换 (Password-authenticated Key Exchange, PAKE) 是一种加密协议, 它允许参与者在低熵秘密上共享密钥, 而无需通过网络传输口令, 从而允许在不安全的通道上安全地传输数据。在 PAKE 中, 通信双方通过口令派生出长期密钥, 以安全地建立会话密钥, 从而防止重要信息的泄露或被盗。随着量子计算技术的快速发展, 传统依赖于一些数学难题 (如大数因子分解、离散对数) 的 PAKE 方案面临严峻挑战。因此, 学界和业界积极研究抗量子的 PAKE 协议。

在 CRYPTO 2022 上, Abdalla 等人将计算椭圆曲线二次扭曲的能力纳入扩展 Alamelu 等人 (ASIACRYPT 2020) 引入的群操作框架, 并提出了两种口令认证密钥交换协议。其中, 协议 X - GA - PAKE _{ℓ} 是一个可证明安全的基于同源的方案, 其口令长度为 ℓ 。该方案通过增加计算和通信开销来提高安全性, 各方需要选择 2ℓ 元素并执行 5ℓ 组操作。

本文首先设计了一种基于同源的单轮 PAKE 方案, 该方案的通信开销比 X - GA - PAKE _{ℓ} 少, 但仍然可抵抗离线字典攻击。在此基础上, 本文进一步设计了一种基于同源的非对称 PAKE 方案。本文的主要贡献如下:

1. 提出了一个基于同源的 PAKE 协议 R - GA - PAKE _{ℓ} , 通过将集合元素的数量减少一半, 显著降低了双方之间的通信开销。为实现这一点, 在生成群操作的中间结果后, 使用由中间结果序列的 Merkle 根和口令共同生成的种子, 对该中间结果进行随机置换。因此, 各方需要执行的组动作数量从 5ℓ 减少到 2ℓ , 但由于随机置换算法中使用了群操作中间结果的 Merkle 根和口令, 攻击者无法从收到的消息中选择目标消息进行发送, 因此该方案可以抵御离线字典攻击。
2. 提出了一个基于同源的非对称 PAKE 协议 aR - GA - PAKE _{ℓ} , 引入注册和登录阶段。注册阶段, 客户端基于口令生成加密存储文件供服务器使用; 登录阶段, 双方计算中间值代替口令进行后续协议交互。通过增加注册与登录阶段, 有效避免服务器存储口令带来的安全风险, 为安全高效的密钥协商提供了更可靠的保障。

此外, 本文对提出的两个方案进行了安全性分析。同时, 本文对第二个方案进行了具体实现, 在实验环境中部署并测试其实际表现。通过功能测试验证各模块的正确性, 并通过性能测试评估方案的计算开销和通信开销, 以衡量其在实际应用中的可行性与高效性。最后, 为了进一步验证方案的可行性, 本文在口令系统内进行了应用场景分析与系统模拟。综上所述, 本文通过方案设计、安全性分析、实验评估及应用模拟, 提出了基于同源的口令认证密钥协商方法及系统。

关键词: 口令认证密钥协商, CSIDH, 同源, 随机置换算法

ABSTRACT

Password-Authenticated Key Exchange (PAKE) is a cryptographic protocol that allows participants to share a key based on a low-entropy secret without transmitting the password over the network. This enables secure data transmission over an insecure channel. In PAKE, both communicating parties derive a long-term key from a password to securely establish a session key, preventing critical information from being leaked or stolen. With the rapid advancement of quantum computing technology, traditional PAKE schemes that rely on mathematical hardness assumptions, such as integer factorization and discrete logarithms, face severe challenges. Consequently, researchers and industry professionals are actively exploring quantum-resistant PAKE protocols.

At CRYPTO 2022, Abdalla et al. incorporated the ability to compute elliptic curve quadratic twists into the extended group action framework introduced by Alami et al. (ASIACRYPT 2020) and proposed two password-authenticated key exchange protocols. Among them, the X - GA - PAKE $_{\ell}$ protocol is a provably secure homomorphic-based scheme with a password length of ℓ . This scheme enhances security by increasing computational and communication costs, requiring each party to select 2ℓ elements and perform 5ℓ group operations.

This paper first designs a single-round PAKE scheme based on homomorphism, which reduces the communication overhead compared to X - GA - PAKE $_{\ell}$, while still resisting offline dictionary attacks. Building on this, we further propose a homomorphic-based asymmetric PAKE scheme. The main contributions of this paper are as follows:

1. We propose a homomorphic-based PAKE protocol R - GA - PAKE $_{\ell}$, which significantly reduces communication overhead by halving the number of set elements. To achieve this, after generating the intermediate results of the group operation, we apply a random permutation based on a seed derived from the Merkle root of the intermediate result sequence and the password. As a result, the number of group operations required by each party is reduced from 5ℓ to 2ℓ . Moreover, since the random permutation algorithm incorporates both the Merkle root of the intermediate results and the password, attackers cannot selectively resend specific messages based on received data, ensuring resistance to offline dictionary attacks.
2. We propose a homomorphic-based asymmetric PAKE protocol aR - GA - PAKE $_{\ell}$, introducing registration and authentication phases. In the registration phase, the client generates an encrypted credential file based on the password and submits it to the server for

storage. During the authentication phase, both parties compute intermediate values to replace the password in subsequent protocol interactions. By introducing these additional phases, our scheme effectively eliminates the security risks associated with server-side password storage, providing a more secure and efficient key exchange mechanism.

Additionally, this paper conducts a security analysis of the proposed schemes. The second scheme is also implemented and evaluated in an experimental environment, where we validate the correctness of each module through functional testing and measure computational and communication costs through performance evaluation. Finally, to further verify the feasibility of our schemes, we conduct application scenario analysis and system simulations within a password-based authentication system.

In conclusion, this paper presents a homomorphic-based PAKE method and system through protocol design, security analysis, experimental evaluation, and application-level simulations.

Key words: Password-authenticated key exchange; CSIDH; isogeny-based; shuffle algorithm

目 录

摘 要	I
ABSTRACT	III
1 绪论	1
1.1 研究背景与意义	1
1.2 国内外研究现状	3
1.2.1 口令认证密钥协商协议	3
1.2.2 基于同源的口令认证密钥协商协议	6
1.2.3 (强)非对称的口令认证密钥协商协议	7
1.3 主要研究内容	8
1.4 论文结构组织	9
2 预备知识	11
2.1 数学基础	11
2.2 密码学原语	13
2.3 计算与密码学基础	15
2.4 群操作	17
2.4.1 基础概念	18
2.4.2 基于同源的群操作	19
2.4.3 计算假设	20
3 基于随机同源置换的口令认证密钥协商方案	23
3.1 安全模型	23
3.2 方案设计	25
3.3 安全性分析与证明	27
3.4 方案对比	37
3.5 本章小结	38
4 基于同源的不对称口令认证密钥协商方案	39
4.1 安全模型	39
4.2 方案设计	42
4.3 安全性分析	44
4.3.1 模拟器设计	45

4.3.2	不可区分性证明	47
4.4	本章小结	48
5	基于同源的口令系统设计与实现	49
5.1	非对称口令认证密钥协商方案实现	49
5.1.1	测试环境	49
5.1.2	基本运算	50
5.1.3	aR-GA-PAKE _ℓ 方案	50
5.2	基于同源的口令系统模拟	55
5.3	本章小结	57
6	总结与展望	59
6.1	本文工作总结	59
6.2	未来工作展望	60
	参考文献	61
	攻硕期间取得的学术成果和参与的项目	67

图片索引

图 1.1	PAKE 分类示意图	4
图 1.2	PAKE 协议 X-GA-PAKE _{ℓ} 的流程图。	7
图 2.1	Knuth-Durstenfeld 算法流程图	17
图 2.2	Merkle 树的构建过程	18
图 3.1	R - GA - PAKE _{ℓ} 方案的整体框架	25
图 3.2	GetRoot 函数的 Merkle 根生成方式	26
图 3.3	R - GA - PAKE _{ℓ} 方案的协议流程。	27
图 3.4	用于证明定理 3.3.1 的游戏 G ₀₋₄ 。	29
图 3.5	定理 3.3.1 证明中敌手 B_1 针对 GA-StCDH 的构造。	31
图 3.6	用于证明定理 3.3.1 的游戏 G ₅ 。	32
图 3.7	用于证明定理 3.3.1 的游戏 G ₆ 。	33
图 3.8	定理 3.3.1 证明中敌手 B_2 针对 Sim-GA-StCDH 的构造。	35
图 4.1	aR - GA - PAKE _{ℓ} 方案的整体框架	42
图 4.2	aR - GA - PAKE _{ℓ} 方案的协议流程。	43
图 5.1	aR - GA - PAKE _{ℓ} 方案的实现框架	51
图 5.2	注册功能测试结果	51
图 5.3	登录成功测试结果	52
图 5.4	登录失败测试结果	52
图 5.5	随机置换与逆置换测试结果	53
图 5.6	Merkle 根生成测试结果	53
图 5.7	密钥协商成功测试结果	54
图 5.8	密钥协商失败测试结果	54
图 5.9	不同 ℓ 取值下密钥协商各模块运行时间	55
图 5.10	口令系统测试页面结果	56

表格索引

表 2.1	文中符号及相关描述	11
表 3.1	本章中符号及相关描述	23
表 3.2	证明中的游戏描述。	28
表 3.3	现有方案与 R-GA-PAKE _{ℓ} 的对比	37
表 4.1	本章中符号及相关描述	39
表 5.1	实验软硬件环境	49
表 5.2	超奇异椭圆曲线上的基本运算函数	50

1 绪论

1.1 研究背景与意义

随着互联网时代的到来和信息技术的快速发展，人们的生活和工作方式经历了深刻的变革。云计算、大数据、人工智能以及物联网等新兴技术的迅猛发展，极大地提高了信息存储、传输和处理的效率，使得信息的存储、传输和处理变得更加高效和便捷。然而，技术的进步不仅带来了便利，也伴随着复杂的数据安全和隐私保护挑战。特别是在多租户环境、大规模数据处理和量子计算技术不断演进的背景下，全球数据流量呈指数级增长，信息交换的频率和复杂性大幅提升，传统的安全机制正面临前所未有的威胁。大规模数据泄露、网络攻击、身份盗窃等安全事件频发，给个人隐私、企业机密和国家安全带来了巨大威胁。这些威胁迫使我们对于数据保护技术提出更高要求，需要采用更强大的加密算法、身份认证机制以及安全协议来应对日益复杂的攻击手段。

与此同时，数据保护法律和法规的不断完善也在加速推动企业和机构在数据存储、传输及访问控制方面采取更严格的安全措施。例如，欧盟的《通用数据保护条例》（GDPR）、美国的《加州消费者隐私法》（CCPA）、以及中国的《个人信息保护法》、《网络安全法》和《数据安全法》等法律的出台，要求企业和机构在数据处理过程中必须严格遵守合规性要求，以确保数据的机密性、完整性和可用性。为了应对这些法规要求，企业必须强化其信息安全策略，确保数据处理过程中的每个环节都能得到充分的保护。在现代网络通信环境中，数据传输的安全性尤为重要，网络攻击手段的不断演进使得数据泄露、身份冒用、中间人攻击（MitM）、离线字典攻击等安全威胁日益严峻。因此，如何确保网络通信安全，保障用户身份的可信性，以及防止敏感信息在传输过程中被窃取或篡改，已成为当前信息安全领域的重点研究方向之一。

作为解决数据传输安全问题的关键技术之一，认证密钥协商（Authenticated Key Exchange, AKE）在现代通信中占据了至关重要的地位。AKE 协议的核心功能是允许通信双方在不可信信道上安全地建立一个共享的秘密密钥，并同时确保双方的身份得到验证。AKE 协议广泛应用于互联网通信、虚拟专用网络（VPN）、电子邮件加密、无线通信安全以及区块链等领域，为数据传输的机密性和完整性提供基础保障。在 AKE 协议的众多变种中，口令认证密钥协商（Password-authenticated key exchange, PAKE）协议是一个重要的分支。PAKE 协议在设计时考虑了用户口令的脆弱性，旨在允许参与方在不通过网络传输口令的情况下，通过低熵秘密共享密钥，从而得以在不安全信道上安全地传输数据。PAKE 协议作为一种基于口令的认证密钥协商技术，在数据安全中的作用至关重要。它通过增强口令认证的安全性、保障数据传输的机密性和完整性、提升用户身份认证的安全性，显著提高了数据保护水平。在当前信息技术迅猛发展的背景下，网

络安全带来的威胁也日益增加，PAKE 协议为构建安全的网络环境提供了强有力的技术支撑。未来，随着信息保护需求的不断提升和技术的持续进步，PAKE 协议将在更多的应用场景中发挥重要作用。特别是在物联网、5G 通信、云计算和区块链等新兴技术的应用领域，PAKE 协议将继续推动数据安全技术的发展，确保用户隐私和敏感信息在传输过程中的安全性。通过不断优化 PAKE 协议的效率和安全性，未来的网络安全防护体系将能够应对更加复杂的安全威胁，保障数据通信的安全性和可靠性。

在 PAKE 协议中，口令作为协议参与方的长期密钥，在没有被泄露或窃取的情况下，用来建立会话密钥，也用于协议的一方认证另一方。PAKE 协议由于口令容易被记忆而受到学者们的广泛关注，设计安全的 PAKE 协议也就成为了一项具有重大现实意义的工作。通常来说，一个安全且实用的 PAKE 协议应满足：

1. 抗攻击能力：协议应能够抵御主动攻击、被动攻击和中间人攻击，确保在通信过程中不会遭到篡改或窃听。同时，它需要防止在线和离线字典攻击，避免攻击者通过猜测口令来破解密钥。
2. 安全性：即使用户使用的是低熵口令（弱口令），协议仍能保证密钥协商的安全性，并限制攻击者获取足够的信息来猜测口令，从而保护用户的隐私。
3. 隐私性：协议设计需确保口令不会直接泄露给通信方或存储在服务器端，防止口令泄露给攻击者，确保用户信息的安全性。
4. 通用性与可用性：PAKE 协议应能够适用于各种不同的应用场景，如远程身份验证、多设备同步以及物联网环境中的安全通信。

为了在协议设计之处就保证协议能够达到其最初的设计目标，目前普遍采用的方法是建立精确的通信模型，给出严格的安全性定义，形式化的证明协议的安全性。由于口令短小、低熵的特点，通信便捷的同时也导致 PAKE 协议容易遭受字典攻击。因此，设计 PAKE 协议时必须要考虑是否可以抵抗字典攻击。字典攻击主要分为在线字典攻击和离线字典攻击。对于在线字典攻击，攻击者用猜测的口令假冒一个通信方交互，如果交互失败，攻击者另选一个口令重新交互。为抵抗此类攻击，就需要限制攻击者重复尝试口令的次数。对于离线字典攻击，攻击者通过窃听记录几次会话，从而尝试排除大量的可能口令，进而猜测出正确口令。为抵抗此类攻击，就需要保证攻击者无法获取到用于验证口令对错的消息。

随着量子计算的快速发展，传统密码学算法迎来的新的挑战，而 PAKE 协议也不例外。量子计算机利用量子力学的特性，能够在某些计算任务上显著超越传统计算机的能力。量子计算机可以利用 Shor 算法在多项式时间内解决大数分解和离散对数问题，这对当前广泛使用的公钥加密算法构成重大威胁。RSA 算法和 ECC 算法等依赖于大数分解和离散对数问题的公钥算法，在量子计算机面前几乎不再安全。例如，2048 位的 RSA 密钥可以被量子计算机在合理时间内破解。为应对量子计算的安全挑战，学术界提出了后量子密码学（Post-Quantum Cryptography, PQC）。其中，同源密码学（Isogeny-based

Cryptography) 是密码学中的一个相对较新的领域, 利用椭圆曲线同源构建密码算法, 在量子计算环境下提供更高的安全性, 同时保持计算效率和通信开销在可接受范围内。

同源密码学作为后量子密码学的一部分, 正在受到越来越多的关注。通过研究基于同源的 PAKE 协议, 可以有效应对量子计算带来的挑战。研究基于同源的 PAKE 协议不仅能够提升 PAKE 协议在量子计算环境下的安全性, 还可以为未来的密码学标准化提供新的理论基础和技术支撑。近年来, NIST (美国国家标准与技术研究院) 等机构正在推动后量子密码学标准化进程, 而基于同源的 PAKE 协议的研究与实践应用, 将是未来密码学领域的一个重要发展方向。然而, 该方向仍面临计算复杂度较高、抗字典攻击能力优化、协议安全性证明、通信开销控制等诸多挑战。因此, 研究一种高效、安全、量子抗性的 PAKE 协议, 不仅具有重要的密码学理论价值, 还能确保 PAKE 协议在量子计算环境下仍能提供可靠的安全保障, 为未来信息安全体系、量子安全认证协议的标准化、云计算和物联网安全应用提供关键技术支持。

1.2 国内外研究现状

本节将分三部分介绍目前国内外对于本文所涉及领域的研究现状, 分别为口令认证密钥协商协议、基于同源的口令认证密钥协商协议以及 (强) 非对称的口令认证密钥协商协议现状。本文提出的方案主要围绕基于同源的口令认证密钥协商协议设计展开。

1.2.1 口令认证密钥协商协议

1992 年, Bellovin 和 Merritt^[1] 最早提出 PAKE 协议的概念, 并设计了 EKE (Encrypted Key Exchange) 协议, 该协议通过加密的方式在通信双方之间传输密钥, 但仍存在某些安全性漏洞。随后, 许多 PAKE 方案被相继提出, 如 EKE2^[2]、SPEKE^[3]、CPace^[4]、OPAQUE^[5]、SPAKE2^[6] 和 J-PAKE^[7] 等。此外, 国内也有许多学者提出了多种 PAKE 方案^[8-10]。主要的 PAKE 设计根据口令的使用可分为三种类型^[11]: 1) 口令作为加密密钥; 2) 口令作为输入字符串来派生生成元; 3) 口令作为模运算中的整数 (乘法群中的指数, 或椭圆曲线上加法群中的标量)。其中, 第三类又可以细分为可信设置、安全两方计算和口令用于派生指数三种。每种方案都面临不同的实例化问题和挑战。

EKE 协议, 作为口令作为加密密钥的经典方案, 包含了多个变体, 如 RSA-EKE、ElGamal-EKE 和 DH-EKE 等。而在这几个变体中, DH-EKE 被认为是最有研究意义的, 且在可证明安全相关的文献中具有重要地位。DH-EKE 协议利用了 Diffie-Hellman 协议的密钥协商机制, 在此基础上引入了对口令的加密验证, 避免了将明文口令直接传输或存储。2000 年, Bellare 等人^[2] 提出了 DH-EKE 协议的改进版本 EKE2, 增强了原始协议的安全性, 尤其是提高了防止离线字典攻击的能力, 并对 DH-EKE 及 EKE2 的安全性进行了正式证明, 其使用的 BRP 安全模型被其他研究人员广泛采用。1993 年, Bellovin 和 Merritt^[12] 提出了一种通用的 “A” 方法, 将 EKE 方案转换为增强版本, 称为 A-EKE。他

分类	口令用途		代表性方案
1	口令作为加密密钥		EKE2
2	口令作为输入字符串导出生成器		Patched SPEKE, Cpace, OPAQUE
3	口令作为模运算中的整数	可信设置	SPAKE2
4		安全两方计算	J-PAKE
5		口令派生指数	SRP-6a

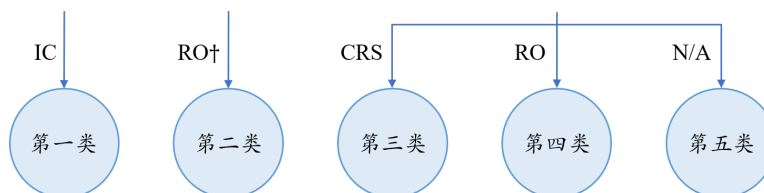


图 1.1 PAKE 分类示意图

们的方法基于数字签名方案，服务器存储用于签名验证的由口令派生的公钥。2021 年，Gu、Jarecki 和 Krawczyk 提出了一种增强方案 HKAPE。HKAPE 提出对基于 HMOV^[13] 或 3DH^[14] 的 EKE 协议进行加密，并在 IC 和随机预言机（RO）模型下基于 Gap-CDH 假设被证明了安全性，但对于 IC 的详细构造和分析并未在文中展开。到目前为止，实例化理想密码的困难阻碍了这一类协议的广泛使用^[15]。

1992 年，Jablon^[3] 提出了第二类协议 SPEKE，其核心思想是使用 H2G 函数将口令映射到素数阶群中的生成元。最初，Jablon 将该 H2G 函数定义为 $f = pw^2 \bmod p$ （ pw 为一个（弱）口令， p 为一个安全素数）。Zhang^[16] 在 2004 年针对 f 分析得出，主动攻击者可以通过利用口令之间的指数等价性一次性测试多个口令。为解决此问题，在 IEEE P1363.2 标准化过程中，通过添加哈希操作将函数修改为 $f = H(pw^2) \bmod p$ 。2014 年，Hao 和 Shahandashti^[17] 指出，SPEKE 容易受到未知密钥共享攻击。因此，SPEKE 更新了修订版本^[18]，在密钥派生函数中包含了用户身份。2019 年，Haase 和 Labrique^[4] 提出了另一个 SPEKE 变体 CPace，该协议在 f 函数中加入了会话标识符和用户身份。此外，Jablon^[19] 在 1997 年提出了一种通用的“B”方法，可以将 SPEKE 方案扩展为增强版本。这种方法基于 Diffie-Hellman 方案，服务器存储由口令派生的 Diffie-Hellman 公钥。其他增强方案包括 PAK-X^[20]、OPAQUE^[5] 和 AuCPace^[4] 等。对于这一类协议，一个核心挑战在于如何在不同的群设置中实例化 f 函数。

第一个基于可信设置的协议是 Katz 等人^[21] 在 2001 年提出的 KOY，其动机是在不依赖随机预言机模型的情况下设计一个可证明安全的 PAKE。在 KOY 之后，出现了其他基于可信设置的 PAKE 方案，包括 Kobara-Imai^[22]、Jiang-Gong^[23]、SPAKE2^[6]、SESPAKE^[24] 和 TBPEKE^[25]。其中，Abdalla 和 Pointcheval 在 2005 年提出的 SPAKE2 被认为是最有效的，已被纳入多个标准（如 IETF）和开源项目。这一类协议的关键问题

是如何实例化可信设置。一种常用的方法是通过可信第三方（TTP）来选择所使用的参数，另一种方法是使用安全哈希函数生成固定的生成元以供标准化使用。然而，实际上并不存在普遍可信的第三方，对于用户基数大的系统而言，长期使用固定值的离散对数关系存在很大风险。目前还没有基于可信设置的方案得到大规模商业应用。

目前唯一的安全两方计算分类协议是 2008 年由 Hao 和 Ryan^[7] 提出的 J-PAKE 协议。J-PAKE 协议的核心思想是将 PAKE 视为一个安全两方计算问题，将 AV-net 多方计算协议^[26] 从让多个参与方计算 boolean-OR 函数，修改为计算等式函数。JJ-PAKE 协议的主要优势在于它不依赖于 H2C（Hash-to-curve）函数，也不需要可信设置，简化了实现过程，并且能够在不依赖外部可信环境的情况下，保证通信双方能够安全地协商出共享的会话密钥。尽管 J-PAKE 协议具有显著的实现优势，但在安全性和计算复杂度方面仍然存在进一步优化的空间。2016 年，Lancrenon 等人^[27] 提出了两个变体，这两个变体通过进一步降低计算量来提升效率。然而，这些改进版本虽然减少了计算量，但分别依赖于 H2C 函数和可信设置。H2C 函数的使用在一定程度上恢复了传统 PAKE 协议的复杂性，而可信设置则意味着协议仍然需要一个可信的第三方来生成并分发系统参数，从而可能带来一些安全隐患。

最后一类协议方案均为增强型 PAKE（Augmented PAKE）。在这类方案中，服务器存储 g^w ，用于在与客户端进行类似 DH 的密钥协商时进行验证。1998 年，Wu^[28] 提出了 SRP，该协议允许客户端和服务端在不传输明文口令的情况下进行相互认证，并协商出会话密钥。其他该类协议的设计与 SRP 类似。2000 年，Kwon^[29] 提出一种高效的 PAKE 协议 AMP。由于被指出主动攻击者可以同时测试两个口令，这两个协议分别被修订为 SRP-6a 和 AMP+。后续 AMP 针对效率和安全性的优化又进行了新的修订。2010 年，Shin 和 Kobara^[30] 提出了 AugPAKE，但 Jarecki 等人^[5] 对其安全证明的有效性提出了质疑。

随着量子计算机的快速发展，Shor 算法的提出使得基于传统公钥密码学（如 RSA、ECC、DH）的加密方案在多项式时间内变得易于破解，这对当前的 PAKE 协议的安全性构成了严峻的挑战。Shor 算法能够有效地解决大数分解和离散对数问题，而这些问题是传统公钥密码学安全性的根基。因此，面对量子计算的威胁，研究人员亟需开发能够抵抗量子攻击的加密方案，尤其是在密码协议领域。为应对量子计算的挑战，学者们积极开展了后量子密码学的研究，提出了多种后量子 PAKE 协议。这些后量子算法的技术路线主要包括以下几种：首先是基于格的方案，这类方案利用格的硬性问题（如最短向量问题）作为安全基础，已成为后量子密码学中最为关注的一条路径；其次是基于编码的方案，这类方案依赖于错误更正码的数学性质，提供了量子计算下的安全性保障；另外，基于哈希的方案通过设计特定的哈希函数来增强加密安全性，且相对简单易实现；此外，基于多变量的方案利用多变量多项式方程的解的困难性，提出了一种新的安全构建方法；最后是基于同源的方案，该技术基于椭圆曲线之间的同源关系，利用计

算同源映射的困难性为密码学提供安全保障。

本文的研究重点聚焦于基于同源的 PAKE 协议。

1.2.2 基于同源的口令认证密钥协商协议

目前,主流的 PAKE 协议大多基于 Diffie-Hellman 密钥协商算法构建。Diffie-Hellman 算法作为公钥密码体系的核心之一,通过允许双方在不直接共享秘密的情况下安全地交换密钥,为加密通信提供了基础。近年来,随着量子计算技术的进步,传统密码学协议面临着严峻的挑战。因此,基于同源的密码协议逐渐成为抗量子密码学领域的重要组成部分。该类协议依赖于计算给定椭圆曲线之间同源的困难性,该问题在量子算法的攻击下仍然保持着(亚)指数级别的时间复杂度,从而具有较强的抗量子能力。

相比其他抗量子密码协议,基于同源的密码体系的主要优势在于其较短的密钥长度,这使得其在实现时具有较小的存储需求。然而,其主要劣势在于协议效率较低,尤其是在执行大量复杂的群操作时,计算开销较大。尽管基于同源的方案在计算复杂度上存在一定挑战,但其理论基础和应用潜力使其成为后量子密码学中备受关注的重要方向。当前,基于同源的密钥协商协议主要分为三类:普通同源 Diffie-Hellman 协议(ordinary isogeny Diffie-Hellman, OIDH)、超奇异同源 Diffie-Hellman 协议(supersingular isogeny Diffie-Hellman, SIDH)和交换超奇异同源 Diffie-Hellman 协议(commutative supersingular isogeny Diffie-Hellman, CSIDH)。

OIDH 协议的实现效率相对较低,且由于其群操作的交换性质,Childs 等人^[31]提出的一个亚指数时间量子算法,能够有效地破解该协议。尽管 SIDH 比 OIDH 具有更高的计算效率,但由于 Castryck 和 Decru^[32]在 2023 年提出了多项式时间的密钥恢复攻击, SIDH 协议被证明在某些情况下不具备足够的安全性。

相对于 SIDH, CSIDH 是 Castryck 等人^[33]基于 SIDH 提出的一种改进密码原语。CSIDH 通过将 Couveignes-Rostovtsev-Stolbunov 方案^[34, 35]应用于超奇异椭圆曲线,提供了比 SIDH 更紧凑的密钥、密文和签名。2022 年, Abdalla 等人^[36]进一步扩展了这一框架,引入了计算椭圆曲线二次扭曲的能力,提出了两种基于交换群运算的 PAKE 协议,其中 X-GA-PAKE_ℓ是其中一种基于同源的单轮协议。

在 X-GA-PAKE_ℓ协议中,协议双方共享一个公共字符串 crs 和一个 ℓ 位口令。为了防止离线字典攻击,该协议通过将集合元素加倍(即各方选择 2ℓ 元素并执行 5ℓ 次群操作)来增强安全性。尽管增加了通信开销,这一设计有效地提升了协议的抗攻击能力。该方案的流程如图 1.2 所示。2023 年, Ishibashi 和 Yoneyama^[37]在 X-GA-PAKE_ℓ的基础上进行了优化,提出了一种将冗余设置参数从 ℓ 减少到 2|crs| 的方案,进一步减少了协议中的冗余操作。然而,该优化方案在通信成本、效率以及安全性方面仍然存在进一步提升的空间。

本文在对上述方案进行深入分析和对比的基础上,提出了一种优化方案,旨在在不

改变协议安全性的前提下，降低通信和计算开销。此外，本文进一步在相同的安全模型下，对该优化方案进行了详细的安全性分析，以确保方案的有效性和可行性。

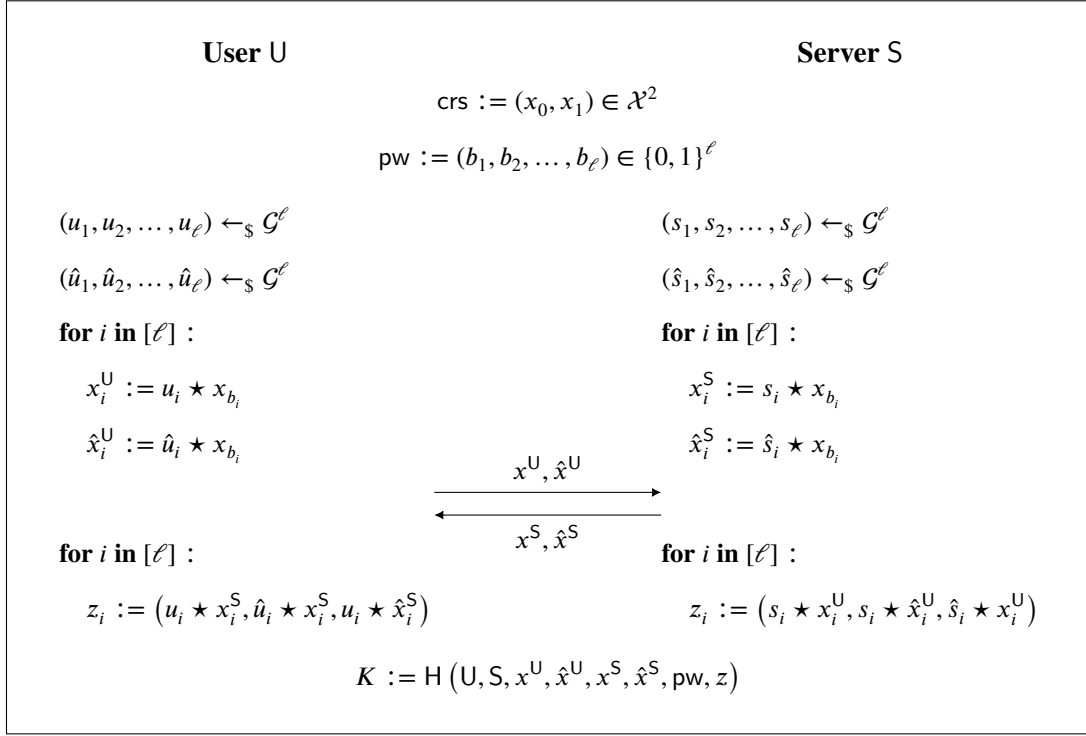


图 1.2 PAKE 协议 X-GA-PAKE_ℓ 的流程图。

1.2.3 （强）非对称的口令认证密钥协商协议

非对称 PAKE (Asymmetric PAKE, aPAKE)^{[38][39]} 是 PAKE 协议在“客户端-服务器”模式下的一个重要变体。与传统 PAKE 协议主要应用于对称角色的环境不同，aPAKE 协议允许服务器仅存储口令的哈希值，从而避免了明文口令存储的安全风险。然而，这种方式容易受到预计算攻击。具体来说，攻击者可能会预先计算并存储一组口令及其哈希值的对 $(\text{pw}, H(\text{pw}))$ ，通过遍历服务器存储的预计算表，快速猜测口令。这种方法不仅提高了攻击效率，还可能使口令恢复变得更为简单。

为了应对预计算攻击，Krawczyk 和 Xu^[5] 提出了强非对称 PAKE (strong asymmetric PAKE, saPAKE) 的概念。该方案的目标是增强服务器被攻陷后攻击者恢复密码的难度。在 saPAKE 协议中，即使服务器遭到入侵，攻击者在尝试恢复用户口令时，也必须投入与口令猜测次数成比例的计算成本，从而有效延缓了恢复口令的过程。

随后的研究中，Haase 和 Labrique^[4] 于 2019 年提出了 strong AuCPace 协议，该协议在（对称）PAKE 协议之前引入了一个修改版的 OPRF (Oblivious Pseudorandom Function)。此方案通过加强 PAKE 协议的初步阶段，进一步提高了抗攻击能力。同年，Bradley 等人^[40] 基于承诺 (Commitment) 和平滑投影哈希函数 (Smooth Projective Hash Function, SPHF) 技术，提出了一种 PAKE 构造，成功实现了 $\mathcal{F}_{\text{saPAKE}}$ 的完整功能。

2022 年, Cremers 等人^[41] 提出了 CRISP, 作为一种将 PAKE 协议编译为强身份绑定 PAKE (strong identity-binding PAKE, siPAKE) 的方法。然而, CRISP 编译器在计算、通信和灵活性方面受到了限制, 原因在于它严重依赖双线性配对技术。此外, 配对的依赖使得 CRISP 编译器没有后量子实例化的可能性, 限制了其在量子计算环境下的适用性。

2023 年, McQuoid 和 Xu^[42] 提出了一种高效且安全的 saPAKE 编译器。与之前的方案相比, 该协议实现了标准的 UC saPAKE 功能, 并且在指数运算次数上明显少于 strong AuCPace 和 Bradley 等人^[40] 的协议。该方案不仅在计算效率上有显著提升, 还在安全性方面表现优越。此外, 该方案可用于基于同源的 CSIDH, 并能通过后量子假设生成 saPAKE 协议, 展示了其在后量子时代的潜力。

本文借鉴了上述方案的优化思路, 结合 McQuoid 和 Xu^[42] 提出的方案, 设计并实现了一个基于同源的非对称 PAKE 协议。

1.3 主要研究内容

本文以口令认证密钥协商为研究核心, 聚焦于基于同源的 PAKE 技术, 旨在为安全通信领域提供更高效、更安全的解决方案。随着信息安全需求的不断提高, 传统的 PAKE 协议在计算开销、通信复杂度以及抗攻击能力等方面仍然存在一定的局限性。本文针对当前相关技术在效率和安全性方面的不足, 展开了深入的研究与探索。基于对现有方案的比较分析, 本文提出了一种基于随机同源置换的口令认证密钥协商方法, 并在此基础上将该方案由对称协议扩展为非对称协议, 使其适用于更加广泛的应用场景, 如服务器认证、端到端加密以及零信任架构下的身份验证。

本文的主要工作如下:

1. 针对现有方案在通信开销和计算复杂度方面的缺陷, 本文提出基于随机同源置换的 PAKE 方案 R-GA-PAKE_ℓ。在保证安全性不变的情况下, 本文通过引入随机同源置换机制, 设计了新的密钥协商流程, 利用 Merkle 根和口令生成的种子对中间结果进行随机置换, 增强了方案对离线字典攻击的抵抗能力。结合有效群操作 (EGAT) 的安全性假设, 本文对 R-GA-PAKE_ℓ 方案进行严格的安全性规约, 证明其能有效抵御多种攻击, 满足 PAKE 协议的安全需求。
2. 在 R-GA-PAKE_ℓ 的基础上, 本文通过强非对称 PAKE 编译器对该方案进行扩展, 提出非对称 PAKE 方案 aR-GA-PAKE_ℓ。该方案不仅继承了 R-GA-PAKE_ℓ 方案在通信和计算方面的优势, 还通过在服务器端采用特定的存储方式替代口令的直接存储, 有效防止了预计算攻击, 增强了对服务器端被攻破场景的抵抗能力。此外, 本文对该方案的不可伪造性和抵抗攻击能力展开了理论分析。
3. 本文结合典型的安全通信应用场景, 在口令系统内对优化方案进行了设计与实现, 并在实验环境下进行了正确性与性能评估。实验结果表明, 该方案不仅能够满足

口令认证密钥协商的安全性要求，还在计算开销、通信复杂度等方面表现出良好的效果，从而展示了方案的实际可行性与应用价值。

1.4 论文结构组织

本文共分为六个章节，具体内容如下：

第一章为绪论章节。从互联网时代信息技术发展带来数据存储和处理复杂化，多租户环境、大规模数据处理及量子计算机出现给数据安全带来新挑战入手，强调在云计算和大数据环境下保障数据安全与隐私的重要性，引出认证密钥协商（AKE）技术，重点阐述其子集口令认证密钥协商（PAKE）在数据安全中的关键作用及面临的字典攻击问题，以及量子计算发展对 PAKE 协议的挑战和基于同源密码学应对挑战的研究方向。接着介绍国内外对于 PAKE 协议的研究现状，分析目前各类协议存在的问题。最后，对本文的主要研究内容和论文整体的结构组织进行了梳理概括。

第二章为预备知识章节。此章主要涵盖论文所涉及的基础知识内容。开篇对数学基础与密码学基础展开介绍，随后针对本文方案在改进过程中所涉及的群操作、本文方案涉及的计算假设、Knuth-Durstenfeld 洗牌算法进行详细阐述，这些知识将为后续深入探讨方案的设计与优化提供必要支撑。

第三章为基于随机同源置换的口令认证密钥协商方案。本章引入了用于分析 PAKE 方案安全性的 BPR（Bellare-Pointcheval-Rogaway）安全模型及其扩展查询机制；其次，详细阐述了所提出方案的整体架构、协议流程及关键技术；随后，根据方案采用的安全模型，分析并证明方案的安全性；最后，评估方案的计算复杂度与通信开销，并将其与现有的相关 PAKE 方案进行对比分析，展现了方案在性能开销上的优势。

第四章为基于同源的不对称口令认证密钥协商方案。本章首先介绍了非对称 PAKE 方案的设计挑战和安全模型；其次，对第三章的方案进行优化，使得服务端无需存储口令即可完成密钥协商，有效规避了口令存储带来的安全风险；最后，基于安全模型对方案的不可伪造性和抵抗攻击能力进行了分析，通过理论分析表明，该方案能够有效降低口令直接存储带来的潜在安全风险，同时保持方案的计算与通信性能。

第五章为基于同源的口令系统设计与实现。本章分别对第四章提出的方案在口令系统内进行了实现，对方案的注册、登录、密钥协商阶段进行模拟，验证方案的正确性，并对系统性能进行了评估。通过系统测试证明了方案在口令系统中的可行性。

第六章为总结和展望章节。本章总结了本文工作内容，并对未来的研究方向做出了展望。

2 预备知识

本章主要介绍了文中方案涉及的预备知识，包括数学基础知识、密码学原语、群操作及其计算假设，并介绍了方案中使用的 Knuth-Durstenfeld 洗牌算法。在本文中，使用的符号定义如表 2.1。

表 2.1 文中符号及相关描述

符号	描述
$[n]$	整数集 $\{1, \dots, n\}$
$x \leftarrow_{\S} \mathcal{X}$	从 x 中均匀独立地随机抽取一个元素 x
$y \leftarrow \mathcal{A}(x_1, \dots, x_n)$	敌手 $\mathcal{A}$ 将 $x_1, \dots, x_n$ 作为输入并输出 y
$\mathcal{A}^{\mathcal{O}}$	敌手 $\mathcal{A}$ 能够访问预言机 $\mathcal{O}$
G_i	第 i 个游戏
$\Pr [\text{Succ}_i]$	游戏 G_i 最终成功（输出 1）的概率

2.1 数学基础

本节主要介绍本文方案涉及的数学基础，包括群、有限域、椭圆曲线、超奇异椭圆曲线和同源等。

(1) 群和有限域

群是一个代数结构，包含了一个非空集合以及该集合上定义的一种代数运算，通常记作 $\langle \mathbb{G}, \cdot \rangle$ ，可简单记为 $\mathbb{G}$ 。 $\mathbb{G}$ 满足下述几种性质：

1. 有封闭性: 对 $\forall a, b \in \mathbb{G}$ ，有 $a \cdot b \in \mathbb{G}$;
2. 结合性: 对 $\forall a, b, c \in \mathbb{G}$ ，有 $(a \cdot b) \cdot c = a \cdot (b \cdot c)$;
3. 单位元: 群 $\mathbb{G}$ 中存在特定的单位元素 e ，对 $\forall a \in \mathbb{G}$ ，有 $e \cdot a = a \cdot e = a$;
4. 逆元: 对 $\forall a \in \mathbb{G}$ ，存在 a 的逆元 $a^{-1} \in \mathbb{G}$ ，使得 $a \cdot a^{-1} = a^{-1} \cdot a = e$ 。

域由一个非空集合 $\mathbb{F}$ 和在 $\mathbb{F}$ 上定义的加法 (+) 和乘法 (·) 两种二元运算组成。域具有以下性质：

1. $\mathbb{F}$ 对加法构成交换群，其加法恒等元（零元）记作 0;
2. $\mathbb{F}$ 中所有非零元素对乘法构成交换群，其乘法恒等元（单位元）记作 1;

3. 在两种代数运算间，有如下分配律：

$$a \cdot (b + c) = a \cdot b + a \cdot c.$$

通常，用 $\mathbb{F}^*$ 表示集合 $\mathbb{F} \setminus \{0\}$ 。根据定义可知， $(\mathbb{F}, +)$ 是一个加法群， $(\mathbb{F}, \cdot)$ 是一个乘法群。若 $\mathbb{F}$ 中的元素数量有限，则称 $\mathbb{F}$ 为有限域，也称伽罗瓦域。

(2) 椭圆曲线和同源

椭圆曲线可以分为 Edward 曲线、Weierstrass 曲线、Montgomery 曲线。在 CSIDH 中，主要使用的是 Montgomery 曲线。Montgomery 曲线在密码学中具有独特的优势，尤其是在标量乘法运算方面。其高效的标量乘法算法，如 Montgomery 梯子算法 (Montgomery ladder algorithm)，能够在不泄露标量信息的情况下快速完成乘法运算，这对于抵抗侧信道攻击（如计时攻击、功耗分析攻击等）非常有效。因此，在实际的密码学应用中，Montgomery 被广泛用于实现安全的密钥协商和数字签名等操作。在特征 $p > 3$ 的有限域 $\mathbb{F}_p$ 上，蒙哥马利曲线的一般方程为：

$$E : By^2 = x^3 + Ax^2 + x,$$

其中 $A, B \in \mathbb{F}$ ，且满足 $A(B^2 - 4) \neq 0$ 。该条件保证了曲线是光滑的，意味着曲线没有尖点或自相交等奇异点，这对于在曲线上进行有效的数学运算和密码学操作至关重要。除曲线 E 上的所有点外，还存在一个特殊的点 O ，记作无穷远点。基于椭圆曲线上的离散对数问题，密码学领域构建了一系列椭圆曲线密码算法，如椭圆曲线 Diffie-Hellman (DH) 密钥协商算法和椭圆曲线数字签名算法 (ECDSA) 等。

椭圆曲线同源指两条椭圆曲线之间的同态，假设有两条定义在有限域 $\mathbb{F}_q$ 上的椭圆曲线 E_1 和 E_2 ，设 $\phi : E_1 \rightarrow E_2$ 是一个态射，且满足 $\phi(O_{E_1}) = O_{E_2}$ ，则称 ϕ 为椭圆曲线之间的同源。由 Tate 定理可知^[43]，当且仅当 $\#E_1(\mathbb{F}_q) = \#E_2(\mathbb{F}_q)$ 时，两条曲线 E_1 和 E_2 是同源的。对于同源 $\phi : E \rightarrow E$ ， ϕ 称为自同态。自同态构成的集合可表示为 $\text{End}(E)$ ，又称为自同态环。

(3) 超奇异椭圆曲线

超奇异椭圆曲线是椭圆曲线的一个特殊子类，相较于普通椭圆曲线，超奇异曲线的自同态环 (endomorphism ring) 具有不同的代数结构，这使得它们在某些计算问题上表现出独特的性质。超奇异椭圆曲线的自同态环 $\text{End}(E)$ 与四元数代数中的一个阶同构^[44]。具体来说，如果一个椭圆曲线 E 定义在 $\mathbb{F}$ 上，其 Frobenius 迹 $t = p + 1 - \#E(\mathbb{F}_p)$ ($\#E(\mathbb{F}_p)$ 为曲线上的点的个数) 满足 $t \equiv 0 \pmod{p}$ (其中 p 是 $\mathbb{F}_p$ 的特征)。在超奇异椭圆曲线之间寻找等价的映射 (同源映射) 被认为是一个困难问题，既难以用经典计算方法求解，也难以用量子算法 (如 Shor 算法) 高效求解。

(4) 数学困难问题

本文的安全性分析依赖于多个计算假设，这些假设均基于下述数学困难问题。令 $\mathbb{G}$ 是一个生成元为 g ，素数阶为 p 的循环群。

定义 2.1.1 (DL 问题): DL (Discrete Logarithm) 问题要求：给定 g^a ，其中 $a \leftarrow \mathbb{Z}_p^*$ ，计算出 a 。

定义 2.1.2 (CDH 问题): CDH (Computational Diffie-Hellman) 问题要求：给定一个元组 (g^a, g^b) ，其中 $(a, b) \leftarrow (\mathbb{Z}_p^*)^2$ ，计算出 g^{ab} 。

定义 2.1.3 (DDH 问题): DDH (Decisional Diffie-Hellman) 问题要求：给定一个元组 (g^a, g^b, g^c) ，其中 $(a, b) \leftarrow (\mathbb{Z}_p^*)^2$ ，判断 c 是否满足 $g^c = g^{ab}$ ，即区分 (g^a, g^b, g^{ab}) 和 (g^a, g^b, g^c) ，其中 $(a, b, c) \leftarrow (\mathbb{Z}_p^*)^3$ 。

敌手 $\mathcal{A}$ 的优势，记为 $\text{Adv}^*(\mathcal{A})$ 。比如： $\text{Adv}^{\text{DL}}(\mathcal{A})$ 表示敌手 $\mathcal{A}$ 解决 DL 问题的优势。相应的计算假设证明，不存在多项式时间可证明 (Probabilistic Polynomial Time, PPT) 敌手，使得 $\text{Adv}^*(\mathcal{A})$ 为非可忽略的，即没有有效的算法可以在合理时间内成功求解该问题。在小节 2.4.3 中，介绍了这些问题在群操作上的扩展。

2.2 密码学原语

本节主要介绍本文方案涉及的密码学原语，包括伪随机函数、伪随机置换、哈希函数和随机预言机等。

(1) 伪随机函数

给定一个函数 $F : \mathcal{K} \cdot \mathcal{X} \rightarrow \mathcal{X}'$ ，其中 $\mathcal{K}$ 是密钥空间， $\mathcal{X}$ 是一个消息空间， $\mathcal{X}'$ 是映射空间。若对所有 PPT 敌手 $\mathcal{A}$ 来说，区分 F 和真随机函数 f 输出的概率是可忽略的，则 F 可被称为伪随机函数 (Pseudorandom function, PRF)，即：

$$\text{Adv}_F^{\text{PRF}}(\mathcal{A}) = \left| \Pr [\mathcal{A}^{F(k, \cdot)}(1^\lambda) = 1] - \Pr [\mathcal{A}^{f(\cdot)}(1^\lambda) = 1] \right| \leq \text{negl}(\lambda),$$

其中， λ 是函数 F 的安全参数，决定 PRF 的安全级别； $k \leftarrow_{\$} \mathcal{K}$ 是伪随机函数的密钥； $\mathcal{A}^{F(k, \cdot)}$ 表示敌手 $\mathcal{A}$ 可以访问伪随机函数 F 的查询接口； $\mathcal{A}^{f(\cdot)}$ 表示攻击者访问的是真正的随机函数 f ； $\text{negl}(\lambda)$ 是一个可忽略函数，即当 λ 增大时，该函数趋近于零。

(2) 伪随机置换

给定一个函数 $P : \mathcal{K} \cdot \mathcal{X} \rightarrow \mathcal{X}$ ，其中 $\mathcal{K}$ 是密钥空间， $\mathcal{X}$ 是定义域和值域相同的有限集合，则 P 是一个置换，并且存在对应的逆函数 P^{-1} 。若对所有 PPT 敌手 $\mathcal{A}$ 来说，给定 P 及其逆函数 P^{-1} 的访问权限后，区分其输出和一个真随机置换 p 及其逆函数 p^{-1}

的概率是可忽略的，则 P 可被称为伪随机置换（Pseudorandom Permutation, PRP），即：

$$\text{Adv}_P^{\text{PRP}}(\mathcal{A}) = \left| \Pr \left[\mathcal{A}^{P(k, \cdot), P^{-1}(k, \cdot)}(1^\lambda) = 1 \right] - \Pr \left[\mathcal{A}^{p(\cdot), p^{-1}(\cdot)}(1^\lambda) = 1 \right] \right| \leq \text{negl}(\lambda),$$

其中， λ 是函数 P 的安全参数，决定伪随机置换的安全性级别； $k \leftarrow_{\$} \mathcal{K}$ 表示伪随机置换的密钥； $\mathcal{A}^{P(k, \cdot), P^{-1}(k, \cdot)}$ 表示敌手 $\mathcal{A}$ 拥有对 P 和 P^{-1} 的访问权限； $\mathcal{A}^{p(\cdot), p^{-1}(\cdot)}$ 表示敌手访问的是一个真正的随机置换 p 及其逆置换 p^{-1} ； $\text{negl}(\lambda)$ 是一个可忽略函数，即当 λ 增大时，该函数趋近于零。

(3) 哈希函数

哈希函数是一种不可逆、确定性的数学映射，在密码学和数据管理中起着至关重要的作用。其安全性依赖于良好的抗碰撞性、单向性和均匀分布特性，使其成为现代计算机安全体系的重要组成部分。哈希函数 $H : 0, 1^* \rightarrow 0, 1^n$ 是一种数学转换，可以将任意长度的数据作为输入，并生成一个固定长度的输出数据。通常，将哈希函数的输出值称为哈希值。哈希函数具有以下几个关键特点：

- 确定性: 对于相同的输入，哈希函数总是产生相同的输出；
- 雪崩效应: 对于不同的输入数据，微小变化能够导致哈希值的显著变化；
- 均匀性: 哈希函数产生的哈希值在输出空间内均匀分布，即对于不同的输入，哈希值不应该集中在输出空间的某个子集上；
- 单向性: 给定一个哈希值，找出能产生这个哈希值的原始输入（即找到哈希值的原像）在计算上是不可行的；
- 抗碰撞性: 找到两个不同的输入，它们能产生相同的哈希值（即哈希碰撞）在计算上是不可行的。

这些特点使得哈希函数在密码学、数据完整性校验、数据去重、快速查找等领域有着广泛的应用。哈希函数在 PAKE 协议中扮演着重要的角色，用于生成密钥协商过程中的关键参数、验证消息完整性以及提高协议的安全性。

(4) 随机预言机

随机预言机（Random Oracle, RO）是一种理想化的数学模型，用于分析和证明加密协议的安全性。在随机预言机模型中，假设存在一个理想的“预言机”，它接受任意输入并生成一个完全随机的输出。该模型基于哈希函数的计算模型，由 Bellare 和 Rogaway^[45] 于 1993 年受 Fiat 和 Shamir^[46] 的启发提出。在他们的研究中，假设通信双方共享一个公开的随机预言机，这一设定为密码学理论与实际应用之间提供了一个有效的联系。具体来说，协议的正确性首先在随机预言机模型中得到验证，然后在实际实现时，选择合适的哈希函数来替代预言机。其核心假设是，攻击者无法在多项式时间内区分理想模型与实际环境。这种设计方式通常能够保持协议实现的效率，与现有方案相匹配。

随机预言机模型为加密协议的设计提供了一个通用的框架。在安全性证明中，哈希

函数的输出被视作完全随机，任何参与者只能通过查询预言机来获取对应的哈希值。通过操控哈希函数的输出，攻击者可能在实际的攻击环境中，利用其能力破解复杂的计算问题。

如果哈希函数 $H : \{0, 1\}^* \rightarrow \{0, 1\}^n$ 满足以下条件，则称其为随机预言机：

1. 均匀性：输出在 $\{0, 1\}^n$ 上均匀分布。
2. 确定性：对于相同的输入， H 的输出始终相同。
3. 有效性：给定输入 x ，计算 $H(x)$ 可以在关于 x 长度的多项式时间内完成，理想情况下为线性时间。

随机预言机模型通常用于证明协议的安全性，特别是在加密协议设计中。通过假设某些加密操作依赖于理想的随机预言机，可以证明协议在特定假设下是安全的。例如，许多基于哈希的加密协议（如数字签名、密钥交换协议等）在安全分析中会依赖随机预言机模型。然而，尽管随机预言机在理论上是理想的，但实际的实现无法达到完全的随机性，因此现实中使用的哈希函数等“伪随机”函数可能与理想的随机预言机存在差异。尽管如此，随机预言机模型仍然在加密学研究中被广泛使用，尤其是在证明协议安全性方面。

2.3 计算与密码学基础

本节主要介绍本文方案涉及的重点数据结构及算法，包括 Diffie-Hellman 算法、Knuth-Durstenfeld 算法和 Merkle 树等。

(1) Diffie-Hellman 算法

Diffie-Hellman (DH) 密钥交换协议是一种经典的公钥密码学方案，允许双方在不安全的信道上协商一个共享密钥，而无需提前共享密钥信息。该协议由 Whitfield Diffie 和 Martin Hellman^[47] 于 1976 年提出，其安全性依赖于离散对数问题 (DL 问题) 的计算复杂性，即在已知基数 g 和幂指数 $g^x \bmod p$ 的情况下，计算 x 具有极高的计算难度。

Diffie-Hellman 算法的核心思想是：双方各自生成一个私有的随机数（私钥），然后利用公开的数学运算生成一个对应的公有值（公钥）。接着，双方交换公钥，并利用对方的公钥和自己的私钥计算一个共享的密钥。由于指数运算的交换性，两方计算出的共享密钥一致，即使攻击者截获了交换的信息，也难以反推出该密钥。

假设 Alice 和 Bob 需要在公开信道上建立一个共享密钥，Diffie-Hellman 协议的执行过程如下：

1. Alice 和 Bob 同时知道两个大的质数 n 、 g ，且 $(n-1)/2$ 和 $(g-1)/2$ 都是质数，这两个数字均为公开，可被除 Alice 和 Bob 外的人知道；
2. Alice 选择私钥 $a \in \mathbb{Z}_p^*$ ，计算公钥 $A = g^a \bmod p$ ，并将公钥 A 发送给 Bob；
3. Bob 选择私钥 $b \in \mathbb{Z}_p^*$ ，计算公钥 $B = g^b \bmod p$ ，并将公钥 B 发送给 Alice；

4. Alice 得到 Bob 发送的公钥 B 后, 根据已确定的私钥 a 计算共享密钥 $K = B^a \bmod p = (g^b \bmod p)^a \bmod p = g^{ab} \bmod p$;

5. Bob 得到 Alice 发送的公钥 A 后, 根据已确定的私钥 b 计算共享密钥 $K = A^b \bmod p = (g^a \bmod p)^b \bmod p = g^{ab} \bmod p$ 。

由于指数运算具有交换性, 双方计算的密钥 K 结果相同, 即 $K = g^{ab} \bmod p$ 。

(2) Knuth-Durstenfeld 算法

Knuth-Durstenfeld 算法^[48] 是对 Fisher-Yates Shuffle 算法^[49] 进行优化的一种高效随机置换算法。Fisher-Yates Shuffle 最初的实现采用的是“洗牌”式的交换方式, 但其原始版本的时间复杂度为 $O(n^2)$, 在大规模数据处理时效率较低。Knuth-Durstenfeld 版本通过改进索引选择的方式, 将时间复杂度优化至 $O(n)$, 并且不需要额外的存储空间, 使其成为一种轻量级、高效的随机排列方法。

Knuth-Durstenfeld 算法的核心思想是从后向前依次交换, 即每次随机选择一个索引, 并将当前未处理元素与该索引对应的元素交换。这样可以在不使用额外存储的情况下, 实现等概率的随机排列。给定输入序列 $c_1, c_2, \dots, c_n$, 和一个随机种子 $seed$, 需要执行如下运算步骤:

1. 根据随机种子 $seed$ 生成一个范围在 $\{0, \dots, n-1\}$ 之间的随机数 x ;
2. 将输入序列的最后一个元素和下标为 x 的元素互换;
3. 重复步骤 1, 生成一个范围为 0 到 $n-2$ 的随机数 x ;
4. 重复步骤 2, 输入序列的倒数第二个元素和下标为 x 的元素互换;
5. 重复上述过程, 直到所有元素都经历过互换后结束。

算法流程图如图2.1所示。

作为示例, 假设初始数组为 $[1, 2, 3, 4, 5]$, 随机生成的交换索引为 $[3, 1, 4, 0, 2]$, 则算法执行: $[1, 2, 3, 4, 5] \rightarrow [1, 2, 5, 4, 3] \rightarrow [4, 2, 5, 1, 3] \rightarrow [4, 2, 3, 1, 5] \rightarrow [2, 4, 3, 1, 5]$ 。Knuth-Durstenfeld 算法以其简单高效的特性, 成为计算机科学中随机置换问题的经典解决方案。

(3) Merkle 树

Merkle 树 (Merkle Tree) 是一种高效的数据结构, 常用于验证数据完整性和一致性, 广泛应用于区块链、分布式存储系统和密码学协议等领域。Merkle 树采用树形结构, 叶子节点存储数据的哈希值, 而非叶子节点则存储其子节点哈希值的组合结果。典型的 Merkle 树是二叉树, 即每个父节点连接两个子节点, 但在特定场景下, 也可以扩展为 N 叉树, 以提高计算和存储效率。

如图2.2所示, Merkle 树的生成遵循自底向上的方式, 通过逐层哈希计算最终得到 Merkle 根 (Merkle Root), 具体步骤如下:

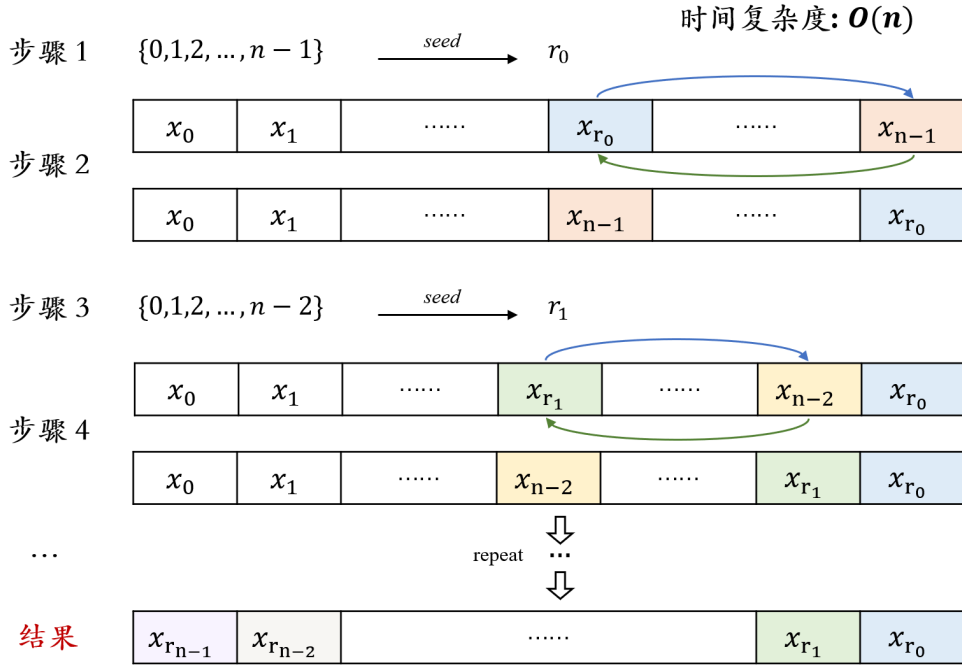


图 2.1 Knuth-Durstenfeld 算法流程图

1. 计算叶子节点哈希值：每个叶子节点存储一个数据块的哈希值，确保即使数据量庞大，也可以通过固定大小的哈希摘要来唯一标识每个数据块。
2. 计算父节点哈希值：将相邻的两个叶子节点的哈希值拼接后计算哈希，所得结果作为它们的父节点哈希值。如果当前层的节点数量为奇数，则最后一个节点通常会被复制一份，以保证树结构的完整性。
3. 递归构建树：对上一层生成的父节点哈希值重复相同操作，依次向上计算，直到生成单个根节点，即 Merkle 根。Merkle 根是整棵树的唯一标识，其哈希值代表所有叶子节点数据的完整性。
4. 存储和验证：生成的 Merkle 树可以存储在数据结构中，或者用于快速验证特定数据块是否存在于数据集中。

Merkle 树的主要用途是验证大量数据的完整性和存在性。由于 Merkle 根是整棵树所有数据的唯一标识，因此只需要验证 Merkle 根，就可以验证整棵树（即所有数据）的完整性。此外，Merkle 树还可以用于快速定位被篡改的数据块，因为只需要重新计算被篡改部分及其祖先节点的哈希值，然后与 Merkle 根进行比较即可。

2.4 群操作

本节将介绍群操作的基础概念、基于同源的群操作定义和相关计算假设。

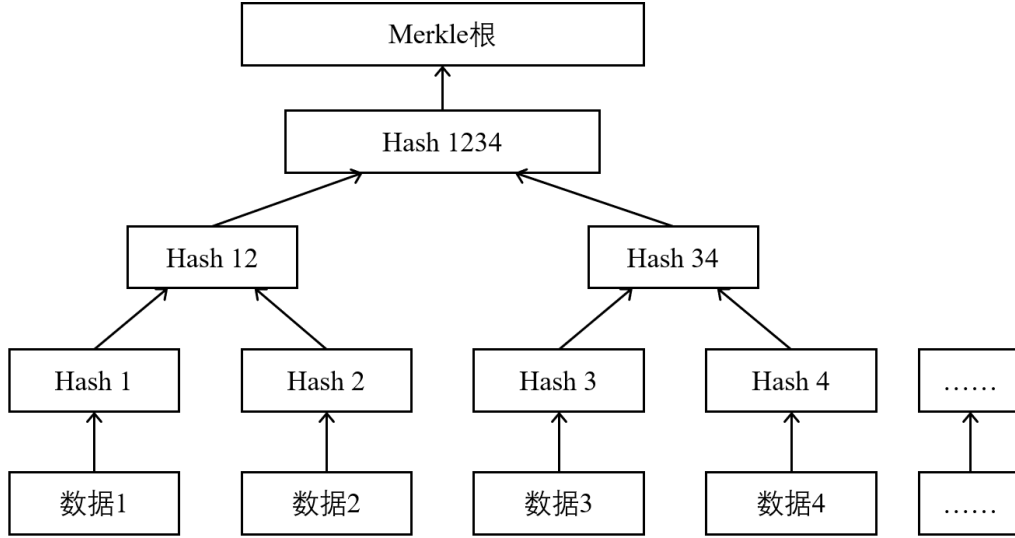


图 2.2 Merkle 树的构建过程

2.4.1 基础概念

本节主要回顾 (受限的) 有效群操作 ((Restricted) Effective Group Actions, ((R)EGAs)) 的定义^[50]。(R)EGAs 提供了一个抽象框架, 用以构建依赖于基于同源的假设 (如 CSIDH) 的加密原语, 对 PAKE 协议的安全性分析至关重要。

定义 2.4.1 (群操作): 令 $(G, \cdot)$ 为一个拥有单元元 $id \in G$ 的有限交换群, $\mathcal{X}$ 是一个集合。一个映射 $\star : G \times \mathcal{X} \rightarrow \mathcal{X}$ 是群操作 (Group Action), 如果它满足以下性质:

1. 单位元: 对于所有 $x \in \mathcal{X}$, $id \star x = x$;
2. 交换律: 对于所有 $g, h \in G$ 且 $x \in \mathcal{X}$, $(g \cdot h) \star x = g \star (h \star x)$ 。

注 2.4.1: 本文仅考虑交换群 G 的群操作。此外, 我们假定群操作是正则的, 即对任意 $x, y \in \mathcal{X}$, 只存在一个 $g \in G$ 满足 $y = g \star x$ 。

定义 2.4.2 (有效群操作): 令 $(G, \mathcal{X}, \star)$ 为一个群操作且满足以下性质:

1. G 是一个有限群, 且存在用于成员资格与等值测试、(随机) 抽样、群运算和逆运算的一组 PPT 算法;
2. $\mathcal{X}$ 是一个有限集合, 且存在用于成员资格测试和计算唯一表示的有效算法;
3. 存在一个已知表示的特殊元素 $\tilde{x} \in \mathcal{X}$;
4. 存在一种计算群操作的有效算法, 即给定 g 和 x , 可以计算 $g \star x$,

则 $(G, \mathcal{X}, \star, \tilde{x})$ 被称为一个有效群操作 (Effective Group Action, EGA), 其基点为 $\tilde{x} \in \mathcal{X}$ 。

定义 2.4.3 (受限的有效群操作): 令 $\mathbf{g} = (g_1, g_2, \dots, g_n)$ 为 G 的生成集, $(G, \mathcal{X}, \star)$ 为一个群操作且满足以下性质:

1. $\mathbb{G}$ 是一个有限群, 且 $n = \text{poly}(\log(\#\mathbb{G}))$;
2. $\mathcal{X}$ 是一个有限集合, 且存在用于成员资格测试和计算唯一表示的有效算法;
3. 存在一个已知表示的特殊元素 $\tilde{x} \in \mathcal{X}$;
4. 存在一种计算群操作的有效算法, 即给定 $g_i \in \mathbf{g}$ 和 $x \in \mathcal{X}$, 可以计算 $g_i \star x$ 和 $g_i^{-1} \star x$,

则 $(\mathbb{G}, \mathcal{X}, \star, \tilde{x})$ 被称为一个受限的有效群操作 (Restricted Effective Group Action, REGA)

定义 2.4.4 (带扭曲的受限有效群操作): 令 $(\mathbb{G}, \mathcal{X}, \star)$ 为一个群操作且满足以下性质:

1. $(\mathbb{G}, \mathcal{X}, \star, \tilde{x})$ 是一个基点为 $\tilde{x} \in \mathcal{X}$ 的 (R)EGA;
2. 存在一种有效算法, 使得给定 $x = g \star \tilde{x} \in \mathcal{X}$, 可以计算 $x^t = g^{-1} \star \tilde{x}$,

则 $(\mathbb{G}, \mathcal{X}, \star, \tilde{x})$ 被称为一个带扭曲的 (受限) 有效群操作 ((Restricted) Effective Group Action with Twists, (R)EGAT)。

2.4.2 基于同源的群操作

基于同源的群操作提供了 REGA 的一种重要实例化方式。本文主要围绕 CSIDH 的设置展开, 下面介绍这一情况下 REGA 的基本定义。

设 p 为形如 $p = 4 \cdot \ell_1 \cdots \ell_n - 1$ 的大素数, 其中 ℓ_i 是若干个小的不同奇素数。固定 E_0 为定义在有限域 $\mathbb{F}_p$ 上的椭圆曲线, 其方程为

$$y^2 = x^3 + x.$$

该曲线 E_0 是超奇异的, 其在 $\mathbb{F}_p$ 上的自同态环为 $\mathcal{O} = \mathbb{Z}[\pi]$ 。其中, π 是 Frobenius 自同态。定义 $\mathcal{E}ll_p(\mathcal{O})$ 为定义在 $\mathbb{F}_p$ 上、端同态环为 $\mathcal{O}$ 的椭圆曲线集合。理想类群 $cl(\mathcal{O})$ 作用在集合 $\mathcal{E}ll_p(\mathcal{O})$ 上, 即存在如下映射:

$$\star : cl(\mathcal{O}) \times \mathcal{E}ll_p(\mathcal{O}) \rightarrow \mathcal{E}ll_p(\mathcal{O})$$

$$([a], E) \mapsto [a] \star E$$

该映射满足定义 2.4.1 中的性质。此外, Castryck 等人^[33] 的分析表明 $(cl(\mathcal{O}), \mathcal{E}ll_p(\mathcal{O}), \star, E_0)$ 确实构成一个 REGA。在 $\mathcal{E}ll_p(\mathcal{O})$ 中, 椭圆曲线的方程形式为 $E_A : y^2 = x^3 + Ax^2 + x$, 可通过 Montgomery 系数 $A \in \mathbb{F}_p$ 来表示它们。

此外, CSIDH 群操作还有一个未在定义 2.4.3 中涵盖的固有性质, 即对于任意曲线 $E_A = [a] \star E_0 \in \mathcal{E}ll_p(\mathcal{O})$, 其它二次扭曲可通过计算 $(E_A)^t = E_{-A}$ 得到, 且满足 $(E_A)^t = [a]^{-1} \star E_0$ 。

2.4.3 计算假设

本节主要回顾三个计算问题 GA - StCDH、SqlInv - GA - StCDH 和 Sim - GA - StCDH (定义2.4.7至2.4.9)^[36]。本文方案的后续安全性证明依赖于这些问题的困难性。在(受限的)有效群操作中,基于离散对数假设和 Diffie-Hellman 假设的自然推广,具备以下困难性假设:

定义 2.4.5 (GA-DL 问题): GA-DL (Group Action Discrete Logarithm) 问题要求给定 $(x, y) \in \mathcal{X}^2$ 和 $y = g \star x$, 求解 $g \in \mathbb{G}$ 。

定义 2.4.6 (GA-CDH 问题): GA-CDH (Group Action Computational Diffie-Hellman) 问题要求给定 $(x, y_0, y_1) \in \mathcal{X}^3$, $y_0 = g_0 \star x$, $y_1 = g_1 \star x$, 其中 $g_0, g_1 \in \mathbb{G}$, 求解 $z = (g_0 \cdot g_1) \star x$ 。

定义计算预言机 $\text{GA-CDH}_x(y_0, y_1) = g_0 \star y_1$ 和决策预言机 $\text{GA-DDH}_x(y_0, y_1, z)$, 其中 $g_0 \in \mathbb{G}$ 且 $y_0 = g_0 \star x$ 。若 $\text{GA-CDH}_x(y_0, y_1) = z$, $\text{GA-DDH}_x(y_0, y_1, z)$ 输出 1, 否则输出 0。若预言机的下标 $x = \tilde{x}$, 则将其省略, 即 $\text{GA-CDH}_{\tilde{x}}(y_0, y_1) = \text{GA-CDH}(y_0, y_1)$, $\text{GA-DDH}_{\tilde{x}}(y_0, y_1, z) = \text{GA-DDH}(y_0, y_1, z)$ 。

定义 2.4.7 (GA-StCDH 问题): GA-StCDH (Group Action Strong Computational Diffie-Hellman) 问题要求基于输入 $(g \star \tilde{x}, h \star \tilde{x}) \in \mathcal{X}^2$ 计算集合元素 $(g \cdot h) \star \tilde{x}$ 。对于敌手 $\mathcal{A}$, 计算一个有效群操作 $\text{EGA}_x \in \{\text{EGA}, \text{REGA}, \text{EGAT}, \text{REGAT}\}$ 的优势为:

$$\text{ADV}_{\text{EGA}_x}^{\text{GA-StCDH}}(\mathcal{A}) := \Pr[\mathcal{A}^{\text{GA-DDH}(g \star \tilde{x}, \cdot, \cdot)}(g \star \tilde{x}, h \star \tilde{x}) \Rightarrow (g \cdot h) \star \tilde{x}],$$

其中 $(g, h) \leftarrow_{\$} \mathbb{G}^2$ 且 $\mathcal{A}$ 可以访问决策预言机 $\text{GA-DDH}(g \star \tilde{x}, \cdot, \cdot)$ 。

定义 2.4.8 (SqlInv-GA-StCDH 问题): SqlInv-GA-StCDH (Square-Inverse Group Action Strong Computational Diffie-Hellman) 问题要求基于输入 $x = g \star \tilde{x}$ 找到一个元组 $(y, y_0, y_1) \in \mathcal{X}^3$, 使得 $y_0 = g^2 \star y$ 且 $y_1 = g^{-1} \star y$ 。对于敌手 $\mathcal{A}$, 计算一个有效群操作 $\text{EGA}_x \in \{\text{EGA}, \text{REGA}, \text{EGAT}, \text{REGAT}\}$ 的优势为:

$$\text{ADV}_{\text{EGA}_x}^{\text{SqlInv-GA-StCDH}}(\mathcal{A}) := \Pr \left[\begin{array}{l} y_0 = \text{GA-CDH}_{x^t}(x, y) \\ y_1 = \text{GA-CDH}(x^t, y) \\ (y, y_0, y_1) \leftarrow \mathcal{A}^{\mathcal{O}}(g \star \tilde{x}) \end{array} \right],$$

其中 $g \leftarrow_{\$} \mathbb{G}$ 且 $\mathcal{O} = \{\text{GA-DDH}_{x^t}(x, \cdot, \cdot), \text{GA-DDH}(x, \cdot, \cdot)\}$ 。

注 2.4.2: 若假设敌手仅限于使用群和扭曲操作, 那么 SqlInv-GA-StCDH 将成为一个具有挑战性的问题。具体而言, $\mathcal{A}$ 仅能根据已知元素 $\tilde{x}$ 、其输入 x 或 x^t 来选择 y 。

对于 $\alpha \in \mathbb{G}$, 若 $\mathcal{A}$ 选择 $y = \alpha \star \tilde{x}$, 则计算 $y_1 = \alpha \star x^t$ 是容易的, 而计算 $y_0 = (\alpha \cdot g^2) \star \tilde{x}$ 是困难的; 若 $\mathcal{A}$ 选择 $y = \alpha \star x$, 则易于计算 $y_1 = \alpha \star \tilde{x}$, 难以计算 $y_0 = (\alpha \cdot g^3) \star \tilde{x}$; 若 $\mathcal{A}$ 选择 $y = \alpha \star x^t$, 则可以算出 $y_0 = \alpha \star x$, 而不容易计算 $y_1 = (\alpha \cdot g^{-2}) \star \tilde{x}$ 。

定义 2.4.9 (Sim-GA-StCDH 问题): Sim-GA-StCDH (Simultaneous Group Action Strong Computational Diffie-Hellman) 问题要求基于输入 $(w, x_0, x_1) = (h \star \tilde{x}, g_0 \star \tilde{x}, g_1 \star \tilde{x})$ 计算 $(y, y_0, y_1) \in \mathcal{X}^3$, 使得 $y_0 = (g_0^{-1} \cdot h) \star y$ 且 $y_1 = (g_1^{-1} \cdot h) \star y$ 。对于敌手 $\mathcal{A}$, 计算群操作 $\text{EGA}_x \in \{\text{EGA}, \text{REGA}\}$ 的优势为:

$$\text{ADV}_{\text{EGA}_x}^{\text{Sim-GA-StCDH}}(\mathcal{A}) := \Pr \left[\begin{array}{l} y_0 = \text{GA-CDH}_{x_0}(w, y) \\ y_1 = \text{GA-CDH}_{x_1}(w, y) \end{array} \middle| \begin{array}{l} (h, g_0, g_1) \leftarrow_{\$} \mathbb{G}^3 \\ (w, x_0, x_1) := (h \star \tilde{x}, g_0 \star \tilde{x}, g_1 \star \tilde{x}) \\ (y, y_0, y_1) \leftarrow \mathcal{A}^{\mathcal{O}}(w, x_0, x_1) \end{array} \right],$$

其中 $\mathcal{O} = \{\text{GA-DDH}_{x_0}(w, \cdot, \cdot), \text{GA-DDH}_{x_1}(w, \cdot, \cdot)\}$.

注 2.4.3: Sim-GA-StCDH 问题可以由敌手通过简单地选择 $(y, y_0, y_1) = (w^t, x_0^t, x_1^t)$ 来解决。然而, 若一个群操作不允许有效的扭曲, Sim-GA-StCDH 问题被认为是困难的。

3 基于随机同源置换的口令认证密钥协商方案

本章提出了一种基于随机同源置换的口令认证密钥协商方案，该方案在 X-GA-PAKE _{ℓ} ^[36] 的基础上进行了改进，通过随机洗牌算法的引入，减少了方案使用的群操作数量和通信开销。本章将从方案使用的安全性模型、方案主体、安全性分析与证明、性能评估对比等几个方面进行展开。本章中使用的符号定义如表 3.1 所示。

表 3.1 本章中符号及相关描述

符号	描述
U	用户的身份标识符
S	服务器的身份标识符
pw_{US}	通信双方 U 和 S 共享的口令
$\mathcal{U}$	用户 U 的命名空间
$\mathcal{S}$	服务器 S 的命名空间
$\mathcal{K}$	共享会话密钥 K 的命名空间
P	参与通信的一方，即 $P \in \{U, S\}$
π_P^i	通信方 P 拥有的一系列实例，每个实例都有各自的状态
H	随机预言机
ℓ	口令的长度
$\mathcal{PW}$	口令空间，有 $\mathcal{PW} \subseteq \{0, 1\}^\ell$
$\Pr [\text{Succ}_{\mathcal{A}}]$	敌手 $\mathcal{A}$ 成功（输出 1）的概率

3.1 安全模型

本方案使用了基于不可区分性的安全模型 BPR^[2]，并参考 Abdalla^[51] 等人的模型定义将其一系列查询进行扩展。

(1) 实例状态

实例 π_P^i 拥有的状态元组如下：

- **e**: 由实例中 P 生成的（秘密）临时值。
- **tr**: 协议执行过程中实例的活动活动轨迹（**trace**），即各方的名称和各方发送/接收的消息。

- K : 双方接受的会话密钥。
 - acc : 一个用于存储实例 π_P^i 是否接受会话密钥 K 的标志。若接受, 设置 $\pi_P^i.\text{acc} = \top$; 否则, $\pi_P^i.\text{acc} = \perp$ 。
- 为访问状态的各个组成部分, 记作 $\pi_P^i.\{e, \text{tr}, K, \text{acc}\}$ 。

(2) 伙伴关系

伙伴关系 (partnering) 通过匹配的对话进行建立。对于两个实例 $\pi_U^{t_0}$ 和 $\pi_S^{t_1}$, 他们会建立伙伴关系当且仅当:

$$\pi_U^{t_0}.\text{acc} = \pi_S^{t_1}.\text{acc} = \top \quad \text{and} \quad \pi_U^{t_0}.\text{tr} = \pi_S^{t_1}.\text{tr}.$$

定义 $\text{PAR}(\pi_U^{t_0}, \pi_S^{t_1})$ 为一个判断伙伴关系的函数, 若 $\pi_U^{t_0}$ 和 $\pi_S^{t_1}$ 是伙伴关系, 则输出 1; 否则, 输出 0。

(3) 安全性实验

在安全性实验中, 挑战者随机选择一个挑战位 α 并生成公共参数, 然后与敌手 $\mathcal{A}$ 共享。此时, $\mathcal{A}$ 可以访问的预言机如下:

- $\text{EXECUTE}(U, t_0, S, t_1)$: 用于对被动敌手进行建模。实例 $\pi_U^{t_0}$ 与 $\pi_S^{t_1}$ 之间有一个完整的协议执行。
- $\text{SENDINTT}, \text{SENDRESP}, \text{SENDTERM}$: 这类 SEND 查询用来对主动敌手进行安全建模。
- $\text{CORRUPT}(U, S)$: 输出用户 U 和服务器 S 之间的共享密钥 pw_{US} 。
- $\text{REVEAL}(P, t)$: 输出实例 π_P^t 的会话密钥。
- $\text{TEST}(P, t)$: 如果挑战位为 $\alpha = 0$, 则输出 π_P^t 的会话密钥, 否则则输出一个随机且均匀选择的密钥。如果 $\pi_P^t.\text{test} = \text{true}$, 则 π_P^t 被标记为已测试 (tested)。

(4) 新鲜性

在密码学协议的安全分析中, 新鲜性 (freshness) 是一个重要的概念, 通常用于确保协议中使用的数据或会话密钥是最新且未被泄露的。本模型中通过 $\text{Fresh}(P, i)$ 来判断一个特定的查询或会话是否新鲜。 π_P^t 被认为是新鲜的 (即 $\text{Fresh}(P, i). \pi_P^t$) 当且仅当满足以下条件:

1. 实例 π_P^t 被接受;
2. 实例 π_P^t 未进行 TEST 或 REVEAL 查询;
3. 满足以下条件之一:
 - (a) 实例 π_P^t 在 EXECUTE 查询中被接受;
 - (b) 存在多个具有伙伴关系的实例;
 - (c) 有一个唯一且新鲜的伙伴实例;
 - (d) 不存在伙伴实例, 且未进行 $\text{CORRUPT}(P, \cdot)$ 查询。

定义 3.1.1 (PAKE 的安全性): 在 PAKE 协议的实验中, 敌手 $\mathcal{A}$ 的优势定义为

$$\text{Adv}_{\text{bPAKE}}(\mathcal{A}) := |\Pr[\text{Succ}_{\mathcal{A}}] - 1/2|.$$

如果一个 PAKE 协议被认为是安全的, 那么在线字典攻击应该是敌手 $\mathcal{A}$ 能够发起的最为显著的威胁。这意味着当口令为 $\text{pw} \leftarrow_{\$} \mathcal{PW}$ 时, 敌手 $\mathcal{A}$ 的优势应当可忽略不计, 接近于 $q_s/|\mathcal{PW}|$ 。其中, q_s 表示 $\mathcal{A}$ 执行的 SEND 查询次数。

3.2 方案设计

本节将详细描述所提出的基于随机同源置换的口令认证密钥协商方案设计。为了防御离线字典攻击, 本章提出了一种基于随机同源置换的协议 R-GA-PAKE $_{\ell}$ 。“R-GA-PAKE $_{\ell}$ ”中的“R”代表“随机(Random)”。群操作的中间结果会产生一个序列, 在本方案中, 将原始序列随机替换成一个新序列并相互发送。用户和服务器恢复对方发送的序列后, 双方分别计算会话密钥, 完成密钥协商。本方案可以在抵抗离线字典攻击的同时, 保持每一方的通信开销为 $O(\ell)$ 。方案整体框架图如3.1。

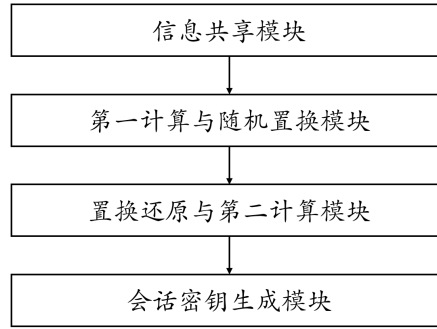


图 3.1 R-GA-PAKE $_{\ell}$ 方案的整体框架

在密钥协商的过程中, 定义通信双方为用户 U 和服务器 S , 双方共享两个元素 $\text{crs} = (x_0, x_1) \in \mathcal{X}^2$ 和口令 $\text{pw} = (b_1, \dots, b_{\ell}) \in \{0, 1\}^{\ell}$, 其中, crs 为公共信息参数, $\mathcal{X}$ 为椭圆曲线集合, x_0, x_1 为 $\mathcal{X}$ 中的两个元素, b_i 为口令中的第 i 位, l 为口令的比特长度, 口令 pw 被映射为元组 $(x_{b_1}, \dots, x_{b_{\ell}}) \in \mathcal{X}^{\ell}$ 。为执行密钥协商, 用户 U 随机生成元素 $(u_1, \dots, u_{\ell})$, 并计算 $x_i^U = u_i \star x_{b_i}$, 然后将其发送到服务器 S 。同样, $(s_1, \dots, s_{\ell})$ 是由服务器 S 随机生成的, 用于计算 $x_i^S = s_i \star x_{b_i}$ 并将其发送给用户 U 。

需要注意的是, 由于 EGATs 中引入了扭曲操作, 如果直接传输 $x_1^U, \dots, x_{\ell}^U$, 攻击者可以根据接收到的消息选择自己的消息, 从而抵消密钥派生中的某些项, 使得会话密钥不再依赖于另一方的输入。为了抵御离线字典攻击, 本方案使用随机置换算法对 $(x_1^U, \dots, x_{\ell}^U)$ 进行洗牌。 $\sigma: [\ell] \times \mathbb{Z}_q \rightarrow [\ell]$ 表示随机置换算法, 这里具体使用的是定义 (2) 介绍的 Knuth-Durstenfeld 算法。对于随机置换 σ , 将种子设为 $\text{seed}^U = H(\text{pw}|\text{root}^U)$, 其中

$root^U$ 是由 $\{x_i^U\}_\ell$ 按自然大小排序形成的 Merkle 根, 生成方式如图3.2所示。同样地, 将 $(x_1^S, \dots, x_\ell^S)$ 的顺序按种子 $seed^S = H(pw||root^S)$ 进行洗牌。定义函数 $GetRoot$ 用于计算 x^P 的 Merkle 根、 $Shuffle$ 用于执行洗牌算法、 $DeShuffle$ 用于执行洗牌还原算法。

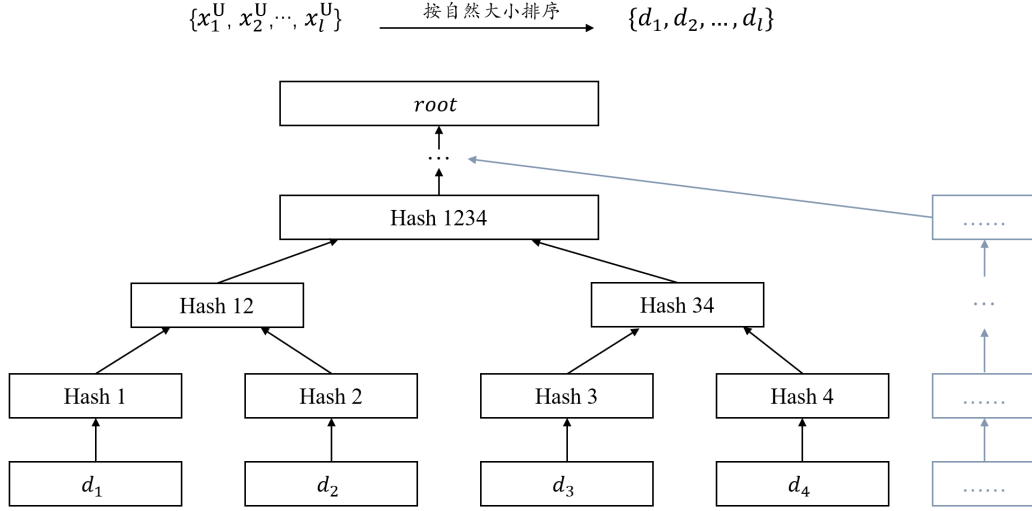


图 3.2 $GetRoot$ 函数的 Merkle 根生成方式

协议细节过程如图3.3, 其中的具体步骤如下:

1. 用户 U 和服务器 S 共享 $crs = (x_0, x_1) \in \mathcal{X}^2$ 和口令 $pw = (b_1, \dots, b_\ell) \in \{0, 1\}^\ell$;
2. 用户 U 生成随机元素 $(u_1, \dots, u_\ell) \leftarrow_{\$} \mathbb{G}^\ell$, 计算 $x_i^U = u_i \star x_{b_i}$, 并进一步计算其 Merkle 根 $root^U$ 。其中, x_{b_i} 为根据 b_i 选择的结果, x_i^U 为群元素 u_i 对 x_{b_i} 进行群操作的计算结果, $root^U$ 为由 $(x_1^U, \dots, x_\ell^U)$ 按照自然顺序生成的 Merkle 根;
3. 服务器 S 生成随机元素 $(s_1, \dots, s_\ell) \leftarrow_{\$} \mathbb{G}^\ell$, 计算 $x_i^S = s_i \star x_{b_i}$, 并进一步计算其 Merkle 根 $root^S$ 。其中, x_{b_i} 为根据 b_i 选择的结果, x_i^S 为群元素 s_i 对 x_{b_i} 进行群操作的计算结果, $root^S$ 为由 $(x_1^S, \dots, x_\ell^S)$ 按照自然顺序生成的 Merkle 根;
4. 用户 U 通过第一种子 $seed^U$ 将第一序列 x^U 进行随机置换 $Shuffle(x^U, seed^U)$, 得到第二序列 $x_{\sigma_1}^U$ 并发送给服务器 S , 其中, $seed^U = H(pw||root^U)$;
5. 服务器 S 通过第二种子 $seed^S$ 将第三序列 x^S 进行随机置换 $Shuffle(x^S, seed^S)$, 得到第四序列 $x_{\sigma_2}^S$ 并发送给用户 U , 其中, $seed^S = H(pw||root^S)$;
6. 用户 U 接收到第四序列 $x_{\sigma_2}^S$ 后, 计算其 Merkle 根 $root^S$ 和第二种子 $seed^S$, 并通过第二种子 $seed^S$ 将第四序列 $x_{\sigma_2}^S$ 进行置换还原 $DeShuffle(x_{\sigma_2}^S, seed^S)$, 得到第三序列 x^S , 并通过 $z_i = u_i \star x_i^S$ 计算用于生成会话密钥的中间计算结果;
7. 服务器 S 接收到第二序列 $x_{\sigma_1}^U$ 后, 计算其 Merkle 根 $root^U$ 和第一种子 $seed^U$, 并通过第一种子 $seed^U$ 将第二序列 $x_{\sigma_1}^U$ 进行置换还原 $DeShuffle(x_{\sigma_1}^U, seed^U)$, 得到第一序列 x^U , 并通过 $z_i = s_i \star x_i^U$ 计算用于生成会话密钥的中间计算结果;
8. 双方各自计算会话密钥为 $K = H(U, S, x^U, pw, z)$ 。

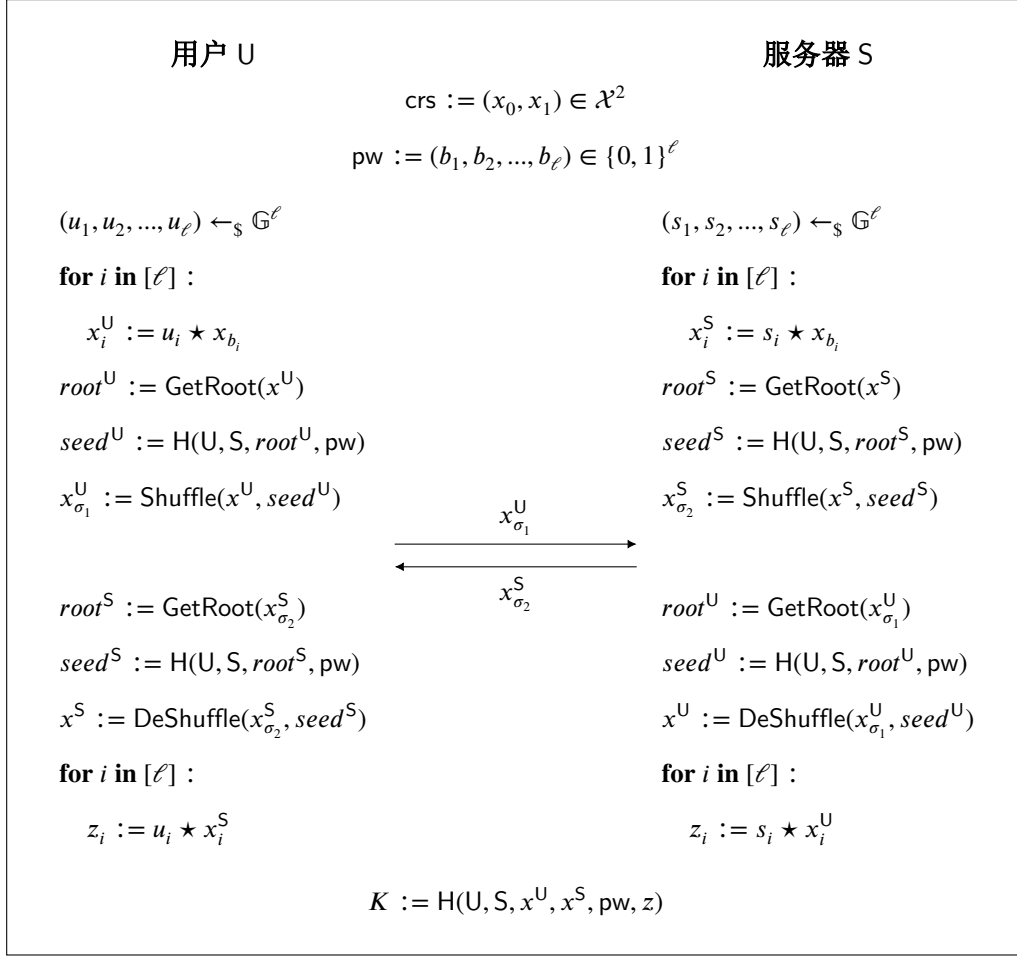

 图 3.3 R - GA - PAKE_ℓ 方案的协议流程。

图3.1中的信息共享模块包括步骤 1；第一随机置换与中间计算结果生成模块包括步骤 2 和步骤 4；第二随机置换与中间计算结果生成模块包括步骤 3 和步骤 5；会话密钥生成模块包括步骤 6、步骤 7 和步骤 8。第一计算与随机置换模块包括步骤 2、步骤 3、步骤 4 和步骤 5，置换还原与第二计算模块包括步骤 6 和步骤 7，会话密钥生成模块包括步骤 8。

3.3 安全性分析与证明

本节将基于3.1中定义的安全模型，分析 EGATs 下 R-GA-PAKE_ℓ 协议的安全性。

定理 3.3.1 (R-GA-PAKE_ℓ 的安全性): 对于任意一个对 R-GA-PAKE_ℓ 至多发起 q_e 个 EXECUTE 查询和 q_s 个 SEND 查询的敌手 $\mathcal{A}$ ，存在一个针对 GA-StCDH 的敌手 $\mathcal{B}_1$ 和一个针对 SqliInv-GA-StCDH 的敌手 $\mathcal{B}_2$ 使得

$$\text{Adv}_{\text{R-GA-PAKE}}(\mathcal{A}) \leq \text{Adv}_{\text{EGAT}}^{\text{GA-StCDH}}(\mathcal{B}_1) + \text{Adv}_{\text{EGAT}}^{\text{SqliInv-GA-StCDH}}(\mathcal{B}_2) + \frac{q_s}{|\mathcal{PW}|} + \frac{(q_s + q_e)^2}{|\mathbb{G}|^\ell}.$$

证明：定义 $\mathcal{A}$ 为一个 R-GA-PAKE_ℓ 的敌手， H 为一个随机预言机。下文将通过六个游戏展开证明，每个游戏的描述如表3.2所示。

表 3.2 证明中的游戏描述。

游戏	描述
G_0	初始游戏
G_1	增加标记 bad_{coll}
G_2	增加新鲜度变量 $\pi_p^t.\text{fr}$
G_3	修改生成 K 的方法
G_4	使用 $\tilde{x}$ 来计算 EXECUTE 中的 x^U 和 x^S
G_5	为 SENT 查询的每个新鲜实例随机选择 K
G_6	删除 SENT 查询中的口令并推迟其生成

图3.4、图3.6及图3.7详细描述了上述六个游戏的模拟，图3.5和3.8分别介绍了敌手 B_1 针对 GA-StCDH 和敌手 B_2 针对 Sim-GA-StCDH 的构造。游戏 G_0 至 G_4 如图3.4所示，其中敌手 A 可以访问的预言机为 $\mathcal{O} := \{\text{EXECUTE}, \text{SENTINIT}, \text{SENTRESP}, \text{SENTTERM}, \text{REVEAL}, \text{TEST}, \text{CORRUPT}, H\}$ 。 B_1 针对 GA-StCDH 的构造如图3.5，其中 $\mathcal{A}$ 可以访问预言机 $\mathcal{O} := \{\text{EXECUTE}, \text{SENTINIT}, \text{SENTRESP}, \text{SENTTERM}, \text{REVEAL}, \text{CORRUPT}, \text{TEST}, H\}$ ，除 EXECUTE 和 H 以外的预言机定义在图 3.4 的 G_2 中。游戏 G_5 如图3.6所示，其中敌手 A 可以访问的预言机和游戏 G_0 至 G_4 保持一致， EXECUTE 、 SENTINIT 、 REVEAL 、 CORRUPT 和 TEST 的定义如图 3.4 中的 G_4 。游戏 G_6 如图3.7所示，相较于游戏 G_5 ，该游戏中敌手 A 可以访问的预言机新增了 H_2 ， REVEAL 、 TEST 和 EXECUTE 的定义如图 3.4 中的 G_4 。敌手 B_2 针对 Sim-GA-StCDH 的构造如图3.8所示，其中敌手 $\mathcal{A}$ 可以访问图 3.7 中定义的所有预言机 $\mathcal{O}$ ， REVEAL 和 TEST 的定义如图 3.4 所示， EXECUTE 的定义如图 3.6 所示。 CORRUPT 和 H_2 的定义如图 3.7 所示。下面对每个游戏进行分析：

游戏 G_0 。作为初始游戏，敌手 $\mathcal{A}$ 的优势为

$$\text{Adv}_{\text{R-GA-PAKE}} \leq |\Pr [\text{Succ}_0] - 1/2|.$$

游戏 G_1 。在这个游戏中，若一个实例计算了一个已被接受的活动轨迹（第67、82行）或者在查询中计算的活动轨迹与任何已被接受的实例发生冲突（第28行），设置一个标记 bad_{coll} ，并令查询返回 $\perp$ 。由差分定理可得，

$$|\Pr [\text{Succ}_1] - \Pr [\text{Succ}_0]| \leq \Pr [\text{bad}_{\text{coll}}].$$

若 $\text{bad}_{\text{coll}} = \text{false}$ ，每个实例都是不同的，只有至多一个伙伴。对于一个实例 π_p^t ，其活动轨迹由 (U, S, x^U, x^S) 构成，其中 U 和 S 均由游戏选择。因此，只有当一个实例的所有 ℓ

- 29 -

个集合元素与另一个实例碰撞时, $\mathbf{bad}_{\text{coll}}$ 才会发生。两个会话之间发生这种情况的概率是 $1/|\mathbb{G}|^\ell$, 由于最多有 $q_e + q_s$ 个会话, 所以

$$|\Pr[\text{Succ}_1] - \Pr[\text{Succ}_0]| \leq \Pr[\mathbf{bad}_{\text{coll}}] \leq \frac{(q_e + q_s)^2}{|\mathbb{G}|^\ell}.$$

游戏 G_2 。在这个游戏中, 新添了变量 $\pi_p^t.\text{fr}$, 用来记录实例 π_p^t 的新鲜度 (freshness)。 $\pi_p^t.\text{fr}$ 会在一下几种情况下更新:

- 在 EXECUTE 查询返回前, 设置 $\pi_p^t.\text{fr} = \mathbf{true}$ (第35行)。
- 在 SENDRESP 和 SENDTERM 查询中, 如果口令没有被 CORRUPT 返回, 设置 $\pi_p^t.\text{fr} = \mathbf{true}$ (第71、86行); 否则, 设置 $\pi_p^t.\text{fr} = \mathbf{false}$ (第72、89行)。
- 在 SENDTERM 查询中, 伙伴实例的新鲜度也会被检查 (第85行)。若实例在敌手 $\mathcal{A}$ 访问 CORRUPT 时便已经生成, 设置 $\pi_p^t.\text{fr} = \mathbf{false}$ 。
- 在 REVEAL 查询中, 当一个实例泄露了其会话密钥, 将该实例及其伙伴实例标记为非新鲜实例 (第41行)。

另外, 在 TEST 查询中, 只对实例的新鲜性进行检查 (第44行)。 G_2 和 G_1 之间只有概念上的区别, 因此

$$\Pr[\text{Succ}_2] = \Pr[\text{Succ}_1].$$

游戏 G_3 。在本游戏中, 修改了生成 K 的方法。在查询执行的实例时, 随机选择密钥 $K \leftarrow_{\mathcal{S}} \mathcal{K}$, 而不是通过计算生成。在这里, 我们构建了一个敌手 B_1 来对抗 GA-StCDH, 并证明 $\Pr[\text{Succ}_3]$ 和 $\Pr[\text{Succ}_2]$ 之间的差异由 $\text{Adv}_{\text{EGAT}}^{\text{GA-StCDH}}(B_1)$ 界定。攻击者 B_1 将挑战 $(x, y) = (g \star \tilde{x}, h \star \tilde{x})$ 作为输入, 并访问 GA-DDH($x, \cdot, \cdot$)。为开始攻击, B_1 像 G_2 一样生成 crs, 并将 (x_0, x_1) 作为输入来运行 $\mathcal{A}$ 。对于 EXECUTE, B_1 模拟以下过程: B_1 为每一方随机选择 $(u_0, \dots, u_\ell)$ 和 $(s_0, \dots, s_\ell)$ 。 B_1 不使用 x_{b_i} , 而是使用 x 和 y 来计算 x^U 和 x^S , 这意味着不需要口令 $\text{pw} := (b_1, \dots, b_\ell)$ 。这个过程可以重新表述为

$$x_i^U = u_i \star x = (u_i \cdot g \cdot g_{b_i} \cdot g_{b_i}^{-1}) \star \tilde{x} = (u_i \cdot g \cdot g_{b_i}^{-1}) \star x_{b_i} = u_i' \star x_{b_i},$$

其中 $u_i' = u_i \cdot g \cdot g_{b_i}^{-1}$ 被设置为用于获取用户会话密钥的群元素。类似地, 设置 $s_i' = s_i \cdot h \cdot g_{b_i}^{-1}$ 。因此, 可得 $z_i = (u_i' \cdot s_i') \star x_{b_i} = (u_i \cdot g) \cdot (s_i \cdot h) \cdot (g_{b_i}^{-1} \star \tilde{x})$ 。

在随机选择会话密钥之前, 检查是否存在与该会话密钥匹配的值被随机预言机 H 查询过。遍历 Q 中与元组 $(U, S, x^U, x^S, \text{pw}_{US})$ 匹配的条目, 并验证 z 中是否有正确的条目。此时, 下面的等价关系成立:

$$\text{GA-CDH}_{x_{b_i}}(x_i^U, x_i^S) = z_i \Leftrightarrow \text{GA-CDH}(x, x_i^S) = (u_i^{-1} \cdot g_{b_i}) \star z_i.$$

通过这种方式, 可以使用 GA-DDH($x, \cdot, \cdot$), 其中第一个输入被限制为 x 。

$B_1^{\text{GA-DDH}(x, \cdot)}(x, y)$	$\text{EXECUTE}(U, t_0, S, t_1)$
01 $(g_0, g_1) \leftarrow_{\mathcal{S}} \mathbb{G}^2$ 02 $(x_0, x_1) := (g_0 \star \tilde{x}, g_1 \star \tilde{x})$ 03 $(C, Q, Q_e) := (\emptyset, \emptyset, \emptyset)$ 04 bad_{coll} := false 05 for $(U, S) \in \mathcal{U} \times \mathcal{S}$ 06 $\text{pw}_{US} \leftarrow_{\mathcal{S}} \mathcal{PW}$ 07 $\beta \leftarrow_{\mathcal{S}} \{0, 1\}$ 08 $\beta' \leftarrow_{\mathcal{S}} \mathcal{A}^O(x_0, x_1)$ 09 Stop . $H(U, S, x^U, x^S, \text{pw}, z)$	19 if $\pi_U^{t_0} \neq \perp$ or $\pi_S^{t_1} \neq \perp$ return $\perp$ 20 $(b_1, \dots, b_{\ell}) := \text{pw}_{US}$ 21 $u := (u_1, \dots, u_{\ell}) \leftarrow_{\mathcal{S}} \mathbb{G}^{\ell}$ 22 $s := (s_1, \dots, s_{\ell}) \leftarrow_{\mathcal{S}} \mathbb{G}^{\ell}$ 23 $x^U := (x_1^U, \dots, x_{\ell}^U) := (u_1 \star x, \dots, u_{\ell} \star x)$ 24 $x^S := (x_1^S, \dots, x_{\ell}^S) := (s_1 \star y, \dots, s_{\ell} \star y)$ 25 $\text{root}^U := \text{GetRoot}(x^U)$ 26 $x_{\sigma_1}^U := \text{Shuffle}(x^U, H(U, S, \text{root}^U, \text{pw}))$ 27 $\text{root}^S := \text{GetRoot}(x^S)$ 28 $x_{\sigma_2}^S := \text{Shuffle}(x^S, H(U, S, \text{root}^S, \text{pw}))$ 29 if $\exists P \in \mathcal{U} \cup \mathcal{S}, t' \text{ s.t. } \pi_P^{t'}.tr = (U, S, x^U, x^S)$ 30 bad_{coll} := true 31 return $\perp$ 32 for $\forall z \text{ s.t. } (U, S, x^U, x^S, \text{pw}_{US}, z) \in Q$ 33 for $i \in [\ell]$ 34 if $\text{GA-DDH}(x, x_i^S, (u_i^{-1} \cdot g_{b_i}) \star z) = 1$ 35 Stop with $(u_i^{-1} \cdot s_i^{-1} \cdot g_{b_i}) \star z$ 36 $Q_e := Q_e \cup \{U, S, x^U, x^S, \text{pw}_{US}, u, s\}$ 37 $K \leftarrow_{\mathcal{S}} \mathcal{K}$ 38 $\pi_U^{t_0} := (u, (U, S, x^U, x^S), K, \text{true})$ 39 $\pi_S^{t_1} := (s, (U, S, x^U, x^S), K, \text{true})$ 40 $(\pi_U^{t_0}.fr, \pi_S^{t_1}.fr) := (\text{true}, \text{true})$ 41 return $(U, x_{\sigma_1}^U, S, x_{\sigma_2}^S)$

 图 3.5 定理 3.3.1 证明中敌手 B_1 针对 GA-StCDH 的构造。

如果找到正确的条目， B_1 终止循环并通过 $(u_i^{-1} \cdot s_i^{-1} \cdot g_{b_i}) \star z_i$ 获得输出 $(g \cdot h) \star \tilde{x}$ 。否则，将活动轨迹、口令和值 u_i, s_i 添加到 Q_e ，以标识 H 的相关查询，然后随机选择 $K \leftarrow_{\mathcal{S}} \mathcal{K}$ 。需要注意的是，如果查询包含活动轨迹 $\pi_P^{t'}.tr$ 和 pw ，将检索 u_i 和 s_i ，以便使用 DDH 预言机来验证 z_i 。若预言机对所有的 z_i 返回 $\perp$ ，那么 B_1 将中止并产生输出 $(g \cdot h) \star \tilde{x}$ 。

游戏 G_4 。在这个游戏中，没有使用 x_{b_i} 作为基元，而是简单地使用 $\tilde{x}$ 来计算 EXECUTE 中的 x^U 和 x^S ，这意味着从这个查询中删除了口令。在前面的游戏中， pw 被设置为洗牌算法使用的种子的一部分。而此游戏中，将 $\mathcal{P}$ 表示为置换的命名空间，并简单地将 Shuffle 函数替换为随机置换函数 PRP。这些值与之前的游戏中的值具有的分布保持一致。此外， u 和 s 不涉及密钥派生。因此，敌手 $\mathcal{A}$ 无法区分游戏和 $\Pr[\text{Succ}_4] = \Pr[\text{Succ}_3]$ 之间的区别。

游戏 G_5 。图 3.6 中显示了 G_5 的流程。在这个游戏中，对于查询 SENTRESP 和 SENTTERM 的每个新鲜实例，随机选择 K 。在这里，新引入一个独立的随机预言机 Q_s ，它通过将跟踪映射到实例 U 或 S 的键，以确保伙伴实例之间的一致性。在攻击者 $\mathcal{A}$ 向 SENTTERM 查询实例和新存在的合作伙伴实例的情况下，我从 Q_s 获取 K 并将其提供给实例。如果实例不是新鲜的，通过随机预言机 H 计算正确的 K 。

对于所有的新会话，如果 Q 和 Q_s 不完全相同，标记 **bad** = **true**。上述情况发生在：

```

game  $G_5$ 
01  $(g_0, g_1) \leftarrow_{\mathcal{S}} \mathbb{G}^2$ 
02  $(x_0, x_1) := (g_0 \star \tilde{x}, g_1 \star \tilde{x})$ 
03  $(C, Q, Q_s) := (\emptyset, \emptyset, \emptyset)$ 
04 bad := false
05  $\alpha \leftarrow_{\mathcal{S}} \{0, 1\}$ 
06 for  $U \in \mathcal{U}$  and  $S \in \mathcal{S}$ 
07    $\text{pw}_{US} \leftarrow_{\mathcal{S}} \mathcal{PW}$ 
08    $\alpha' \leftarrow \mathcal{A}^{\theta}(x_0, x_1)$ 
09   return  $\llbracket \alpha = \alpha' \rrbracket$ 
SENDRESP( $S, t, U, x_{\sigma_1}^U$ )
10 if  $\pi_S^t \neq \perp$  return  $\perp$ 
11  $(b_1, \dots, b_{\ell}) := \text{pw}_{US}$ 
12  $s := (s_1, \dots, s_{\ell}) \leftarrow_{\mathcal{S}} \mathbb{G}^{\ell}$ 
13  $x^S := (x_1^S, \dots, x_{\ell}^S) := (s_1 \star x_{b_1}, \dots, s_{\ell} \star x_{b_{\ell}})$ 
14  $\text{root}^S := \text{GetRoot}(x^S)$ 
15  $x_{\sigma_2}^S := \text{Shuffle}(x^S, H(U, S, \text{root}^S, \text{pw}))$ 
16  $\text{root}^U := \text{GetRoot}(x_{\sigma_1}^U)$ 
17  $x^U := \text{DeShuffle}(x_{\sigma_1}^U, H(U, S, \text{root}^U, \text{pw}))$ 
18 if  $\exists P \in \mathcal{U} \cup S, t' \text{ s.t. } \pi_P^{t'}.tr = (U, S, x^U, x^S)$ 
19   return  $\perp$ 
20 if  $U \notin C$  and  $S \notin C$ 
21    $\pi_S^t.fr := \text{true}$ 
22   if  $\exists z \text{ s.t. } (U, S, x^U, x^S, \text{pw}_{US}, z) \in Q$ 
23     and  $z_i := s_i \star x_i^U \forall i \in [\ell]$ 
24     bad := true
25      $K \leftarrow_{\mathcal{S}} \mathcal{K}$ 
26      $Q_s[U, S, x^U, x^S] := (S, s, K)$ 
27   else
28      $\pi_S^t.fr := \text{false}$ 
29     for  $i \in [\ell]$ 
30        $z_i := s_i \star x_i^U$ 
31      $z := (z_1, \dots, z_{\ell})$ 
32      $K := H[U, S, x^U, x^S, \text{pw}_{US}, z]$ 
33      $\pi_S^t := (s, (U, S, x^U, x^S), K, \text{true})$ 
34   return  $(S, x_{\sigma_2}^S)$ 
SENDTERM( $U, t, S, x_{\sigma_2}^S$ )
34 if  $\pi_U^t \neq (u, (U, S, x^U, \perp), \perp, \perp)$  return  $\perp$ 
35  $\text{root}^S := \text{GetRoot}(x_{\sigma_2}^S)$ 
36  $x^S := \text{DeShuffle}(x_{\sigma_2}^S, H(U, S, \text{root}^S, \text{pw}))$ 
37 if  $\exists P \in \mathcal{U}, t' \text{ s.t. } \pi_P^{t'}.tr = (U, S, x^U, x^S)$ 
38   return  $\perp$ 
39 if  $\exists t' \text{ s.t. } \pi_S^{t'}.tr = (U, S, x^U, x^S)$  and  $\pi_S^{t'}.fr = \text{true}$ 
40    $\pi_U^t.fr = \text{true}$ 
41    $(S, s, K) := Q_s[U, S, x^U, x^S]$ 
42 else if  $U \notin C$  and  $S \notin C$ 
43    $\pi_U^t.fr = \text{true}$ 
44   if  $\exists z \text{ s.t. } (U, S, x^U, x^S, \text{pw}_{US}, z) \in Q$ 
45     and  $z_i := u_i \star x_i^S \forall i \in [\ell]$ 
46     bad := true
47      $K \leftarrow_{\mathcal{S}} \mathcal{K}$ 
48      $Q_s[U, S, x^U, x^S] := (U, u, K)$ 
49   else
50      $\pi_U^t.fr := \text{false}$ 
51     for  $i \in [\ell]$ 
52        $z_i := u_i \star x_i^S$ 
53      $z := (z_1, \dots, z_{\ell})$ 
54      $K := H[U, S, x^U, x^S, \text{pw}_{US}, z]$ 
55      $\pi_U^t := (u, (U, S, x^U, x^S), K, \text{true})$ 
56   return true
H( $U, S, x^U, x^S, \text{pw}, z$ )
56 if  $Q[U, S, x^U, x^S, \text{pw}, z] = K \neq \perp$  return  $K$ 
57 if  $(U, S, x^U, x^S) \in Q_s$  and  $\text{pw} = \text{pw}_{US}$ 
58   if  $Q_s[U, S, x^U, x^S] = (U, u, K)$ 
59     for  $i \in [\ell]$ 
60        $z'_i := u_i \star x_i^S$ 
61      $z' := (z'_1, \dots, z'_{\ell})$ 
62     if  $Q_s[U, S, x^U, x^S] = (S, s, K)$ 
63       for  $i \in [\ell]$ 
64          $z'_i := s_i \star x_i^U$ 
65        $z' := (z'_1, \dots, z'_{\ell})$ 
66     if  $z = z'$ 
67       if  $U \in C$  and  $S \in C$  return  $K$ 
68       if  $U \notin C$  and  $S \notin C$ 
69         bad := true
70      $Q[U, S, x^U, x^S, \text{pw}, z] \leftarrow_{\mathcal{S}} \mathcal{K}$ 
71   return  $Q[U, S, x^U, x^S, \text{pw}, z]$ 
    
```

 图 3.6 用于证明定理 3.3.1 的游戏 G_5 。

1. 在 **SENTRESP** 查询中，当实例即将计算会话密钥时，口令没有损坏，但是在 Q （22到23行）中已经存在正确的口令和 z 。
2. 在 **SENDTERM** 查询中，当实例即将计算会话密钥时，不存在伙伴实例，口令也没有损坏，但是在 Q （44到45行）中已经存在正确的口令和 z 。
3. 在随机的预言机 H 中，将查询 Q_s 中存在的一些跟踪以及正确的口令和 z （56到69行）。在这种情况下，如果口令已损坏，并且攻击者发出了正确的查询，输出存储在 Q_s （67行）中的密钥 K 。

当 **bad** 未发生时， G_4 与 G_5 相同。因此，

$$|\Pr[\text{Succ}_5] - \Pr[\text{Succ}_4]| \leq \Pr[G_5 \Rightarrow \text{bad}].$$

游戏 G_6 。在这个游戏中，删除了 **SENT** 查询中的口令，并将其生成时间尽可能推迟。

game G_6

```

01  $(g_0, g_1) \leftarrow_{\mathcal{S}} \mathbb{G}^2$ 
02  $(x_0, x_1) := (g_0 \star \tilde{x}, g_1 \star \tilde{x})$ 
03  $(C, Q, Q_s, Q_{\text{bad}}, Q_{\sigma}) := (\emptyset, \emptyset, \emptyset, \emptyset, \emptyset)$ 
04  $(\text{bad}_{\text{corr}}, \text{bad}_{\text{pw}}) := (\text{false}, \text{false})$ 
05  $\alpha \leftarrow_{\mathcal{S}} \{0, 1\}$ 
06  $\alpha' \leftarrow \mathcal{A}^{\theta}(x_0, x_1)$ 
07 for  $U \in \mathcal{U} \setminus C$  and  $S \in \mathcal{S} \setminus \mathcal{C}$ 
08    $\text{pw}_{\text{US}} \leftarrow_{\mathcal{S}} \mathcal{PW}$ 
09   if  $\exists \text{pw}, \text{pw}', (U, S, x^U, x^S, z, z')$ 
10     s.t.  $(U, S, x^U, x^S, \text{pw}, z) \in Q_{\text{bad}}$ 
11     and  $(U, S, x^U, x^S, \text{pw}', z') \in Q_{\text{bad}}$ 
12      $\text{bad}_{\text{pw}} := \text{true}$ 
13   else
14     if  $\exists U, S, x^U, x^S, z$  s.t.  $(U, S, x^U, x^S, \text{pw}_{\text{US}}, z)$ 
15        $\text{bad}_{\text{guess}} := \text{true}$ 
16   return  $\llbracket \alpha = \alpha' \rrbracket$ 
CORRUPT( $U, S$ )
15 if  $U \in C$  and  $S \in C$  return  $\perp$ 
16 for  $P \in \{U, S\}$ 
17   if  $\exists t$  s.t.  $\pi_P^t \cdot \text{test} = \text{true}$ 
18     and  $\exists P' \in \mathcal{U} \cup \mathcal{S}, t'$  s.t.  $\text{PAR}(\pi_P^t, \pi_{P'}^{t'}) = 1$ 
19     return  $\perp$ 
20    $\forall \pi_P^t$  : if  $\exists P' \in \mathcal{U} \cup \mathcal{S}, t'$  s.t.  $\text{PAR}(\pi_P^t, \pi_{P'}^{t'}) = 1$ 
21      $\pi_P^t \cdot \text{fr} = \text{false}$ 
22    $C := C \cup \{(U, S)\}$ 
23    $\text{pw}_{\text{US}} \leftarrow_{\mathcal{S}} \mathcal{PW}$ 
24   return  $\text{pw}_{\text{US}}$ 
SENDINIT( $U, t, S$ )
24 if  $\pi_U^t \neq \perp$  return  $\perp$ 
25  $u := (u_1, \dots, u_{\ell}) \leftarrow_{\mathcal{S}} \mathbb{G}^{\ell}$ 
26  $x^U := (x_1^U, \dots, x_{\ell}^U) := (u_1 \star \tilde{x}, \dots, u_{\ell} \star \tilde{x})$ 
27  $\text{root}^U := \text{GetRoot}(x^U)$ 
28  $\sigma_1 := H_2(U, S, \text{root}^U, \text{pw})$ 
29  $x_{\sigma_1}^U = F_{\sigma_1}(x^U)$ 
30  $\pi_U^t := (u, (U, S, x^U, \perp), \perp, \perp)$ 
31  $\pi_U^t \cdot \text{fr} := \text{false}$ 
32 return  $(U, x_{\sigma_1}^U)$ 
SENDRESP( $S, t, U, x_{\sigma_1}^U$ )
33 if  $\pi_S^t \neq \perp$  return  $\perp$ 
34  $s := (s_1, \dots, s_{\ell}) \leftarrow_{\mathcal{S}} \mathbb{G}^{\ell}$ 
35  $x^S := (x_1^S, \dots, x_{\ell}^S) := (s_1 \star \tilde{x}, \dots, s_{\ell} \star \tilde{x})$ 
36  $\text{root}^S := \text{GetRoot}(x^S)$ 
37  $\sigma_2 := H_2(U, S, \text{root}^S, \text{pw})$ 
38  $x_{\sigma_2}^S = F_{\sigma_2}(x^S)$ 
39  $\text{root}^U := \text{GetRoot}(x_{\sigma_1}^U)$ 
40  $\sigma_2 := H_2(U, S, \text{root}^U, \text{pw})$ 
41  $x^U = F_{\sigma_2}^{-1}(x_{\sigma_1}^U)$ 
42 if  $\exists P \in \mathcal{U} \cup \mathcal{S}, t'$  s.t.  $\pi_P^{t'} \cdot \text{tr} = (U, S, x^U, x^S)$ 
43   return  $\perp$ 
44 if  $U \notin C$  and  $S \notin C$ 
45    $\pi_S^t \cdot \text{fr} := \text{true}$ 
46   if  $\forall \text{pw}, z$  s.t.  $(U, S, x^U, x^S, \text{pw}, z) \in Q$ 
47      $(b_1, \dots, b_{\ell}) := \text{pw}$ 
48     for  $i \in [\ell]$ 
49        $z'_i := s_i \cdot g_{b_i}^{-1} \star x_i^U$ 
50      $z' := (z'_1, \dots, z'_{\ell})$ 
51     if  $z = z'$ 
52        $Q_{\text{bad}} := Q_{\text{bad}} \cup \{(U, S, x^U, x^S, \text{pw}, z)\}$ 
53      $K \leftarrow_{\mathcal{S}} \mathcal{K}$ 
54      $Q_s[U, S, x^U, x^S] := (S, s, K)$ 
55   else

```

```

56    $\pi_S^t \cdot \text{fr} := \text{false}$ 
57    $(b_1, \dots, b_{\ell}) := \text{pw}_{\text{US}}$ 
58   for  $i \in [\ell]$ 
59      $z_i := s_i \cdot g_{b_i}^{-1} \star x_i^U$ 
60    $z := (z_1, \dots, z_{\ell})$ 
61    $K := H(U, S, x^U, x^S, \text{pw}_{\text{US}}, z)$ 
62    $\pi_S^t := (s, (U, S, x^U, x^S, K), \text{true})$ 
63   return  $(S, x_{\sigma_2}^S)$ 
SENDTERM( $U, t, S, x_{\sigma_2}^S$ )
64 if  $\pi_U^t \neq (u, (U, S, x^U, \perp), \perp, \perp)$  return  $\perp$ 
65  $\text{root}^S := \text{GetRoot}(x_{\sigma_2}^S)$ 
66  $\sigma_2 := H_2(U, S, \text{root}^S, \text{pw})$ 
67  $x^S = F_{\sigma_2}^{-1}(x_{\sigma_2}^S)$ 
68 if  $\exists P \in \mathcal{U}, t'$  s.t.  $\pi_P^{t'} \cdot \text{tr} = (U, S, x^U, x^S)$  return  $\perp$ 
69 if  $\exists t'$  s.t.  $\pi_S^{t'} \cdot \text{tr} = (U, S, x^U, x^S)$  and  $\pi_S^{t'} \cdot \text{fr} = \text{true}$ 
70    $\pi_U^t \cdot \text{fr} = \text{true}$ 
71    $(S, s, K) := Q_s[U, S, x^U, x^S]$ 
72 else if  $U \notin C$  and  $S \notin C$ 
73    $\pi_U^t \cdot \text{fr} = \text{true}$ 
74    $\forall \text{pw}, z$  s.t.  $(U, S, x^U, x^S, \text{pw}, z) \in Q$ 
75      $(b_1, \dots, b_{\ell}) := \text{pw}$ 
76     for  $i \in [\ell]$ 
77        $z'_i := u_i \cdot g_{b_i}^{-1} \star x_i^S$ 
78      $z' := (z'_1, \dots, z'_{\ell})$ 
79     if  $z = z'$ 
80        $Q_{\text{bad}} := Q_{\text{bad}} \cup \{(U, S, x^U, x^S, \text{pw}, z)\}$ 
81      $K \leftarrow_{\mathcal{S}} \mathcal{K}$ 
82      $Q_s[U, S, x^U, x^S] := (U, u, K)$ 
83   else
84      $\pi_U^t \cdot \text{fr} := \text{false}$ 
85      $(b_1, \dots, b_{\ell}) := \text{pw}_{\text{US}}$ 
86     for  $i \in [\ell]$ 
87        $z_i := u_i \cdot g_{b_i}^{-1} \star x_i^S$ 
88      $z := (z_1, \dots, z_{\ell})$ 
89      $K := H(U, S, x^U, x^S, \text{pw}_{\text{US}}, z)$ 
90      $\pi_U^t := (u, (U, S, x^U, x^S, K), \text{true})$ 
91   return true
H( $U, S, x^U, x^S, \text{pw}, z$ )
92 if  $Q[U, S, x^U, x^S, \text{pw}, z] = K \neq \perp$  return  $K$ 
93 if  $(U, S, x^U, x^S) \in Q_s$ 
94    $(b_1, \dots, b_{\ell}) := \text{pw}$ 
95   if  $Q_s[U, S, x^U, x^S] = (U, u, K)$ 
96     for  $i \in [\ell]$ 
97        $z'_i := u_i \cdot g_{b_i}^{-1} \star x_i^S$ 
98      $z' := (z'_1, \dots, z'_{\ell})$ 
99     if  $Q_s[U, S, x^U, x^S] = (S, s, K)$ 
100       for  $i \in [\ell]$ 
101          $z'_i := s_i \cdot g_{b_i}^{-1} \star x_i^U$ 
102        $z' := (z'_1, \dots, z'_{\ell})$ 
103     if  $z = z'$ 
104       if  $U \in C$  and  $S \in C$  and  $\text{pw} = \text{pw}_{\text{US}}$  : return  $K$ 
105       if  $U \notin C$  and  $S \notin C$ 
106          $Q_{\text{bad}} := Q_{\text{bad}} \cup \{(U, S, x^U, x^S, \text{pw}, z)\}$ 
107        $Q[U, S, x^U, x^S, \text{pw}, z] \leftarrow_{\mathcal{S}} \mathcal{K}$ 
108     return  $Q[U, S, x^U, x^S, \text{pw}, z]$ 
H2( $U, S, \text{root}, \text{pw}$ )
109 if  $Q_{\sigma}[U, S, \text{root}, \text{pw}] = \sigma \neq \perp$  return  $\sigma$ 
110 if  $U \in C$  and  $S \in C$  and  $\text{pw} = \text{pw}_{\text{US}}$ 
111    $\sigma := \text{Shuffle}(\perp, H(U, S, \text{root}, \text{pw}))$ 
112    $Q_{\sigma}[U, S, \text{root}, \text{pw}] := \sigma$ 
113   return  $\sigma$ 
114    $Q_{\sigma}[U, S, \text{root}, \text{pw}] \leftarrow_{\mathcal{S}} \mathcal{P}$ 
115   return  $Q_{\sigma}[U, S, \text{root}, \text{pw}]$ 

```

 图 3.7 用于证明定理 3.3.1 的游戏 G_6 。

具体来说, 口令是在攻击者发出 **CORRUPT** 查询 (8行) 或返回结果 β' (22行) 时生成的。在 **SENTINIT** 和 **SENTRESP** 查询中, 群元素 u_i 和 s_i 是被均匀随机选择的, 但 x_i^U 和 x_i^S 的计算变为使用源元素 $\tilde{x}$ (行26和35)。那么, 根据口令的选择, 消息被设置为

$$\begin{aligned} x_i^U &= u_i \cdot \tilde{x} = (u_i \cdot g_{b_i}^{-1}) \star x_{b_i} = (u_i \cdot g_0^{-1}) \star x_0 = (u_i \cdot g_1^{-1}) \star x_1, \\ x_i^S &= s_i \cdot \tilde{x} = (s_i \cdot g_{b_i}^{-1}) \star x_{b_i} = (s_i \cdot g_0^{-1}) \star x_0 = (s_i \cdot g_1^{-1}) \star x_1. \end{aligned}$$

由于无法使用口令执行洗牌, 添加一个新表 Q_σ 来存储游戏中的置换顺序。在打乱 x^U 和 x^S 的顺序之前, 从随机预言机 H_2 (109到115行) 获得查询结果 σ 。若口令已被损坏, 且攻击者发出了正确的查询, 只需计算并输出正确的 σ ; 否则, 返回在 Q_σ 中记录的值, 或者随机生成一个 σ 并将其记录到表中。此外, 定义一个新函数 $F_\sigma(x)$, 该函数可以根据 σ 及其对应的逆函数 $F_\sigma^{-1}(x)$ 对输入序列 x 进行排列。此外, 游戏中需要使用口令的置换与还原函数 **Shuffle** 和 **DeShuffle** (27到29, 36到41, 65到67行) 也需做出相应调整。对于所有非新鲜实例, 使用 $z_i = s_i \cdot g_{b_i}^{-1} \star x_i^U$ (59行) 或 $z_i = u_i \cdot g_{b_i}^{-1} \star x_i^S$ (87行) 计算正确的会话密钥。需注意的是, 口令已经在这些实例的 **CORRUPT** 查询中生成了。

回顾游戏 G_5 , 当随机预言机查询结果与新鲜实例的密钥不匹配时, 就会引发事件 **bad**。在 G_6 中, 我们 **bad** 事件分成以下两种情况:

1. **bad_{pw}** 用于捕获以下事件: 对于新鲜实例的相同活动轨迹, 在 Q 中存在多个有效条目, 但使用不同的口令。
2. **bad_{guess}** 用于捕获 **bad_{pw}** 没有引发的事件, 并且在 Q 中存在具有新鲜实例活动轨迹和正确口令的有效条目。在此情况下, 查询 H 时, 口令没有被损坏。

为识别这两个事件, 添加一个新的集合 Q_{bad} 。在 **SENTRESP** 和 **SENTTERM** 中, 遍历 Q 中每个新鲜实例包含的相应活动轨迹的所有条目。通过计算正确的值 z' 来验证给定口令和 z 的有效性, 方法与针对非新鲜实例的相同。如果 $z = z'$, 将条目添加到 Q_{bad} (46到52, 74到80行)。当在 Q_s 中出现的活动轨迹中查询随机预言机 H 时, 操作相同。在这种情况下, 攻击者指定口令, 我们使用存储在 Q_s 中的实例 u_i 或 s_i 来检查 z 的有效性。如果 z 有效并且实例是新鲜实例, 将查询添加到 Q_{bad} (93到106行)。如果在此期间口令已损坏, 则返回存储在 Q_s 中的口令。

在攻击者输出结果 β' 之后, 检查事件 **bad_{pw}** (10行) 或 **bad_{guess}** (13行) 是否被触发。这里需要注意的是, 只要在 G_5 中设置了 **bad**, 就会相应地在 G_6 中设置 **bad_{pw}** 或 **bad_{guess}** 标志。因此,

$$\Pr [G_5 \Rightarrow \mathbf{bad}] \leq \Pr [G_6 \Rightarrow \mathbf{bad}_{\text{pw}}] + \Pr [G_6 \Rightarrow \mathbf{bad}_{\text{guess}}].$$

接下来, 计算这两个事件发生的概率。首先是 **bad_{pw}**, 针对模拟 G_6 的 **Sim - GA - StCDH** 构造攻击者 B_2 。 B_2 输入 (x_0, x_1, w) 。其中 $x_0 = g_0 \star \tilde{x}$ 、 $x_1 = g_1 \star \tilde{x}$ 和 $w = h \star \tilde{x}$, 群元素 g_0, g_1, h 从 G 中均匀随机选择。此外, B_2 可以访问预言机 $\text{GA - DDH}_{x_j}(w, \cdot, \cdot)$, 以

```

 $B_2^{\{GA-DDH_{x_j}(w, \cdot)\}_{j \in \{0,1\}}}(x_0, x_1, w)$ 
01  $(C, Q, Q_s, Q_{bad}, Q_\sigma) := (\emptyset, \emptyset, \emptyset, \emptyset, \emptyset)$ 
02  $(bad_{corr}, bad_{pw}) := (false, false)$ 
03  $\alpha \leftarrow_{\$} \{0, 1\}$ 
04  $\alpha' \leftarrow \mathcal{A}^{\mathcal{O}}(x_0, x_1)$ 
05 for  $U \in \mathcal{U} \setminus C$  and  $S \in \mathcal{S} \setminus C$ 
06    $pw_{US} \leftarrow_{\$} \mathcal{PW}$ 
07   if  $\exists pw, pw', (U, S, x^U, x^S, z, z')$ 
    s.t.  $(U, S, x^U, x^S, pw, z) \in Q_{bad}$ 
    and  $(U, S, x^U, x^S, pw', z') \in Q_{bad}$ 
08      $(b_1, \dots, b_\ell) := pw$ 
09      $(b_1, \dots, b_\ell) := pw'$ 
10     Find first index  $i$  such that  $b_i \neq b_i$ 
11     W.l.o.g. let  $b_i = 0, b_i = 1$ 
12     if  $Q_s[U, S, x^U, x^S] = (U, u, K)$ 
13       Stop with  $(x_i^S, u_i^{-1} \star z_i)$ 
14     if  $Q_i[U, S, x^U, x^S] = (S, s, K)$ 
15       Stop with  $(x_i^U, s_i^{-1} \star z_i)$ 
SENDINIT( $U, t, S$ )
16 if  $\pi_U^t \neq \perp$  return  $\perp$ 
17  $u := (u_1, \dots, u_\ell) \leftarrow_{\$} \mathbb{G}^\ell$ 
18  $x^U := (x_1^U, \dots, x_\ell^U) := (u_1 \star w, \dots, u_\ell \star w)$ 
19  $root^U := \text{GetRoot}(x^U)$ 
20  $\sigma := H_2(U, S, root^U, pw)$ 
21  $x_{\sigma_1}^U = F_\sigma(x^U)$ 
22  $\pi_U^t := (u, (U, S, x^U, \perp), \perp, \perp)$ 
23 return  $(U, x_{\sigma_1}^U)$ 
SENDRESP( $S, t, U, x_{\sigma_1}^U$ )
24 if  $\pi_S^t \neq \perp$  return  $\perp$ 
25  $s := (s_1, \dots, s_\ell) \leftarrow_{\$} \mathbb{G}^\ell$ 
26  $x^S := (x_1^S, \dots, x_\ell^S) := (s_1 \star w, \dots, s_\ell \star w)$ 
27  $root^S := \text{GetRoot}(x^S)$ 
28  $\sigma_2 := H_2(U, S, root^S, pw)$ 
29  $x_{\sigma_2}^S = F_{\sigma_2}(x^S)$ 
30  $root^U := \text{GetRoot}(x_{\sigma_1}^U)$ 
31  $\sigma_1 := H_2(U, S, root^U, pw)$ 
32  $x^U = F_{\sigma_1}^{-1}(x_{\sigma_1}^U)$ 
33 if  $\exists P \in \mathcal{U} \setminus S, t' \text{ s.t. } \pi_P^{t'}.tr = (U, S, x^U, x^S)$ 
34   return  $\perp$ 
35 if  $U \notin C$  and  $S \notin C$ 
36    $\pi_S^t.fr := true$ 
37   if  $\forall pw, z \text{ s.t. } (U, S, x^U, x^S, pw, z) \in Q$ 
38      $(b_1, \dots, b_\ell) := pw$ 
39     if  $GA-DDH_{x_{b_i}}(w, x_i^U, s_i^{-1} \star z_i) = 1 \forall i \in [\ell]$ 
40        $Q_{bad} := Q_{bad} \cup \{(U, S, x^U, x^S, pw, z)\}$ 
41    $K \leftarrow_{\$} \mathcal{K}$ 
42    $Q_s[U, S, x^U, x^S] := (S, s, K)$ 
43 else
44    $\pi_S^t.fr := false$ 
45    $(b_1, \dots, b_\ell) := pw_{US}$ 
46   if  $\exists z \text{ s.t. } (U, S, x^U, x^S, pw_{US}, z) \in Q$ 
47     and  $GA-DDH_{x_{b_i}}(w, x_i^U, s_i^{-1} \star z_i) = 1 \forall i \in [\ell]$ 
48      $K := Q[U, S, x^U, x^S, pw_{US}, z]$ 
49   else
50      $K \leftarrow_{\$} \mathcal{K}$ 
51      $Q[U, S, x^U, x^S, pw_{US}, (s, 0)] := K$ 
52    $\pi_S^t := (s, (U, S, x^U, x^S), K, true)$ 
52 return  $(S, x_{\sigma_2}^S)$ 
SENDTERM( $U, t, S, x_{\sigma_2}^S$ )
53 if  $\pi_U^t \neq (u, (U, S, x^U, \perp), \perp, \perp)$  return  $\perp$ 
54  $root^S := \text{GetRoot}(x_{\sigma_2}^S)$ 
55  $\sigma_2 := H_2(U, S, root^S, pw)$ 
56  $x_{\sigma_2}^S = F_{\sigma_2}^{-1}(x_{\sigma_2}^S)$ 
57 if  $\exists P \in \mathcal{U}, t' \text{ s.t. } \pi_P^{t'}.tr = (U, S, x^U, x^S)$ 
58   return  $\perp$ 
59 if  $\exists t' \text{ s.t. } \pi_S^{t'}.tr = (U, S, x^U, x^S)$ 
60   and  $\pi_S^{t'}.fr = true$ 
61    $\pi_U^t.fr = true$ 
62    $(S, s, K) := Q_s[U, S, x^U, x^S]$ 
63 else if  $U \notin C$  and  $S \notin C$ 
64    $\pi_U^t.fr = true$ 
65    $\forall pw, z \text{ s.t. } (U, S, x^U, x^S, pw, z) \in Q$ 
66      $(b_1, \dots, b_\ell) := pw$ 
67     if  $GA-DDH_{x_{b_i}}(w, x_i^U, u_i^{-1} \star z_i) = 1 \forall i \in [\ell]$ 
68        $Q_{bad} := Q_{bad} \cup \{(U, S, x^U, x^S, pw, z)\}$ 
69    $K \leftarrow_{\$} \mathcal{K}$ 
70    $Q_s[U, S, x^U, x^S] := (U, u, K)$ 
71 else
72    $\pi_U^t.fr := false$ 
73    $(b_1, \dots, b_\ell) := pw_{US}$ 
74   if  $\exists z \text{ s.t. } (U, S, x^U, x^S, pw_{US}, z) \in Q$ 
75     and  $GA-DDH_{x_{b_i}}(w, x_i^U, s_i^{-1} \star z_i) = 1 \forall i \in [\ell]$ 
76      $K := Q[U, S, x^U, x^S, pw_{US}, z]$ 
77   else
78      $K \leftarrow_{\$} \mathcal{K}$ 
79      $Q[U, S, x^U, x^S, pw_{US}, (u, 0)] := K$ 
80    $\pi_U^t := (u, (U, S, x^U, x^S), K, true)$ 
81 return true
H( $U, S, x^U, x^S, pw, z$ )
82 if  $Q[U, S, x^U, x^S, pw, z] = K \neq \perp$ 
83   return  $K$ 
84 if  $(U, S, x^U, x^S) \in Q_s$ 
85    $(b_1, \dots, b_\ell) := pw$ 
86   if  $Q_s[U, S, x^U, x^S] = (U, u, K)$ 
87     if  $GA-DDH_{x_{b_i}}(w, x_i^U, u_i^{-1} \star z_i) = 1 \forall i \in [\ell]$ 
88       if  $U \notin C$  and  $S \notin C$ 
89          $Q_{bad} := Q_{bad} \cup \{(U, S, x^U, x^S, pw, z)\}$ 
90       if  $U \in C$  and  $S \in C$  and  $pw = pw_{US}$ 
91         return  $K$ 
92       if  $Q_s[U, S, x^U, x^S] = (S, s, K)$ 
93         if  $GA-DDH_{x_{b_i}}(w, x_i^U, s_i^{-1} \star z_i) = 1 \forall i \in [\ell]$ 
94           if  $U \notin C$  and  $S \notin C$ 
95              $Q_{bad} := Q_{bad} \cup \{(U, S, x^U, x^S, pw, z)\}$ 
96           if  $U \in C$  and  $S \in C$  and  $pw = pw_{US}$ 
97             return  $K$ 
98           if  $\exists u \text{ s.t. } (U, S, x^U, x^S, pw, (u, 0)) \in Q$ 
99              $(b_1, \dots, b_\ell) := pw$ 
100             if  $GA-DDH_{x_{b_i}}(w, x_i^U, u_i^{-1} \star z_i) = 1 \forall i \in [\ell]$ 
101               return  $Q[U, S, x^U, x^S, pw, (u, 0)]$ 
102             else if  $\exists s \text{ s.t. } (U, S, x^U, x^S, pw, (s, 0)) \in Q$ 
103                $(b_1, \dots, b_\ell) := pw$ 
104               if  $GA-DDH_{x_{b_i}}(w, x_i^U, s_i^{-1} \star z_i) = 1 \forall i \in [\ell]$ 
105                 return  $Q[U, S, x^U, x^S, pw, (s, 0)]$ 
106              $Q[U, S, x^U, x^S, pw, z] \leftarrow_{\$} \mathcal{K}$ 
107             return  $Q[U, S, x^U, x^S, pw, z]$ 

```

 图 3.8 定理 3.3.1 证明中敌手 B_2 针对 Sim-GA-StCDH 的构造。

获取 $j \in \{0, 1\}^2$ 。 B_2 运行 $\mathcal{A}^{\mathcal{O}}(x_0, x_1)$ ，模拟 SENTINIT 查询如下： B_2 均匀随机选择一个群元素 u_i 进行计算

$$x_i^U = u_i \star w = (u_i \cdot h \cdot g_0^{-1}) \star x_0 = (u_i \cdot h \cdot g_1^{-1}) \star x_1.$$

同样， B_2 以同样的方式在 SENTRESP 查询中设置 x_i^S 。如果服务器实例 S 是一个新鲜实例，则需要验证 $\mathcal{Q}$ 中是否已经存在冲突的条目。遍历 $\mathcal{Q}$ 中包含此实例活动轨迹的所有条目，检查每个 pw 和 z 。具体来说，检查下述式子是否成立：

$$z_i = \text{GA-CDH}_{x_{b_i}}(x_i^U, x_i^S) \Leftrightarrow \text{GA-CDH}_{x_{b_i}}(w, x_i^U) = s_i^{-1} \star z_i,$$

可以使用 DDH 预言机 $\text{GA-DDH}_{x_{b_i}}(w, \cdot, \cdot)$ 进行模拟。如果对于所有 i , z_i 都是正确的，将条目添加到 $\mathcal{Q}_{\text{bad}}$ (37到40行)。

对于非新鲜实例，必须为其计算正确的键。如上所述，检查 $\mathcal{Q}$ 中是否存在有效的条目 z ：如果存在，则将此值赋给会话密钥 (47行)；如果没有，从 $\mathcal{K}$ 中随机选择一个密钥，并向 $\mathcal{Q}$ 添加一个特殊条目。这个条目不包含 z ，而是包含秘密群元素 s_i (第50行)，以便后续对随机预言机进行修补。类似地，使用秘密群元素 u_i 对 SENTTERM 进行模拟。

对于一个随机预言机查询，如果活动轨迹包含在 $\mathcal{Q}_s$ 中，表明发送查询时对应的实例是新鲜的，那么使用 GA-DDH 预言机检查 z 是否有效 (95、99行)。如果共享值 z 有效，首先检查实例是否保持新鲜，即口令没有损坏：如果是，将查询包含在 $\mathcal{Q}_{\text{bad}}$ 中 (87、93行)；否则，口令已损坏并在查询中显示，则检索存储在 $\mathcal{Q}_s$ (89、95行) 中的会话密钥。接下来，检查查询是否与在 SENTRESP 或 SENTTERM 中为一个非新鲜实例添加到 $\mathcal{Q}$ 的特殊条目匹配。在这种情况下，须输出先前返回的相同会话密钥。此外，可以使用 GA-DDH 预言机来区分属于用户和服务器的实例 (96到99行)。

一旦 $\mathcal{A}$ 完成其操作并产生了输出 β' ， B_2 就会继续选择在 CORRUPT 查询中没有生成的口令。如果出现 bad_{pw} (第 05-13 行)，那么对于相同的活动轨迹，在 $\mathcal{Q}_{\text{bad}}$ 中必须存在两个条目，分别使用不同的口令 $\text{pw} \neq \text{pw}'$ ，以及各自的值 z 和 z' 。设 i 是 pw 和 pw' 第一个不同的索引，即 $b_i \neq b'_i$ 。不失一般性，假设 $b_i = 0$ 和 $b'_i = 1$ ，否则，交换 (pw, z) 和 (pw', z') 来满足这个条件。如果 $\mathcal{Q}_{\text{bad}}$ 包含来自用户实例的条目，从集合 $\mathcal{Q}_s$ 检索秘密群元素 u_i 。

此处，本方案通过 root 限制敌手直接使用扭曲 (twist)。如果敌手使用扭曲结果作为输入， root 将被重新计算。需要注意的是，如果没有 pw ，就不能为 shuffle 函数生成正确的 seed 。因此，伙伴实例无法恢复实际输入，进而无法共享正确的会话密钥。已知，若一个群体的行动不允许有效的扭曲，那么 Sim - GA - StCDH 问题被认为是困难的。如果实例是用户实例，则 B_2 设置 $y = x_i^S$ ，并输出 y 和

$$y_0 = u_i^{-1} \star z = \text{GA-CDH}_{x_0}(u_i^{-1} \star x_i^U, x_i^S) = \text{GA-CDH}_{x_0}(w, x_i^S),$$

$$y_1 = u_i^{-1} \star z' = \text{GA-CDH}_{x_1}(u_i^{-1} \star x_i^U, x_i^S) = \text{GA-CDH}_{x_1}(w, x_i^S),$$

同样，如果实例是服务器实例， B_2 将设置 $y = x_i^U$ ，并输出 y 和

$$y_0 = s_i^{-1} \star z = \text{GA-CDH}_{x_0}(s_i^{-1} \star x_i^S, x_i^U) = \text{GA-CDH}_{x_0}(w, x_i^U),$$

$$y_1 = s_i^{-1} \star z' = \text{GA-CDH}_{x_1}(s_i'^{-1} \star x_i^S, x_i^U) = \text{GA-CDH}_{x_1}(w, x_i^U),$$

对 bad_{pw} 的分析已经结束。如果 bad_{pw} 发生， B_2 可以解决 Sim - GA - StCDH 问题。

接下来进行对 $\text{bad}_{\text{guess}}$ 的分析。如上所述， $\text{bad}_{\text{guess}}$ 仅在 bad_{pw} 未发生时发生。因此，每个实例在 Q_{bad} 中最多只有一个条目，且 Q_{bad} 的大小不会超过 q_s 。由于所有条目都是在采样对应口令之前添加的，其概率满足

$$\Pr [G_6 \Rightarrow \text{bad}_{\text{guess}}] \leq \frac{q_s}{|\mathcal{PW}|}.$$

如果 G_6 中未发生任何 bad 事件， TEST 生成的会话密钥是均匀随机的。敌手 $\mathcal{A}$ 无法从 TEST 中获取任何信息，从而只能猜测 β 的值。因此，

$$\Pr [G_6 \Rightarrow 1] = \frac{1}{2}.$$

□

3.4 方案对比

在表 3.3 中，展示了现有方案与本方案之间的对比。此外，本小节提供了不同方案的开销及效率在特定参数下的具体数值。表中， $\text{X-GA-PAKE}_{\ell, N}^t$ 是 X-GA-PAKE_{ℓ} 的变体，它将公共参数的数量增加到 2^N ，并将 pw 的长度划分为 ℓ/N 。通过相同的技术，可得 $\text{R-GA-PAKE}_{\ell, N}^t$ 。作为示例，设置 $\ell = 256$ 和 $N = 8$ 。

表 3.3 现有方案与 R-GA-PAKE_{ℓ} 的对比

协议	计算假设	轮数	lcrsl	通信开销	群操作次数
$\text{X-GA-PAKE}_{\ell}^{[36]}$	SqInv-GA-StCDH	1	2 2	2ℓ 512	5ℓ 1280
$\text{X-GA-PAKE}_{\ell, N}^t^{[36]}$	SqInv-GA-StCDH	1	2^{N-1} 128	$2\ell/N$ 64	$5\ell/N$ 160
$\text{CrsX-GA-PAKE}_{\ell}^{[37]}$	SqInv-GA-StCDH	1	2 2	$\ell + 2$ 258	$4\ell + 2$ 1026
$\text{CrsX-GA-PAKE}_{\ell, N}^t^{[37]}$	SqInv-GA-StCDH	1	2^{N-1} 128	$\ell/N + 2^{N-1}$ 160	$4\ell/N + 2^{N-1}$ 64
R-GA-PAKE_{ℓ}	SqInv-GA-StCDH	1	2 2	ℓ 256	2ℓ 512
$\text{R-GA-PAKE}_{\ell, N}^t$	SqInv-GA-StCDH	1	2^{N-1} 128	ℓ/N 32	$2\ell/N$ 64

对于 $X\text{-GA-PAKE}_\ell$, ℓ 表示口令的长度。在 $X\text{-GA-PAKE}_\ell$ 中, 每一方需要发送 2ℓ 个集合元素作为通信开销, 并执行 5ℓ 次群操作。在 CrsX-GA-PAKE_ℓ 中, 对于每一方, 集合元素的数量减少到 $\ell + |\text{crs}|$, 群操作的次数减少到 $4\ell + |\text{crs}|$ 。本方案中同样通过某个可信方生成两个随机集合元素 crs , 通过使用洗牌函数, 不需要向消息中添加任何额外的元素, 从而抹除为加强安全性而增加的开销。本方案将每一方发送的集合元素数量从 2ℓ 减少到 ℓ , 并将群操作的次数减少到每方 2ℓ 次。此外, 虽然 $X\text{-GA-PAKE}_\ell$ 和 CrsX-GA-PAKE_ℓ 中用于密钥派生的 z 的消息长度均为 3ℓ , 但本方案成功将其减少到 ℓ 。在变体 $\text{R-GA-PAKE}_{\ell,N}^t$ 中, 实现了相同比例的优化。在 $X\text{-GA-PAKE}_{\ell,N}^t$ 和 $\text{R-GA-PAKE}_{\ell,N}^t$ 中, N 表示将 ℓ 划分为 ℓ/N 的长度, t 表示设置中使用的扭曲次数。根据表中给出的示例值, 可以更清楚地看到优化效果。当 ℓ (ℓ/N) 的值增加时, 协议之间的通信和计算开销差异也会增加。 $X\text{-GA-PAKE}_\ell$ 、 CrsX-GA-PAKE_ℓ 以及本方案都是基于 SqlInv-GA-StCDH 假设的可证明安全的一轮 PAKE 协议。

3.5 本章小结

本章提供一种基于随机同源置换的口令认证密钥协商方法, 在不增加通信开销的同时, 利用随机置换算法, 将同源计算的中间结果随机置换为完全不同的随机序列结果, 从而抵御离线字典攻击。在共享口令和两个元素的情况下, 用户和服务器首先分别生成随机元素, 通过群操作获得对应序列。如果对该序列直接进行传输, 无法抵御离线字典攻击。因此本方案提出, 将原始序列随机置换成新的序列并发送给对方, 服务器和用户将对方发送的序列还原后, 双方各自计算会话密钥, 完成密钥协商。在随机置换的过程中, 基于口令和 **Merkle** 根哈希生成种子, 以保证每次生成不同的顺序序列。本方案在抵御离线字典攻击的同时, 保持通信开销仍为 $O(2\ell)$ 。

4 基于同源的非对称口令认证密钥协商方案

传统的 PAKE 方案通常要求服务器端存储某种形式的口令信息，从而引发了潜在的安全风险，如口令泄露、离线字典攻击等。为了解决这一问题，非对称 PAKE 应运而生，它允许客户端使用口令进行认证，而服务器端则无需存储明文或可逆推的口令信息，从而增强了系统的安全性。本章提出一个基于强非对称 PAKE 编译器^[42]构造的非对称 PAKE 方案，将章节3提出的方案进一步优化，使得服务端在无需存储口令的情况下，依旧能够顺利完成密钥协商过程。客户端和服务端双方通过注册和登录过程，计算生成一致的口令，有效规避了因服务端直接存储口令所带来的潜在安全风险。

本章将主要从挑战分析、方案设计、安全性分析等方面进行介绍。本章中使用的符号定义如表 4.1所示，底层 PAKE 协议涉及的符号如表 3.1所示。

表 4.1 本章中符号及相关描述

符号	描述
κ	安全参数
C	客户端的身份标识符
S	服务器端的身份标识符
P	参与通信的一方，即 $P \in \{C, S\}$
sid	会话标识符，用于区分不同的会话实例
$ssid$	子会话标识符，用于进一步细化会话标识
$d \leftarrow D$	按照某个集合上的概率分布 D 采样一个元素 d
$s \leftarrow S$	从无明显伴随分布的集合 S 中按照均匀分布采样得到元素 s

4.1 安全模型

本节将回顾 Jarecki 等人^[5]提出的非对称 PAKE 的安全模型及理想函数。所有功能只具备隐式认证，这在 PAKE 相关文献中属于常见的标准情况，而显式认证可以通过添加单个密钥确认流^[52]来实现。

(1) 非对称 PAKE 函数 $\mathcal{F}_{\text{saFAKE}}$

非对称 PAKE 的功能 $\mathcal{F}_{\text{saFAKE}}$ 。由于非对称 PAKE 是对 PAKE 的扩展建模，通信双方分别为客户端 C 和服务端 S 。 C 通过接口 `ClientSession` 在口令 pw' 上运行一个非对称 PAKE 会话，而 S 通过接口 `StorePwdFile` 在口令文件 `file[(sid, C, S)]` 上运行该会话，该文

件表示服务器端存储的口令摘要。与 PAKE 类似，如果中间人攻击敌手不干扰会话，当且仅当 $\text{file}[(\text{sid}, C, S)] = \text{pw}'$ 时，两个诚实的参与者获得相同的密钥 k 。接口 StorePwdFile 代表客户端注册，但传统上该行为与服务器端非交互，可能是通过一个经过认证的安全信道接收带外口令，然后在存储文件后安全地擦除口令。

除了在线攻击接口 TestPwd， $\mathcal{F}_{\text{saFAKE}}$ 还通过接口 StealPwdFile 对服务器端被自适应攻破的情况进行了建模。发送 StealPwdFile 之后，理想情况下的敌手可以获得另外两个接口的访问权限：OfflineTestPwd（离线测试口令）和 Impersonate（冒充），这两个接口分别允许敌手进行离线字典攻击（也就是说，无需调用在线会话即可猜测口令）以及冒充诚实客户端进行身份验证。为了排除预计算攻击，在发送 StealPwdFile 之前，OfflineTestPwd 接口会忽略所有消息。

对于 $\mathcal{F}_{\text{saFAKE}}$ ，需存储四个映射：fileStatus、file、sessionStatus 和 session。具体理想函数如下：

① 口令注册

当从服务器端 S 接收到 (StorePwdFile, sid, C, pw) 消息时，若 $\text{file}[(\text{sid}, C, S)]$ 未定义，则将其设置为 pw，并将 $\text{fileStatus}[(\text{sid}, C, S)]$ 设置为“未泄露”状态。

② 窃取口令数据

当从敌手 $\mathcal{A}^*$ 接收到 (StealPwdFile, sid) 消息时：

1. 若 $\text{fileStatus}[(\text{sid}, C, S)]$ 未定义，向 $\mathcal{A}^*$ 返回“无口令文件”；
2. 否则：
 - (a) 若 $\text{fileStatus}[(\text{sid}, C, S)]$ 处于“未泄露”状态，修改状态为“已泄露”；
 - (b) 向 $\mathcal{A}^*$ 返回“口令文件已被窃取”。

当从敌手 $\mathcal{A}^*$ 接收到 (OfflineTestPwd, sid, pw*) 消息时：

1. 若 $\text{fileStatus}[(\text{sid}, C, S)]$ 状态为“已泄露”：
 - (a) 从 $\text{file}[(\text{sid}, C, S)]$ 中取出 pw；
 - (b) 若 $\text{pw} = \text{pw}^*$ ，向 $\mathcal{A}^*$ 返回“猜测正确”；
 - (c) 否则，向 $\mathcal{A}^*$ 返回“猜测错误”。

③ 口令认证

当从客户端 C 接收到 (ClientSession, sid, ssid, S, pw') 消息时：

1. 向 $\mathcal{A}^*$ 转发 (ClientSession, sid, pw') 消息；
2. 若 $\text{sessionStatus}[(\text{sid}, \text{ssid}, C, S)]$ 未定义，则将 $\text{session}[(\text{sid}, \text{ssid}, C, S)]$ 设置为 pw'，并将 $\text{sessionStatus}[(\text{sid}, \text{ssid}, C, S)]$ 设置为“新会话”。

当从服务器端 S 接收到 (ServerSession, sid, ssid) 消息时：

1. 若 $\text{file}[(\text{sid}, \text{C}, \text{S})]$ 未定义，忽略此查询；
2. 否则，从 $\text{file}[(\text{sid}, \text{C}, \text{S})]$ 中取出 pw ，向 $\mathcal{A}^*$ 转发 $(\text{ClientSession}, \text{sid}, \text{pw}')$ 消息；
3. 若 $\text{sessionStatus}[(\text{sid}, \text{ssid}, \text{S}, \text{C})]$ 未定义，则将 $\text{session}[(\text{sid}, \text{ssid}, \text{S}, \text{C})]$ 设置为 pw ，并将 $\text{sessionStatus}[(\text{sid}, \text{ssid}, \text{S}, \text{C})]$ 设置为“新会话”。

④ 主动会话攻击

当从敌手 $\mathcal{A}^*$ 接收到 $(\text{TestPwd}, \text{sid}, \text{ssid}, \text{P}, \text{pw}^*)$ 消息时：

1. 若 $\text{sessionStatus}[(\text{sid}, \text{ssid}, \text{P}, \text{P}')] 未定义，忽略此查询；$
2. 否则，从 $\text{session}[(\text{sid}, \text{ssid}, \text{P}, \text{P}')] 中取出 pw' ；$
3. 若 $\text{sessionStatus}[(\text{sid}, \text{ssid}, \text{P}, \text{P}')] 为“新会话”：$
 - (a) 若 $\text{pw}' = \text{pw}^*$ ，向 $\mathcal{A}^*$ 返回“猜测正确”，并将 $\text{sessionStatus}[(\text{sid}, \text{ssid}, \text{P}, \text{P}')] 设置为“已泄露”；$
 - (b) 否则，将 $\text{sessionStatus}[(\text{sid}, \text{ssid}, \text{P}, \text{P}')] 设置为“中断”，并向 $\mathcal{A}^*$ 返回“猜测错误”。$

当从敌手 $\mathcal{A}^*$ 接收到 $(\text{Impersonate}, \text{sid}, \text{ssid})$ 消息时：

1. 若 $\text{sessionStatus}[(\text{sid}, \text{ssid}, \text{C}, \text{S})]$ 为“新会话”：
 - (a) 若 $\text{fileStatus}[(\text{C}, \text{S})]$ 状态为“已泄露”且 $\text{file}[(\text{sid}, \text{C}, \text{S})] = \text{session}[(\text{sid}, \text{ssid}, \text{C}, \text{S})]$ ，将 $\text{sessionStatus}[(\text{sid}, \text{ssid}, \text{C}, \text{S})]$ 设置为“已泄露”，并向 $\mathcal{A}^*$ 返回“猜测正确”；
 - (b) 否则，将 $\text{sessionStatus}[(\text{sid}, \text{ssid}, \text{C}, \text{S})]$ 设置为“中断”，并向 $\mathcal{A}^*$ 返回“猜测错误”。

⑤ 密钥生成

当从敌手 $\mathcal{A}^*$ 接收到 $(\text{NewKey}, \text{sid}, \text{ssid}, \text{P}, k^*)$ 消息，且 $|k^*| = \kappa$ 时：

1. 若 $\text{sessionStatus}[(\text{sid}, \text{ssid}, \text{P}, \text{P}')] 已定义但未完成：$
 - (a) 若 $\text{sessionStatus}[(\text{sid}, \text{ssid}, \text{P}, \text{P}')] 状态为“已泄露”，则令 $k = k^*$ ；$
 - (b) 若 $\text{sessionStatus}[(\text{sid}, \text{ssid}, \text{P}, \text{P}')] 和 $\text{sessionStatus}[(\text{sid}, \text{ssid}, \text{P}', \text{P})]$ 均为“新会话”， $(\text{sid}, \text{ssid}, k')$ 已发送给 P' ， $\text{session}[(\text{sid}, \text{ssid}, \text{P}, \text{P}')] = \text{session}[(\text{sid}, \text{ssid}, \text{P}', \text{P})]$ ，设置 $k = k'$ ；$
 - (c) 否则，从 $\{0, 1\}^\kappa$ 中随机采样生成 k 。
2. 最后，设置 $\text{sessionStatus}[(\text{sid}, \text{ssid}, \text{P}, \text{P}')] 为“已完成”，并将 $(\text{sid}, \text{ssid}, k')$ 发送给 P 。$

(2) 随机预言机函数 $\mathcal{F}_{\text{RO}}$

对于理想函数 $\mathcal{F}_{\text{RO}}$ ，有值域区间 $\mathcal{H}$ ，映射 $\mathcal{H} : 0, 1^* \rightarrow \mathcal{H}$ 。

- 当从 P 接收到 $(\text{Eval}, \text{sid}, x)$ 时：
 1. 若 $\mathcal{H}[x]$ 未定义，则从 $\mathcal{H}$ 中采样并赋值给 $\mathcal{H}[x]$ 。

2. 将 $(\text{Eval}, \text{sid}, H[x])$ 发送给 P。

(3) 群操作函数 $\mathcal{F}_{\text{GA}^\top}$

对于理想函数 $\mathcal{F}_{\text{GA}^\top}$ ，有初始参数：群 $(\mathbb{G}, +)$ ，句柄集 $\mathcal{X} \subseteq \{0, 1\}^\kappa$ ，并存储映射 $\text{DL} : \mathbb{G} \rightarrow \mathcal{D}$ 。下面讨论接收到不同查询时的情况：

- 当从 P 接收到 $(\text{op}, \text{sid}, g', x)$ 时：
 1. 检索 $g := \text{DL}[x]$ ，并且如果不存在 $x' \in \mathcal{D}$ 使得 $\text{DL}[x'] := g' + g$ ，则从 $\mathcal{D}$ 中采样 x' ，并设置 $\text{DL}[x'] := g' + g$ 。
 2. 将 $(\text{op}, \text{sid}, x')$ 发送给 P。
- 当从 P 接收到 $(\text{tw}, \text{sid}, g', x)$ 时：
 1. 检索 $g := \text{DL}[x]$ ，并且如果不存在 $x' \in \mathcal{D}$ 使得 $\text{DL}[x'] := -g$ ，则从 $\mathcal{D}$ 中采样 x' ，并设置 $\text{DL}[x'] := -g$ 。
 2. 将 $(\text{op}, \text{sid}, x')$ 发送给 P。

4.2 方案设计

本节将详细描述所提出的基于随机同源置换的非对称口令认证密钥协商方案设计。通过强非对称 PAKE 编译器，本章基于 GA-CDH 假设在章节3提出的方案基础上，提出一种基于随机同源置换的非对称协议 aR-GA-PAKE_ℓ 。“aR-GA-PAKE_ℓ”中的“a”代表“非对称（asymmetric）”。服务器端和客户端在群操作计算过程中，虽然涉及到群元素与集合元素的映射及相关操作，但通过合理的预处理和缓存机制，减少了重复计算。方案整体框架图如4.1。

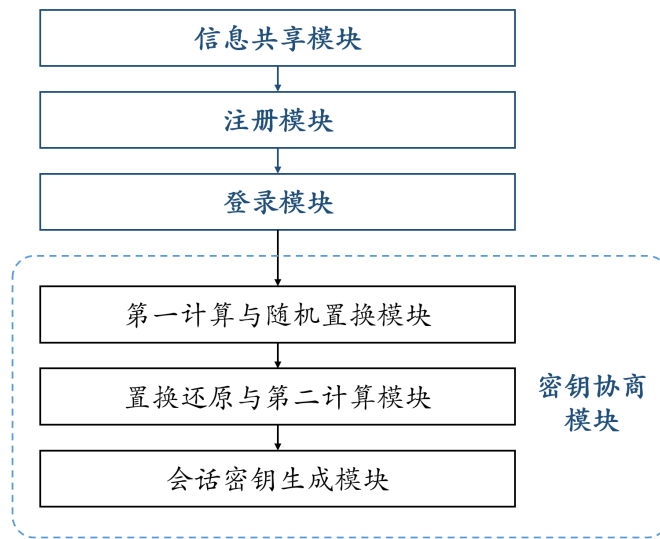
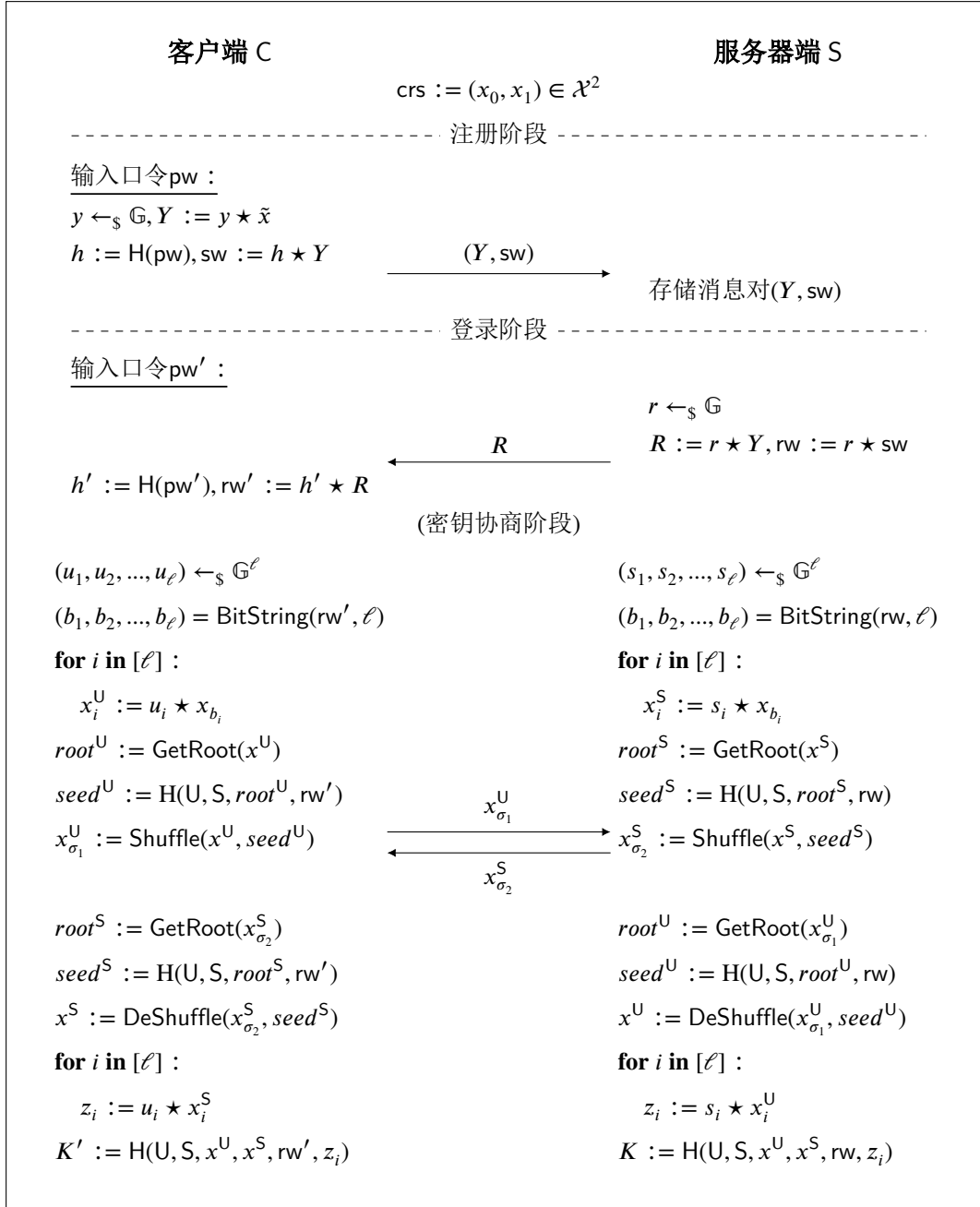


图 4.1 aR-GA-PAKE_ℓ 方案的整体框架


 图 4.2 aR - GA - PAKE_ℓ 方案的协议流程。

在本方案中，定义通信双方为客户端 U 和服务端 S，双方共享两个元素 $\text{crs} = (x_0, x_1) \in \mathcal{X}^2$ ，而无需像 R - GA - PAKE_ℓ 中一样共享口令 pw 。方案主要分为注册、登录两个阶段，注册时客户端基于输入口令进行群操作及哈希运算，将结果传至服务器端存储；登录时客户端、服务器端分别运算并交换信息，再借助 PAKE 协议生成会话密钥，实现基于口令的安全认证与密钥协商。定义函数 $\text{BitString}(x, y)$ 用于将数 x 转换成固定长度 y 的比特串。整体协议流程如图 4.2，其中的具体步骤如下：

1. 客户端 C 和服务端 S 共享 $\text{crs} = (x_0, x_1) \in \mathcal{X}^2$ ；
2. 客户端 C 输入口令 pw 进行注册，从集合 $\mathbb{G}$ 随机选取元素 y ，计算随机衍生值

- $Y := y \star \tilde{x}$ 和口令哈希值 $h := H(\text{pw})$, 进一步计算得到 $\text{sw} := h \star Y$, 并将消息对 (Y, sw) 发送给服务器端 S;
3. 服务器端 S 接收并存储消息对 (Y, sw) , 完成注册;
 4. 客户端 C 输入口令 pw' 进行登录;
 5. 服务器端 S 从集合 $\mathbb{G}$ 随机选取元素 r , 计算随机衍生值 $R := r \star Y$ 和 $\text{rw} := r \star \text{sw}$, 并将 R 发送给客户端 C;
 6. 客户端 C 计算口令哈希值 $h' := H(\text{pw}')$, 进一步计算得到 $\text{rw}' := h' \star R$;
 7. 客户端 C 生成随机元素 $(u_1, \dots, u_\ell) \leftarrow_{\$} \mathbb{G}^\ell$, 将 rw' 通过 $\text{BitString}(\text{rw}', \ell)$ 转换为 ℓ 位长的比特串, 计算 $x_i^U = u_i \star x_{b_i}$, 并进一步计算其 Merkle 根 root^U 。其中, x_{b_i} 为根据 b_i 选择的结果, x_i^U 为群元素 u_i 对 x_{b_i} 进行群操作的计算结果, root^U 为由 $(x_1^U, \dots, x_\ell^U)$ 按照自然顺序生成的 Merkle 根;
 8. 服务器端 S 生成随机元素 $(s_1, \dots, s_\ell) \leftarrow_{\$} \mathbb{G}^\ell$, 将 rw 通过 $\text{BitString}(\text{rw}, \ell)$ 转换为 ℓ 位长的比特串, 计算 $x_i^S = s_i \star x_{b_i}$, 并进一步计算其 Merkle 根 root^S 。其中, x_{b_i} 为根据 b_i 选择的结果, x_i^S 为群元素 s_i 对 x_{b_i} 进行群操作的计算结果, root^S 为由 $(x_1^S, \dots, x_\ell^S)$ 按照自然顺序生成的 Merkle 根;
 9. 客户端 C 通过第一种子 seed^U 将第一序列 x^U 进行随机置换 $\text{Shuffle}(x^U, \text{seed}^U)$, 得到第二序列 $x_{\sigma_1}^U$ 并发送给服务器端 S, 其中, $\text{seed}^U = H(\text{rw}' || \text{root}^U)$;
 10. 服务器端 S 通过第二种子 seed^S 将第三序列 x^S 进行随机置换 $\text{Shuffle}(x^S, \text{seed}^S)$, 得到第四序列 $x_{\sigma_2}^S$ 并发送给客户端 C, 其中, $\text{seed}^S = H(\text{rw} || \text{root}^S)$;
 11. 客户端 C 接收到第四序列 $x_{\sigma_2}^S$ 后, 计算其 Merkle 根 root^S 和第二种子 seed^S , 并通过第二种子 seed^S 将第四序列 $x_{\sigma_2}^S$ 进行置换还原 $\text{DeShuffle}(x_{\sigma_2}^S, \text{seed}^S)$, 得到第三序列 x^S , 并通过 $z_i = u_i \star x_i^S$ 计算用于生成会话密钥的中间计算结果;
 12. 服务器端 S 接收到第二序列 $x_{\sigma_1}^U$ 后, 计算其 Merkle 根 root^U 和第一种子 seed^U , 并通过第一种子 seed^U 将第二序列 $x_{\sigma_1}^U$ 进行置换还原 $\text{DeShuffle}(x_{\sigma_1}^U, \text{seed}^U)$, 得到第一序列 x^U , 并通过 $z_i = s_i \star x_i^U$ 计算用于生成会话密钥的中间计算结果;
 13. 双方各自计算会话密钥为 $K = H(U, S, x^U, \text{rw}', z)$ 和 $K = H(U, S, x^U, \text{rw}, z)$ 。

图4.1中的信息共享模块包括步骤 1; 注册模块包括步骤 2 和步骤 3; 登录模块包括步骤 4、步骤 5 和步骤 6; 密钥协商模块包括步骤 7 到步骤 13, 其中, 第一计算与随机置换模块包括步骤 7、步骤 8、步骤 9 和步骤 10, 置换还原与第二计算模块包括步骤 11 和步骤 12, 会话密钥生成模块包括步骤 13。

4.3 安全性分析

本节将依据小节 4.1 中定义的非对称 PAKE 功能, 对 aR - GA - PAKE_ℓ 方案的安全性展开分析。参考 McQuoid 等人^[42] 的安全模型, 本节将介绍模拟器构造, 并通过混合实验证明方案的不可区分性。

4.3.1 模拟器设计

(1) 口令文件泄露及离线查询

- 当收到来自敌手 $\mathcal{A}$ 发送给服务器端 S 的 (StealPwFile, sid) 请求时:
 1. 向 $\mathcal{F}_{\text{saFAKE}}$ 转发该指令。
 2. 若 $\mathcal{F}_{\text{saFAKE}}$ 返回“口令文件已泄露”:
 - (a) 标记服务器端 S 为“已泄露”。
 - (b) 若文件记录 file[sid] 未定义: 生成 $(Y, sw) \leftarrow_{\mathcal{G}} \mathcal{G}^2$, 并将 (Y, sw) 作为服务器端 S 的响应返回给敌手 $\mathcal{A}$; 创建形式变量 $\mathbb{P}$ 用来表示 sw 相对于 Y 的离散对数, 生成 $y \leftarrow_{\mathcal{G}} \mathbb{G}$; 存储 $DL[Y] := y$ 和 $DL[sw] := y + \mathbb{P}$ 。
 3. 否则, 向敌手 $\mathcal{A}$ 返回“无口令文件”。
- 当收到来自敌手 $\mathcal{A}$ 发送给 $\mathcal{F}_{\text{RO}}$ 的 (Eval, sid, z) 请求时:
 1. 若 $H(z)$ 未定义, 随机选取 $m \leftarrow_{\mathcal{G}} \mathbb{G}$, 并记录 $H(z) := m$ 。
 2. 若存在 $z' \neq z$ 满足 $H(z') = H(z)$, 输出 coll 并终止。
 3. 若服务器端 S 被标记为“已泄露”:
 - (a) 向 $\mathcal{F}_{\text{saFAKE}}$ 发送 (OfflineTestPw, sid, z)。
 - (b) 若 $\mathcal{F}_{\text{saFAKE}}$ 返回“猜测正确”, 在后续流程中, 将 $\mathbb{P}$ 替换为 $H(z)$, 并存储 $serverPW[sid] := z$ 。
 4. 向敌手 $\mathcal{A}$ 返回 Eval, sid, $H(z)$ 。
- 当收到敌手 $\mathcal{A}$ 发送给 $\mathcal{F}_{\text{GA}^\top}$ 的 (op, sid, g, x) 或 (tw, sid, x) 请求时:
 1. 若原点 $\tilde{x}$ 的离散对数 $DL[\tilde{x}]$ 未定义, 设置 $DL[\tilde{x} = 0]$ 。
 2. 若 $DL[x]$ 未定义, 随机选取 $DL[x] \leftarrow_{\mathcal{G}} \mathbb{G}$ 。
 3. 将 $DL[x]$ 解析为形如 $a + b\mathbb{P}$ (其中 $b \in \{-1, 0, 1\}$), 并记录操作结果 γ :
 - (a) 对于 op 操作: $\gamma := (a_1 + g_1 + b\mathbb{P}_1, \dots, a_n + g_n + b\mathbb{P}_n)$ 。
 - (b) 对于 tw 操作: $\gamma := -a - b\mathbb{P}$ 。
 4. 若服务器端 S 被标记为“已泄露”:
 - (a) 假设这是敌手第 t 次在服务器端泄露后向 $\mathcal{F}_{\text{GA}^\top}$ 发起查询。记录前 $t + 2$ 次操作的离散对数为 $a_1 + b_{t+2}\mathbb{P}, \dots, a_{t+2} + b_{t+2}\mathbb{P}$ (初始值为 $(s, 0)$ 和 $(s, 1)$)。
 - (b) 求解方程组 $(b_i - b_{t+2})h_{t+2,i} = a_{t+2} - a_i$ (其中 $i \in [t + 1]$)。
 - (c) 对每个满足 $H(g_{t+2,i}) = h_{t+2,i}$ 的查询, 发送 (OfflineTestPw, sid, $g_{t+2,i}$) 至 $\mathcal{F}_{\text{saPAKE}}$ 。若需发送的指令数超过 $t + 1$, 输出 OfflineFailure 并终止。
 - (d) 若 $\mathcal{F}_{\text{saPAKE}}$ 返回“猜测正确”, 将所有响应中的 $\mathbb{P}$ 替换为 $h_{t+2,i}$, 并存储 $serverPW[sid] := g_{t+2,i}$ 。
 5. 若 γ 是新的离散对数, 采样 $X \leftarrow_{\mathcal{G}} \mathcal{G}$ 并设置 $DL[X] := \gamma$; 否则, 输出 coll 并终止。

6. 根据请求操作向敌手 $\mathcal{A}$ 返回 (op, sid, X) 或 (tw, sid, X) 。

(2) 口令认证阶段

- 当收到 $\mathcal{F}_{saPAKE}$ 发送的 $(ServerSession, sid, ssid, C, S)$ 指令时：
 1. 若文件记录 $file[sid]$ 未定义：生成 $(Y, sw) \leftarrow_{\mathcal{G}} \mathcal{G}^2$ ，并将 (Y, sw) 作为服务器端 S 的响应返回给敌手 $\mathcal{A}$ ；创建形式变量 $\mathbb{P}$ 用来表示 sw 相对于 Y 的离散对数，生成 $y \leftarrow_{\mathcal{G}} \mathbb{G}$ ；存储 $DL[Y] := y$ 和 $DL[sw] := y + \mathbb{P}$ 。
 2. 若 $serverSession[sid, ssid]$ 未定义，设置为 $serverSession[sid, ssid]_x := (C, S, \perp)$ 。
 3. 随机采样 $r \leftarrow_{\mathcal{G}} \mathbb{G}$ ，计算 $R := r \star \tilde{x}$ ，并将 $(sid, ssid, R)$ 发送给客户端 C 。
 4. 向敌手 $\mathcal{A}$ 发送 $(NewSession, sid || ssid || R, S)$ ，并将 $serverSession[sid, ssid]$ 标记为“PAKE 活跃”。
- 当收到 $\mathcal{F}_{saPAKE}$ 发送的 $(ClientSession, sid, ssid, C, S)$ 指令时：
 1. 若 $ClientSession[sid, ssid]$ 未定义，设置为 $ClientSession[sid, ssid]_x := (C, S, \perp)$
 2. 等待服务器端 S 发送给客户端 C 的 $(sid, ssid, [R^*]_x)$
 3. 向敌手 $\mathcal{A}$ 发送 $(NewSession, sid || ssid || R^*, C)$ ，并将 $clientSession[sid, ssid]$ 标记为“PAKE 活跃”。

(3) 主动会话攻击检测

- 当收到敌手 $\mathcal{A}$ 发送给 $\mathcal{F}_{PAKE}$ 的 $(TestPwd, sid || ssid || R, S, [rw^*]_x)$ ，且存在状态为“PAKE 活跃”的记录 $serverSession[sid, ssid] = (C, S, R)$ ：
 1. 检查是否存在 Z 使得 $rw^* = H(z) \star R$ 。若不存在，设 $z := \perp$ 。
 2. 向 $\mathcal{F}_{saPAKE}$ 发送 $(TestPwd, sid, ssid, S, z)$ ，并转发响应至敌手 $\mathcal{A}$ 。
 3. 若响应为“猜测正确”，将后续所有 $\mathbb{P}$ 替换为 $H(z)$ ，并存储 $serverPW[sid] := z$ 。
- 当收到敌手 $\mathcal{A}$ 发送给 $\mathcal{F}_{PAKE}$ 的 $(TestPwd, sid || ssid || R, C, [rw^*]_x)$ ，且存在状态为“PAKE 活跃”的记录 $clientSession[sid, ssid] = (C, S, R^*)$ ：
 1. 若服务器端 S 被标记为“已泄露”且敌手 $\mathcal{A}$ 持有 (Y, sw) ，并满足 $([R^*]_x = r^* + [Y]_x \wedge [rw^*]_x = r^* + [sw]_x) \vee ([R^*]_x = r^* + [-sw]_x \wedge [rw^*]_x = r^* + [-Y]_x) = \mathbf{true}$ ，则向 $\mathcal{F}_{saPAKE}$ 发送 $(Impersonate, sid, ssid)$ 并转发响应至敌手 $\mathcal{A}$ 。
 2. 否则：
 - (a) 检查是否存在 Z 使得 $rw^* = H(z) \star R$ 。若不存在，设 $z := \perp$ 。
 - (b) 向 $\mathcal{F}_{saPAKE}$ 发送 $(TestPwd, sid, ssid, C, z)$ ，并转发响应至敌手 $\mathcal{A}$ 。
 - (c) 若响应为“猜测正确”，将后续所有 $\mathbb{P}$ 替换为 $H(z)$ 。

(4) 密钥生成阶段

- 当收到敌手 $\mathcal{A}$ 发送给 $\mathcal{F}_{\text{PAKE}}$ 的 $(\text{NewKey}, \text{sid}||\text{ssid}||R, C, k^*)$ ，且存在状态为“PAKE 活跃”的记录 $(C, S, R) := \text{clientSession}[\text{sid}, \text{ssid}]$ ：
 1. 若存在服务器端会话且 $R^* \neq R$ ，向 $\mathcal{F}_{\text{saPAKE}}$ 发送 $(\text{TestPwd}, \text{sid}, \text{ssid}, C, \perp)$ 。
 2. 向 $\mathcal{F}_{\text{saPAKE}}$ 发送 $(\text{NewKey}, \text{sid}, \text{ssid}, C, k^*)$ ，并将 $\text{clientSession}[\text{sid}, \text{ssid}]$ 标记为“PAKE 已完成”。
- 收到敌手 $\mathcal{A}$ 发送给 $\mathcal{F}_{\text{PAKE}}$ 的 $(\text{TestPwd}, \text{sid}||\text{ssid}||R, S, [\text{rw}^*]_x)$ ，且存在状态为“PAKE 活跃”的记录 $(C, S, R) := \text{serverSession}[\text{sid}, \text{ssid}]$ ：
 1. 若存在客户端会话且 $R^* \neq R$ ，向 $\mathcal{F}_{\text{saPAKE}}$ 发送 $(\text{TestPwd}, \text{sid}, \text{ssid}, C, \perp)$ 。
 2. 向 $\mathcal{F}_{\text{saPAKE}}$ 发送 $(\text{NewKey}, \text{sid}, \text{ssid}, S, k^*)$ ，并将 $\text{serverSession}[\text{sid}, \text{ssid}]$ 标记为“PAKE 已完成”。

4.3.2 不可区分性证明

实验 H_0 。作为初始实验，在概率多项式时间环境 $\mathcal{Z}$ 下，敌手在客户端 C 和服务端 S 之间进行中间人攻击，客户端和服务端端的密码分别为 pw' 和 pw 。

实验 H_1 。在 H_1 中，排除随机预言机和通用群碰撞。若存在 $z \neq z'$ 使得 $H(z) = H(z')$ ，或 $A \neq A' \in \mathcal{D}$ 但被识别为同一元素，则输出 coll 并终止。假设敌手 $\mathcal{A}$ 进行 q_{RO} 次 Eval 查询和 q_{GA} 次 op 与 tw 联合查询，可得 $|\Pr[H_1 \Rightarrow 1] - \Pr[H_0 \Rightarrow 1]| \leq \Pr[\text{coll}] \leq (q_{\text{RO}}^2 + q_{\text{GA}}^2)/2|\mathbb{G}|$ ，由于 $2^\kappa \leq |\mathbb{G}| < 2^{\kappa+1}$ ，该概率在安全参数 κ 下可忽略。

实验 H_2 。在 H_2 中，若 $R^* \neq R$ 且敌手 $\mathcal{A}$ 未发送相应 $(\text{TestPwd}, \text{sid}||\text{ssid}||R, P, \cdot)$ ，那么当 $\mathcal{A}$ 发送 $(\text{NewKey}, \text{sid}||\text{ssid}||R, P, \cdot)$ 时，从 $\{0, 1\}^\kappa$ 向 P 提供一个均匀独立密钥。此时 $|\Pr[H_2 \Rightarrow 1] - \Pr[H_1 \Rightarrow 1]| = 0$ 。

实验 H_3 。在 H_3 中，测试服务器端口令。当处理 $(\text{TestPwd}, \text{sid}||\text{ssid}||R, S, [\text{rw}^*]_x)$ 且服务器端 PAKE 子会话活跃时，若敌手查询 $H(\text{pw}) = h$ 且 $\text{rw}^* = h \star R$ ，则返回“猜测成功”并标记子会话被攻破，否则返回“猜测错误”并标记子会话中断。与 H_2 相比，差异仅在于敌手未查询 $z = H(\text{pw})$ 就发送 $\text{rw}^* = H(\text{pw}) \star R$ 的情况（记为事件 GuessSrw ）。构建针对 GA-CDH 问题的敌手 $\mathcal{B}_3$ ，假设存在最多 m 个子会话，可得 $|\Pr[H_3 \Rightarrow 1] - \Pr[H_2 \Rightarrow 1]| \leq \Pr[\text{GuessSrw}] \leq m \cdot \text{Adv}_{\text{EGAT}}^{\text{GA-CDH}}(\mathcal{B}_3)$ ，该概率在安全参数 κ 下可忽略。

实验 H_4 。在 H_4 中，检测身份冒充攻击。为处理客户端 PAKE 子会话活跃时的 $(\text{TestPwd}, \text{sid}||\text{ssid}||R^*, C, [\text{rw}^*]_x)$ ，需根据服务器端情况及相关条件进行判断：若 S 被攻破且满足特定等式关系，如 $[R^*]_x = r^* + [Y]_x$ ， $[\text{rw}^*]_x = r^* + [\text{sw}]_x$ 等，若 $\text{pw}' = \text{pw}$ ，返回“猜测正确”并标记子会话被攻破；否则返回“猜测错误”并标记子会话中断。通过分析相关元素构成的群操作 Diffie-Hellman 元组，可证明 $|\Pr[H_4 \Rightarrow 1] - \Pr[H_3 \Rightarrow 1]| = 0$ 。

实验 H_5 。在 H_5 中，测试客户端口令。当处理 $(\text{TestPwd}, \text{sid}||\text{ssid}||R^*, C, [\text{rw}^*]_x)$ 且客

户端 PAKE 子会话活跃时, 若不满足 H_4 中的特定条件, 根据敌手是否查询到 $H(pw') = h$ 和 $rw^* = h \star R^*$ 返回结果。与 H_4 相比, 差异在于敌手未查询 $z = H(pw')$ 就发送 $rw^* = H(pw') \star R^*$ 的情况 (记为事件 GuessCrw)。若 $pw' \neq pw$, GuessCrw 发生概率为 $1/|\mathbb{G}|$; 若 $pw' = pw$, 构建针对 GA-DL 问题的敌手 $\mathcal{B}_4$, 可得 $|\Pr[H_5 \Rightarrow 1] - \Pr[H_4 \Rightarrow 1]| \leq \Pr[\text{GuessCrw}] \leq \max\{m \cdot \text{Adv}_{\text{EGAT}}^{\text{GA-DL}}(\mathcal{B}_4), 1/|\mathbb{G}|\}$, 该概率在安全参数 κ 下可忽略。

实验 H_6 。 H_6 用于进行离线攻击处理。服务器端口令文件通过 $(Y, sw) \leftarrow_{\S} \mathbb{G}^2$ 选取, 而不再通过计算获得 sw , 当敌手计算 $H(pw) \star Y$ 时, 将结果设置为 sw 。在概率多项式时间环境下, $|\Pr[H_6 \Rightarrow 1] - \Pr[H_5 \Rightarrow 1]| = 0$ 。

综合上述混合实验, 可得在安全参数 κ 下可忽略的概率

$$|\Pr[H_6 \Rightarrow 1] - \Pr[H_0 \Rightarrow 1]| \leq \frac{(q_{\text{RO}}^2 + q_{\text{GA}}^2)}{2|\mathbb{G}|} + m \cdot (\text{Adv}_{\text{EGAT}}^{\text{GA-CDH}}(\mathcal{B}_3) + \text{Adv}_{\text{EGAT}}^{\text{GA-DL}}(\mathcal{B}_4)).$$

同时, aR - GA - PAKE_ℓ 方案仍保持底层 PAKE 的抵抗攻击能力:

- **抵抗离线字典攻击:** R - GA - PAKE_ℓ 方案通过洗牌算法和 Merkle 根抵抗离线字典攻击。在本方案中, 登录阶段仍利用口令相关的哈希值作为种子对中间结果进行洗牌, 这使得攻击者难以从接收消息中选择有效信息, 与原方案的抵抗机制一致。强非对称 PAKE 编译器自身通过特定的存储方式防止预计算攻击, 增强了整体抵抗离线字典攻击的能力。
- **抵抗主动攻击:** R - GA - PAKE_ℓ 方案在面对主动攻击时, 通过对各种查询操作 (如 SENTINIT 、 SENDRESP 、 SENDTERM 等) 的处理来保证安全性。扩展为非对称协议后, 在主动攻击场景下, 本方案能够根据接收到的消息和已有安全机制判断消息合法性。在 SENDRESP 查询中, 接收到的洗牌后消息 $x_{\sigma_1}^U$ 和 $x_{\sigma_2}^S$ 需要通过相应种子进行反洗牌操作, 这一过程与原方案类似, 且由于编译器的存在, 不会增加攻击者伪造合法消息的可能性, 从而保持了对主动攻击的抵抗能力。

4.4 本章小结

本章针对对称 PAKE 所存在的安全性不足问题, 通过强非对称 PAKE 编译器对章节 3 中的 R-GA-PAKE_ℓ 方案进行了优化, 设计出一种基于同源的非对称 PAKE 方案 aR-GA-PAKE_ℓ, 使得服务端无需存储口令即可完成密钥协商, 并分析了方案的不可伪造性和抵抗攻击能力。本方案提出, 在 R-GA-PAKE_ℓ 方案基础上添加注册、登录阶段。在注册阶段, 客户端基于输入口令经一系列群操作和哈希运算生成口令文件发送给服务端存储; 在登录阶段, 双方分别计算 rw' 和 rw 等中间值用于替代 R-GA-PAKE_ℓ 方案中的口令 pw ; 在密钥协商阶段, 通过复杂的随机值选取、群操作、哈希计算、置换与逆置换等操作, 最终生成会话密钥。在此过程中, 双方通过特定计算生成一致的结果, 有效避免了服务端直接存储口令引发的安全隐患, 为密钥协商的安全性提供了更可靠的保障。

5 基于同源的口令系统设计与实现

本章对第四章提出的非对称 PAKE 方案 aR - GA - PAKE_ℓ 进行了算法实现，并对协议各阶段的性能进行测试分析。其次，将该 PAKE 方案集成到实际的口令系统中，增强其可用性和实用性。该系统无需在服务器端直接存储口令，提供了安全、灵活的口令认证密钥协商方式。本章的工作不仅在实验上验证了 aR - GA - PAKE_ℓ 方案的可行性，也为其在实际口令认证系统中的应用提供了有力支持。

5.1 非对称口令认证密钥协商方案实现

本节将详细介绍实验的测试环境、基本运算和主要函数的实现，以及所提出的 aR - GA - PAKE_ℓ 方案的实现框架，并对实验结果进行功能测试与性能分析。在本实验中，选取曲线模型为蒙哥马利（Montgomery）曲线模型，素数参数为 p512，公式类型为半 Velu 公式（Half-Velu Formula），指数参数为 10。

5.1.1 测试环境

本实验在一台搭载 12th Gen Intel Core i7-12700H 处理器、16GB RAM、64 位 Windows 10/11 操作系统的计算机上，基于 Anaconda 环境使用 Python 3.9 及相关密码学库进行 PAKE 方案的算法实现与性能测试。实验的基本测试环境如表5.1所示。

表 5.1 实验软硬件环境

类别	环境描述
硬件环境	
处理器	12th Gen Intel(R) Core(TM) i7-12700H @ 2.30 GHz
内存	16.0 GB RAM (15.6 GB 可用)
系统架构	64 位操作系统, 基于 x64 的处理器
软件环境	
操作系统	Windows 10/11 64-bit
Python 版本	Python 3.9
运行环境	Anaconda (Conda 虚拟环境)
依赖库	sibc ^[53] , random, hashlib, matplotlib, struct 等

5.1.2 基本运算

本实验依赖于 `sibc` 库^[53] 以支持在 CSIDH 框架下的数学基本运算。该库提供了一些基于同源的加密原语实现，可使用 Montgomery 曲线的 x-only 射影坐标进行计算，并结合了传统公式与 `velusqrt` 公式，以提高计算效率和安全性。实验中主要使用的具体运算包括：随机向量生成、群操作、仿射坐标转换等，这些运算的具体描述如表 5.2 所示。这些数学操作在 `sibc` 库的支持下，使得本实验能够准确地实现基于 CSIDH 的 PAKE 协议，并为功能测试和性能分析提供了可靠的数学基础。

表 5.2 超奇异椭圆曲线上的基本运算函数

函数名称	描述
<code>GAE_at_0</code>	计算在 0 处的广义同态类变换
<code>GAE_at_A</code>	计算在任意 A 处的广义同态类变换
<code>random_exponents</code>	生成均匀随机的指数向量
<code>affine_to_projective</code>	将仿射坐标转换为射影坐标
<code>dh</code>	进行 Diffie-Hellman (DH) 密钥交换
<code>secret_key</code>	生成私钥 (随机选取)
<code>public_key</code>	计算公钥 (基于私钥)
<code>keygen</code>	密钥生成

此外，本实验依据章节 3.2 中对函数 `GetRoot`、`Shuffle`、`DeShuffle` 的详细定义，以及章节 4.2 中对函数 `BitString` 的描述，分别对这些核心函数进行了完整的代码实现。其中，函数 `GetRoot` 主要用于对输入数据进行排序，并基于 Merkle 树结构计算其 Merkle 根，以确保数据完整性和可验证性；函数 `Shuffle` 负责执行随机置换算法，使输入数据经过安全随机化处理，以提高协议的安全性；函数 `DeShuffle` 则实现了逆置换算法，能够有效恢复置换前的数据，以保证协议的可逆性和正确性；函数 `BitString` 旨在将口令字符串转换为比特串，为后续密码学计算提供标准化的数据表示形式。

5.1.3 aR-GA-PAKE _{ℓ} 方案

本实验的代码基于章节 4 的各个阶段进行了编程实现，并通过不同测试用例和测试参数，对结果进行功能测试与性能分析。代码实现框架如图 5.1 所示。在下面的实验中，设置素数 $p = 512$ ，指数 $exp = 10$ 。

(1) 功能测试

功能测试涵盖从基本的用户操作到关键的密码学过程，包括的注册与登录、数据的随机置换与逆置换、Merkle 根的生成以及密钥协商等环节。通过测试，本实验将验证方

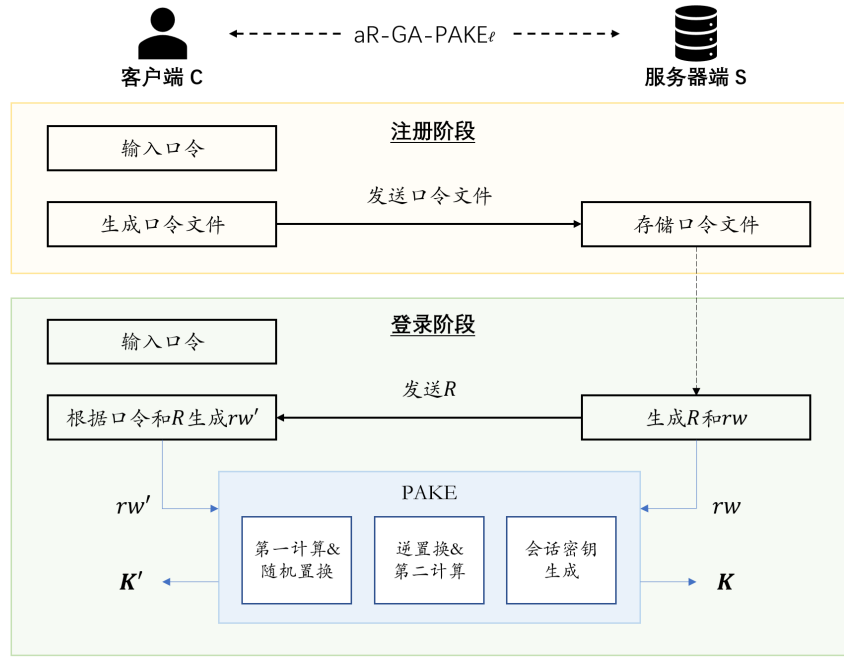


图 5.1 aR - GA - PAKE_ρ 方案的实现框架

案各项功能是否符合设计预期。

① 注册功能

如图 5.2 所示，对注册阶段进行功能测试。实验结果表明，客户端能够根据输入的口令，随机选取私有参数 y ，并正确计算生成相应的公钥 Y 、哈希值 h 以及相关的加密辅助值 sw 。随后，服务端成功接收并存储客户端发送的口令文件 (Y, sw) ，确保了注册信息的正确存储和管理。测试结果表明，注册功能运行正常。

```
#####开始注册#####
客户端输入口令pw: password
y = b'\xff\x00\x10\x0b\xf9\xe\x12\x08\xfa\x0e\xf0\t\x0e\x0e\xeb\xef\x0f\r\x03\xfd\xe9\x13\xed\r
\x06\xfc\x08\x01\x04\x00\x03\xfc\xff\xff\x0b\x01\t\xf7\xff\x08\xfe\x02\x08\x09\x00\x02\xfa\x02\x0
4\x09\xfe\xfc\x02\x02\x05\x05\x03\x01\x03\x01\xfd\xff\xfb\xfd\x00\x02\x00\x00\xfe\x02\x04\x00
\x01'
Y = b'_g\xa6z\xc1\x1b\x8eM\xfd\x92\xcf\xbd\x13\xe8\xf6\xa6\x8ac\x95\x83\i\x8f\xac\xb7\x8d+\x12g
eM\x83\xd3\xd7\xa0\x98\xdc\x1b\r\x0f\xba\xf8\xe3\x1e\t\xdf\xe5\xc5w4\x95\xa6\xab;\x84l\x09wX\xce\x
84\xca\xa8\x1c'
h = b'\r\xf4\xfe\x13\x13\x0c\xf2\x14\x02\xec\x16\x03\n\x06\x0b\x07\x0f\xe9\x03\x03\xfd\xef\x13\t
\x00\xf0\xf2\xf5\x0e\x04\x07\x04\t\xfd\xf5\xfb\x07\xf7\xff\x02\xf8\xfa\x00\x07\x04\xfe\x06\x02\x0
0\xf9\x04\x06\xfc\x00\x01\x03\xfb\x05\x05\xfb\x05\x01\xfb\xfb\x04\xfe\x00\x00\x02\xfc\x04\x02\xfe
\xfd'
sw = b'\xb6\xe0y\xd7G\xd7\x94\x06\xcc\xbf\xb23N9\x8d\xbe\x14\xe6R\xa2\x98\xec\xee\xdd%i\x1d\x14
\xc6\xf0\x1e2\x10\xaf\xb9\xcaN;Q\x1c\x81\xec\xff\xd9Q\x81\xd0\t\x11'\x99\x8bFG\xdd\xbd\x8f\xa5\t\
x8dj\xcb&'
服务端已存储(Y, sw)
#####完成注册#####
```

图 5.2 注册功能测试结果

```

#####开始登录#####
客户端输入口令pw': password
r = b'\xf1\xfe\xee\xed\x11\xf2\x06\x10\xfc\x00\xf6\x0f\xf2\xf6\x11\x17\x07\xf7\r\xf7\xe9\xeb\xeb
\xf7\xf8\xf2\n\x0b\x06\xfa\xf9\x02\xfd\xff\xff\x05\t\xfd\xff\xf8\x00\x00\xfa\x05\xf8\x04\xfa\x06\
\x06\xfd\x06\xfe\x02\x00\x03\xfd\xff\xfb\x03\xff\xff\xff\xfd\xfe\xfe\x02\xfc\xfe\x04\x00\x
02\x01'
R = b'H\n:3}\xb0\x95kLz\xaa\x0b\xba\x82\x10\x07\x8c\xc00\x97\x83\xe8\xd7\xc6)\x05\xd0\x06\xcbg+
xc6\x8d0+7E#\x94\xea\xd2\x1b\xf0\xf8\xe3I@\xd4\xf3g\x0e<\xb0\x9fE\x9e\x0b\x5\xdfi\xba\xd7%0'
rw = b'\x9c\xeb\xebZ\xf5\xee\x99\xf9_\x1b\xcfBe!DP\x8f\x0c\x86\xff\xaf\xed\t_\x1a\xa7\xa6~\xdc\x
8f\xe1\xc5W\xe3\x9e\xe8W\xecBu\xb8\x1e1\x19\xd6\xabw\xe87\x9c~b\xe70t_\xa9\xa6\xd2n/C\xc9a'
h1 = b'\r\xf4\xfe\x13\x13\x0c\xf2\x14\x02\xec\x16\x03\n\x06\x0b\x07\x0f\xe9\x03\x03\xfd\xef\x13\
t\x00\xf0\xf2\xf5\x0e\x04\x07\x04\t\xfd\xfb\x07\xf7\xff\x02\xf8\xfa\x00\x07\x04\xfe\x06\x02\x
00\xf9\x04\x06\xfc\x00\x01\x03\xfb\x05\xfb\x05\x01\xfb\xfb\x04\xfe\x00\x00\x02\xfc\x04\x02\xfc
e\xfd'
rw' = b'\x9c\xeb\xebZ\xf5\xee\x99\xf9_\x1b\xcfBe!DP\x8f\x0c\x86\xff\xaf\xed\t_\x1a\xa7\xa6~\xdc\x
8f\xe1\xc5W\xe3\x9e\xe8W\xecBu\xb8\x1e1\x19\xd6\xabw\xe87\x9c~b\xe70t_\xa9\xa6\xd2n/C\xc9a'
*rw' == rw*
#####完成登录#####
    
```

图 5.3 登录成功测试结果

② 登录功能

如图5.3所示，对登录阶段进行功能测试。实验结果表明，服务端能够随机选取 r ，计算值 R 和新口令 rw 。随后，客户端接收 R 并计算哈希值 h' 以及新口令 rw' 。根据计算，有：

$$rw = r \star sw = (r \cdot h) \star Y = (r \cdot H(pw) \cdot y) \star \tilde{x} \quad (5.1)$$

$$rw' = h' \star R = (H(pw') \cdot r) \star Y = (r \cdot H(pw') \cdot y) \star \tilde{x} \quad (5.2)$$

因此，当满足 $H(pw') = H(pw)$ 时，应有等式5.1=等式5.2，即 $H(rw') = H(rw)$ 。假设登录阶段输入的口令 $H(pw)$ 为“password”，测试登录阶段输入正确口令与错误口令的结果如图5.3及图5.4。测试结果表明，登录功能运行正常。

```

#####开始登录#####
客户端输入口令pw': abc123
r = b'\x01\x08\x02\x05\xfb\x08\xea\x0e\x12\xee\xfc\x05\x08\x00\x11\x07\xed\xfd\t\x13\x07\x01
\xed\x0c\x08\x0c\t\xfc\xfd\x0b\x04\xfb\x01\x07\xfd\x05\xfb\x02\xfe\x00\x00\x05\xf8\x00\x00\x0
6\x00\x01\x04\x06\xfe\x02\x05\xff\x05\xfb\xfd\x03\x05\xfd\x05\x01\x00\x00\xfe\xfc\x02\x04\x04\xfe
\xfc\xff'
R = b'\xb5VF\xf9\xaf\x89+A%\xe5\xbd@\x82\x93\xdf\xbb\x06\x82>B\xcd\xff\xccL\x05(\xf46\x18\xde\xfd
9\x99\x84\xbdd?\xcbF\xb5\x81\xc2!\x1f\xfc&T\x9a5\xd7\xec\xbd\xff\x32\x3\x9ew\xa4T_\xdd\xc6\x9aT'
rw = b'\x9c\x94j\xa1\xf4\xbe\xa2\x8e\x94\x7r\xca\x00\x00\x05\x1cQ\x1f\x99Y\xe7\x18\x82q\x85\x03\x
90\xd6Uh:mw\xc5.#\x1f\xa8\x84\x06D\x83\xfc\xccn\x86(\xea\xedkA\xb0h$J\xe7\x18e\xbe\x07\x1cckk:'
h1 = b'\t\x00\x12\x0b\xf3\x12\n\n\x0c\xfe\xfd\r\xfe\x12\r\x0b\t\xed\xfd\x03\xfd\xfd\r\x0f\xfd\x0
6\xfa\x05\n\xfc\xfd\x06\xfd\xfd\xfd\xfb\xfb\xfb\xfd\x02\x08\x04\x06\xfd\x09\xfe\x00\x06\x02\xfd\xfd\x
fa\x04\xfc\x00\xfb\x01\xfd\x01\xfd\x03\xfb\x05\xff\xfb\x02\xfe\xfe\x00\x00\x00\xfc\xfe\x04\xfd'
rw' = b'\x14R\xae\x86\xaf\x891e\xfd\xfbw\x00}\xcb\xfa\x00\x90\x03\x0e\xab\x05Q8_\xe5\x99\xfcscz]\
x86\x92\xbf\xfc\xfd\x85d\x01\xfbay\x06\x1bA\x9a\xeb\\\xf7\x81\x01\xfd60\x89\x95\x05\x98\x0e\x0a1\x
c0\xea\xc0x2'
*rw' != rw*
#####登录失败#####
    
```

图 5.4 登录失败测试结果

③ 随机置换与逆置换

如图5.5所示，对随机置换（Shuffle）与逆置换（DeShuffle）功能进行测试。首先，将初始数组设置为 [1, 2, 3, 4, 5, 6, 7, 8, 9]，并选取两个不同的种子值 $seed_1$ 和 $seed_2$ 进行对比测试。可以看到，使用对初始数组进行随机置换后，使用同样的种子可以将置换结果

```
#####随机置换#####
初始数组: [1, 2, 3, 4, 5, 6, 7, 8, 9]
通过seed1随机置换结果arr1: [6, 7, 8, 5, 4, 1, 9, 2, 3]
通过seed2随机置换结果arr2: [4, 7, 5, 6, 8, 3, 9, 2, 1]
通过seed1逆置换arr1结果: [1, 2, 3, 4, 5, 6, 7, 8, 9]
通过seed2逆置换arr2结果: [1, 2, 3, 4, 5, 6, 7, 8, 9]
通过seed2逆置换arr1结果: [3, 2, 1, 6, 8, 5, 7, 4, 9]
通过seed1逆置换arr2结果: [3, 2, 1, 8, 6, 4, 7, 5, 9]
#####完成逆置换#####
```

图 5.5 随机置换与逆置换测试结果

逆置换为初始数组。而若使用与置换时不同的种子进行逆置换操作，则无法还原原始数组。测试结果表明，随机置换与逆置换结果与预期一致。

④ Merkle 根生成

如图5.6所示，对 Merkle 根生成进行功能测试。设置元素相同但排列顺序不同的两个数组 $[1, 2, 3, 4, 5, 6, 7, 8, 9]$ 和 $[8, 7, 6, 2, 1, 5, 9, 4, 3]$ ，分别计算它们的 Merkle 根 $root_1$ 和 $root_2$ 。根据方案中的定义，GetRoot 会将数组按自然顺序大小排序后，再进行 Merkle 根计算。因此，应有 $root_1 = root_2$ ，与测试结果一致。

```
#####生成Merkle根#####
数组1: [1, 2, 3, 4, 5, 6, 7, 8, 9]
数组2: [8, 7, 6, 2, 1, 5, 9, 4, 3]
root1: e6f639f0b32f5602f36bdeb8540b5bdc4e922f55d079cdc6d81b20601f5a7d87
root2: e6f639f0b32f5602f36bdeb8540b5bdc4e922f55d079cdc6d81b20601f5a7d87
#####完成生成#####
```

图 5.6 Merkle 根生成测试结果

⑤ 密钥协商

最后，对密钥协商功能进行测试，该过程整合并依赖于前述多个核心功能模块。设计两组测试用例，分别代表两次输入口令相同与不同的情况。当两次输入口令相同，如图5.7所示，登录阶段 $rw' = rw$ ，后续基于该值生成的 $seed^U = cal_seed^U$ ， $seed^S = cal_seed^S$ ，因此服务器端和客户端可以恢复出对方发来的原始数据，并最终获得相同的共享密钥 $K' = K$ 。当两次输入口令不同时，如图5.8所示，登录阶段 $rw' \neq rw$ ，后续基于该值生成的 $seed^U \neq cal_seed^U$ ， $seed^S \neq cal_seed^S$ ，因此服务器端和客户端无法恢复对方发来的原始数据并计算得到相同结果，因此无法成功共享。

测试结果表明，密钥协商功能能够正确运行，并符合协议设计预期。当用户输入口令一致时，系统能够正确生成共享密钥；而当输入口令不匹配时，系统能够有效检测异常情况，防止不合法的密钥协商。

值无关，平均注册时间为 8.81s，平均登录时间为 13.97s。

对于密钥协商阶段，设置 $\ell = \{1, 2, 4, 8, 16, 32\}$ 为测试参数，在不同 ℓ 取值下密钥协商各模块的运行时间如图5.9所示，图中的模块 1、2、3 分别对应图4.1中的第一计算与随机置换模块、置换还原与第二计算模块、会话密钥生成模块。可以看到，各模块运行时间与 ℓ 取值大致呈正比例增长关系。

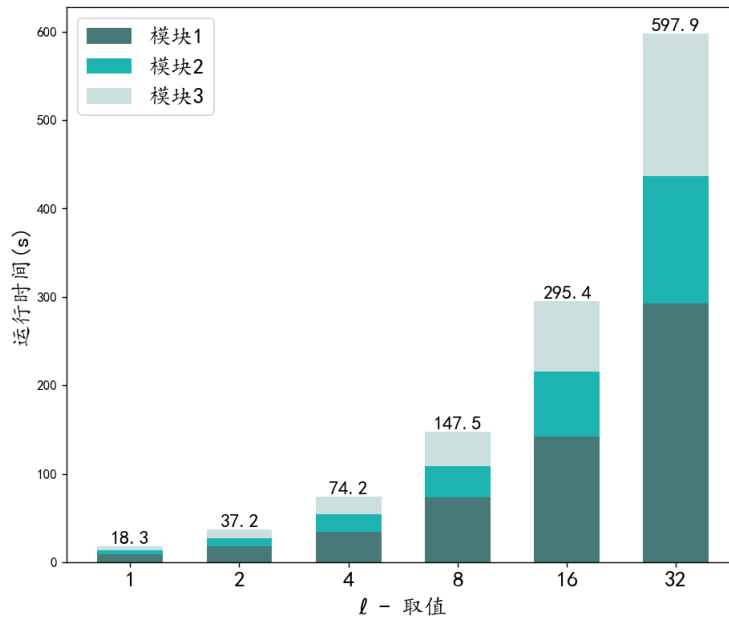


图 5.9 不同 ℓ 取值下密钥协商各模块运行时间

5.2 基于同源的口令系统模拟

本节将上述密钥协商方案集成至口令系统，并对其在系统中的各项功能进行测试与验证。该口令系统采用 C/S 架构（Client-Server），服务器端负责口令存储、身份认证及密钥协商，客户端执行用户交互逻辑，并与服务器协同计算安全会话密钥。从系统功能来看，涵盖多个核心模块，包括用户注册、账户管理、身份认证、口令更新及密钥协商等。下面对各个功能进行说明：

- 用户注册。用户注册是口令系统的起始环节，用户在此阶段创建账户并设定口令。系统会根据用户输入的口令计算相应的公钥信息，并生成相关的安全参数。随后，这些信息将安全地存储在服务器端，以备后续身份认证和密钥协商使用。
- 账户管理。账户管理功能涉及用户的基本信息维护，如修改用户名、删除账户等。
- 身份认证。身份认证是确保用户身份合法性的关键环节。在用户尝试登录时，系统会验证其输入的口令，并执行相关的认证协议。如果认证成功，用户将获得访问权限；若失败，则拒绝登录请求。
- 口令更新。口令更新允许用户在必要时更改其口令，例如定期更换口令或在检测

到安全威胁后进行紧急更新。更新过程需要进行身份验证，以确保只有合法用户可以修改口令。同时，系统可采用历史口令检查机制，防止用户重复使用旧口令，从而增强安全性。

- 密钥协商。密钥协商是该口令系统的核心功能之一，旨在为通信双方建立安全的会话密钥。系统利用基于同源的 PAKE 在不直接传输口令的情况下，确保客户端和服务端能够协同计算出相同的会话密钥。该过程结合随机数生成、群运算及哈希计算，以防止离线字典攻击，从而保证通信的机密性和完整性。

下面通过具体的案例，模拟用户对该口令系统的使用。首先，用户进入系统首页，可以看到页面如图5.10a，包含注册、登录、账户管理、口令更新、密钥协商几个功能。由于该用户是初次使用此口令系统，需要先进行注册操作。用户进入注册页面后，输入用户名“username”和口令“password”，完成注册操作，如图5.10b。在系统服务器内不会直接存放用户口令，而是通过一系列基于同源的操作计算并存储口令文件。需要注意的是，如果用户想再次通过用户名“username”进行注册，会显示“用户名已存在”。接着，用户可以用注册的用户名及口令进行登录，如图5.10c。如果客户端和服务端使用该口令和存储的口令文件能生成相同的计算结果，则登录成功，否则无法登录。在已登录的情况下，用户可以进行密钥协商，生成共享密钥，如图5.10d。若用户想要修改口令，可以进入口令更新页面，输入用户名、旧口令、新口令，从而完成更新，如图5.10e。如果输入未注册的用户名或用户名和旧口令不匹配，则无法进行口令更新。此外，用户可以通过账户管理页面，进行用户名的修改以及账户的删除操作，如图5.10f。



图 5.10 口令系统测试页面结果

5.3 本章小结

本章围绕基于同源的口令系统设计与实现，详细探讨了非对称口令认证密钥协商（aR-GA-PAKE ℓ ）方案的具体实现过程。首先，介绍了实验的测试环境和基本运算，确保方案能够在合理的计算资源下高效执行。接着，详细描述了 aR-GA-PAKE ℓ 方案的功能测试与性能测试，验证其正确性及计算效率。最后，结合基于同源的口令系统模拟，进一步评估了该方案在实际应用中的可行性与适用性。

6 总结与展望

6.1 本文工作总结

随着信息技术的飞速发展，网络环境中的身份认证与安全通信需求日益增长。传统的口令认证密钥协商（PAKE）方案虽然在一定程度上能够提供安全保障，但仍然面临计算开销较高、易受离线字典攻击以及服务器口令存储风险等问题。此外，随着量子计算技术的不断进步，部分经典的基于离散对数和整数因子分解问题的密码学算法逐渐暴露出潜在的安全性威胁，因此研究更高效、更安全的 PAKE 方案显得尤为重要。

针对上述挑战，本文围绕基于同源的 PAKE 技术展开研究，提出了一种基于随机同源置换的 PAKE 方案，并进一步通过强非对称 PAKE 编译器优化为非对称 PAKE 方案，使得服务器无需存储口令即可完成密钥协商。在此基础上，本文从安全模型、协议设计、安全性分析、性能评估等多个方面进行了深入研究，并取得了如下研究成果：

- 提出了一种基于随机同源置换的口令认证密钥协商方案，在不增加通信开销的前提下，通过随机置换算法对同源计算的中间结果进行转换，使其成为完全不同的随机序列，从而有效抵御离线字典攻击。在该方案中，用户和服务器在共享口令和两个随机元素的情况下，首先各自生成随机元素，并利用群操作计算得到相应的序列。由于直接传输这些序列可能会暴露关键信息，进而被攻击者利用进行离线字典攻击，因此本方案采用随机置换机制，将原始序列转换为新的随机序列，并将其发送给对方。服务器和用户在接收对方的序列后，通过还原操作恢复出正确的计算结果，最终各自计算出会话密钥，完成安全的密钥协商。此外，为了确保每次协商过程中生成的置换序列完全不同，本方案基于口令和 Merkle 根哈希动态生成置换种子，从而提升安全性。实验和理论分析表明，该方法在有效抵御离线字典攻击的同时，仍能保持通信开销为 $O(2\ell)$ ，实现了安全性和高效性的良好平衡。
- 针对对称 PAKE 方案存在的安全性不足问题，利用强非对称 PAKE 编译器对提出的第一个方案进行了优化，提出了一种基于同源的非对称 PAKE 方案。该方案无需服务端存储口令即可完成密钥协商，从而有效降低服务器存储泄露带来的安全风险。为实现这一优化，本方案增加了注册和登录阶段：在注册阶段，客户端基于用户输入的口令，经过一系列群操作和哈希计算生成口令文件，并将其发送给服务器存储。该口令文件替代了传统的明文口令存储方式，从而消除了因服务器泄露导致的直接攻击风险。在登录阶段，用户与服务器分别计算结果相同的中间值，以替代原始方案中的口令，确保协议执行过程中不会直接暴露口令信息。在密钥协商阶段，通过随机值选取、群操作、哈希计算、置换与逆置换等计算，最终生成安全的一致会话密钥。在整个过程中，本方案利用随机化机制和特定计算流

程，确保用户和服务端在密钥协商过程中始终得到相同的最终结果，同时避免服务器存储口令所带来的安全隐患。

- 在实验环境中实现并评估方案的实际性能。本文基于典型的安全通信场景，对所提出的 PAKE 方案进行了实现，并基于功能测试和性能测试验证在实验环境下测试其正确性和计算性能。为了进一步验证方案的可行性，本文结合基于同源的口令系统模拟，测试方案在实际部署中的表现。实验结果表明，本文方案在实际应用中具有一定的可行性与适用性。

6.2 未来工作展望

尽管本文提出的基于同源的 PAKE 方案在安全性和效率方面取得了一定突破，但仍有多方向值得进一步探索和优化。未来的研究可以在以下几个方面进行深入探讨：

- 抗量子安全性：本文的安全性证明基于所定义的安全模型展开。未来可以研究如何在量子随机预言机（QROM）下证明本文方案的安全性，并探索结合格密码或其他抗量子密码技术，设计更安全的 PAKE 方案。
- 更广泛的应用场景：当前方案主要针对标准的 PAKE 场景，未来可以进一步研究其在分布式系统、云计算、物联网（IoT）等复杂环境中的适用性和优化方案。例如，探索如何在资源受限的设备（如 IoT 传感器）上高效执行 PAKE 协议，提升其计算和存储效率。
- 更强的安全保证：研究如何在更严格的安全模型（如可证明的 UC 安全性）下进一步提升方案的安全性，确保方案能够抵御更强的攻击威胁。
- 协议优化：目前提出的 PAKE 方案虽然对初始方案进行了效率优化，但仍有很大优化空间。未来可以进一步减少协议的计算和通信开销，以适应资源受限设备或高并发环境，提升方案在实际应用中的可行性。此外，可以使用 GPU 或 FPGA 加速，提高群操作的运算效率。

综上所述，本文提出的基于随机同源置换的 PAKE 方案为口令认证密钥协商技术提供了一种新的思路。未来，随着抗量子计算技术的发展、密码学安全模型的进一步完善以及协议优化的持续推进，基于同源的 PAKE 方案有望在更多应用场景中发挥重要作用，为安全通信和身份认证提供 stronger 的保障。

参考文献

- [1] Bellare S M, Merritt M. Encrypted key exchange: Password-based protocols secure against dictionary attacks[J]. ACM Transactions on Computer Systems (TOCS), 1992, 10(4): 421 – 431.
- [2] Bellare M, Pointcheval D, Rogaway P. Authenticated key exchange secure against dictionary attacks[A]. International conference on the theory and applications of cryptographic techniques[C], 2000: 139 – 155.
- [3] Jablon D P. Strong password-only authenticated key exchange[J]. ACM SIGCOMM Computer Communication Review, 1996, 26(5): 5 – 26.
- [4] Haase B, Labrique B. AuCPace: Efficient verifier-based PAKE protocol tailored for the IIoT[J]. Cryptology ePrint Archive, 2018.
- [5] Jarecki S, Krawczyk H, Xu J. OPAQUE: an asymmetric PAKE protocol secure against pre-computation attacks[A]. Advances in Cryptology–EUROCRYPT 2018: 37th Annual International Conference on the Theory and Applications of Cryptographic Techniques, Tel Aviv, Israel, April 29-May 3, 2018 Proceedings, Part III 37[C], 2018: 456 – 486.
- [6] Abdalla M, Pointcheval D. Simple password-based encrypted key exchange protocols[A]. Topics in Cryptology–CT-RSA 2005: The Cryptographers’ Track at the RSA Conference 2005, San Francisco, CA, USA, February 14-18, 2005. Proceedings[C], 2005: 191 – 208.
- [7] Hao F, Ryan P Y. Password authenticated key exchange by juggling[A]. Security Protocols XVI: 16th International Workshop, Cambridge, UK, April 16-18, 2008. Revised Selected Papers 16[C], 2011: 159 – 171.
- [8] 李文敏, 温巧燕, 张华. 基于验证元的三方口令认证密钥交换协议 [J]. 通信学报, 2008, 10.
- [9] 隋爱芬, 杨义先, 钮心忻, et al. 基于椭圆曲线密码的可认证密钥协商协议的研究 [J]. 北京邮电大学学报, 2004, 27(3): 28.
- [10] 王晓峰, 张璟, 王尚平, et al. 基于口令认证的移动 Ad Hoc 网密钥协商方案 [D]. [S.l.]: Citeseer, 2006.
- [11] Hao F, van Oorschot P C. SoK: Password-Authenticated Key Exchange–Theory, Practice, Standardization and Real-World Lessons[A]. Proceedings of the 2022 ACM on Asia Conference on Computer and Communications Security[C], 2022: 697 – 711.
- [12] Bellare S M, Merritt M. Augmented Encrypted Key Exchange: A Password-Based Protocol Secure Against Dictionary Attacks and Password File Compromise[A/OL]. Proceed-

- ings of the 1st ACM Conference on Computer and Communications Security (CCS)[C], New York, NY, USA : ACM, 1993 : 244 – 250 [1993]. <https://doi.org/10.1145/168588.168618>.
- [13] Menezes A. Another look at HMQV[J]. Journal of Mathematical Cryptology, 2007, 1(1) : 47 – 64.
- [14] Kudla C, Paterson K G. Modular security proofs for key agreement protocols[A]. International conference on the theory and application of cryptology and information security[C], 2005 : 549 – 565.
- [15] Zhao Z, Dong Z, Wang Y. Security analysis of a password-based authentication protocol proposed to IEEE 1363[J]. Theoretical Computer Science, 2006, 352(1-3) : 280 – 287.
- [16] Zhang M. Analysis of the SPEKE password-authenticated key exchange protocol[J]. IEEE Communications Letters, 2004, 8(1) : 63 – 65.
- [17] Hao F, Shahandashti S F. The SPEKE protocol revisited[A]. International Conference on Research in Security Standardisation[C], 2014 : 26 – 38.
- [18] Hao F, Metere R, Shahandashti S F, et al. Analyzing and patching SPEKE in ISO/IEC[J]. IEEE Transactions on Information Forensics and Security, 2018, 13(11) : 2844 – 2855.
- [19] Jablon D P. Extended password key exchange protocols immune to dictionary attack[A]. Proceedings of ieee 6th workshop on enabling technologies: Infrastructure for collaborative enterprises[C], 1997 : 248 – 255.
- [20] Mackenzie P. More efficient password-authenticated key exchange[A]. Topics in Cryptology—CT-RSA 2001: The Cryptographers’ Track at RSA Conference 2001 San Francisco, CA, USA, April 8–12, 2001 Proceedings[C], 2001 : 361 – 377.
- [21] Katz J, Ostrovsky R, Yung M. Efficient password-authenticated key exchange using human-memorable passwords[A]. International Conference on the Theory and Applications of Cryptographic Techniques[C], 2001 : 475 – 494.
- [22] Kobara K, Imai H. Pretty-simple password-authenticated key-exchange under standard assumptions[J]. Cryptology ePrint Archive, 2003.
- [23] Jiang S, Gong G. Password based key exchange with mutual authentication[A]. International Workshop on Selected Areas in Cryptography[C], 2004 : 267 – 279.
- [24] Smyshlyaev S, Alekseev E, Oshkin I, et al. The security evaluated standardized password-authenticated key exchange (SESPAKEYE) protocol[R]. 2017.
- [25] Pointcheval D, Wang G. VTBPEKE: Verifier-based two-basis password exponential key exchange[A]. Proceedings of the 2017 ACM on Asia Conference on Computer and Communications Security[C], 2017 : 301 – 312.
- [26] Hao F, Zieliński P. A 2-round anonymous veto protocol[A]. International Workshop on

- Security Protocols[C], 2006 : 202 – 211.
- [27] Lancrenon J, Škrobot M, Tang Q. Two more efficient variants of the J-PAKE protocol[A]. Applied Cryptography and Network Security: 14th International Conference, ACNS 2016, Guildford, UK, June 19-22, 2016. Proceedings 14[C], 2016 : 58 – 76.
- [28] Wu T D, others. The Secure Remote Password Protocol.[A]. NDSS : Vol 98[C], 1998 : 97 – 111.
- [29] Kwon T. Authentication and key agreement via memorable password[J]. Cryptology ePrint Archive, 2000.
- [30] Shin S, Kobara K. Efficient augmented password-only authentication and key exchange for IKEv2[R]. 2012.
- [31] Childs A, Jao D, Soukharev V. Constructing elliptic curve isogenies in quantum subexponential time[J]. Journal of Mathematical Cryptology, 2014, 8(1): 1 – 29.
- [32] Castryck W, Decru T. An efficient key recovery attack on SIDH[A]. Annual International Conference on the Theory and Applications of Cryptographic Techniques[C], 2023 : 423 – 447.
- [33] Castryck W, Lange T, Martindale C, et al. CSIDH: an efficient post-quantum commutative group action[A]. Advances in Cryptology–ASIACRYPT 2018: 24th International Conference on the Theory and Application of Cryptology and Information Security, Brisbane, QLD, Australia, December 2–6, 2018, Proceedings, Part III 24[C], 2018 : 395 – 427.
- [34] Couveignes J-M. Hard homogeneous spaces[J]. Cryptology ePrint Archive, 2006.
- [35] Rostovtsev A, Stolbunov A. Public-key cryptosystem based on isogenies[J]. Cryptology ePrint Archive, 2006.
- [36] Abdalla M, Eisenhofer T, Kiltz E, et al. Password-authenticated key exchange from group actions[A]. Annual International Cryptology Conference[C], 2022 : 699 – 728.
- [37] Ishibashi R, Yoneyama K. Compact Password Authenticated Key Exchange from Group Actions[A]. Australasian Conference on Information Security and Privacy[C], 2023 : 220 – 247.
- [38] Bellare S M, Merritt M. Augmented encrypted key exchange: A password-based protocol secure against dictionary attacks and password file compromise[A]. Proceedings of the 1st ACM Conference on Computer and Communications Security[C], 1993 : 244 – 250.
- [39] Canetti R, Dachman-Soled D, Vaikuntanathan V, et al. Efficient password authenticated key exchange via oblivious transfer[A]. Public Key Cryptography–PKC 2012: 15th International Conference on Practice and Theory in Public Key Cryptography, Darmstadt, Germany, May 21-23, 2012. Proceedings 15[C], 2012 : 449 – 466.
- [40] Bradley T, Jarecki S, Xu J. Strong asymmetric PAKE based on trapdoor CKEM[A]. Ad-

- vances in Cryptology—CRYPTO 2019: 39th Annual International Cryptology Conference, Santa Barbara, CA, USA, August 18–22, 2019, Proceedings, Part III 39[C], 2019 : 798 – 825.
- [41] Cremers C, Naor M, Paz S, et al. CHIP and CRISP: protecting all parties against compromise through identity-binding PAKEs[A]. Annual International Cryptology Conference[C], 2022 : 668 – 698.
- [42] McQuoid I, Xu J. An efficient strong asymmetric PAKE compiler instantiable from group actions[A]. International Conference on the Theory and Application of Cryptology and Information Security[C], 2023 : 176 – 207.
- [43] Galbraith S D. Mathematics of public key cryptography[M]. [S.l.] : Cambridge University Press, 2012.
- [44] Jao D, De Feo L. Towards quantum-resistant cryptosystems from supersingular elliptic curve isogenies[A]. Post-Quantum Cryptography: 4th International Workshop, PQCrypto 2011, Taipei, Taiwan, November 29–December 2, 2011. Proceedings 4[C], 2011 : 19 – 34.
- [45] Bellare M, Rogaway P. Random oracles are practical: A paradigm for designing efficient protocols[A]. Proceedings of the 1st ACM Conference on Computer and Communications Security[C], 1993 : 62 – 73.
- [46] Fiat A, Shamir A. How to prove yourself: Practical solutions to identification and signature problems[A]. Conference on the theory and application of cryptographic techniques[C], 1986 : 186 – 194.
- [47] Diffie W, Hellman M E. New directions in cryptography[G] // Democratizing Cryptography: The Work of Whitfield Diffie and Martin Hellman. 2022 : 365 – 390.
- [48] Knuth D E. The art of computer programming : Vol 3[M]. [S.l.] : Pearson Education, 1997.
- [49] Fisher R A, Yates F. Statistical tables for biological, agricultural and medical research[M]. [S.l.] : Hafner Publishing Company, 1953.
- [50] Alamati N, De Feo L, Montgomery H, et al. Cryptographic group actions and applications[A]. Advances in Cryptology—ASIACRYPT 2020: 26th International Conference on the Theory and Application of Cryptology and Information Security, Daejeon, South Korea, December 7–11, 2020, Proceedings, Part II 26[C], 2020 : 411 – 439.
- [51] Abdalla M, Fouque P-A, Pointcheval D. Password-based authenticated key exchange in the three-party setting[A]. Public Key Cryptography-PKC 2005: 8th International Workshop on Theory and Practice in Public Key Cryptography, Les Diablerets, Switzerland, January 23-26, 2005. Proceedings 8[C], 2005 : 65 – 84.
- [52] Groce A, Katz J. A new framework for efficient password-based authenticated key exchange[A]. Proceedings of the 17th ACM conference on Computer and communications

security[C], 2010: 516–525.

- [53] Rodríguez-Henríquez F. SIBC: A Python-3 library for designing and implementing efficient isogeny-based protocols[J], 2021.

攻硕期间取得的学术成果和参与的项目

- 发表的论文:

1. XXX, XX, XXX, XXX, XX. XXXXXXXXXXXXXXXX[C]//International Conference on Information Security Practice and Experience. Singapore: Springer Nature Singapore, 2024. (CACR-C, 第一作者, 对应论文第三章内容)

- 参与的项目:

1. XXX

- 发表的软著:

1. XXXXXXXXXXXXXXXXXXXXXXXX V1.0 2023SR0460267, 2023-04-11。